

Event Stream Design Recommendation — Analysis & Brainstorm

Date: 2026-08-18

Status: Design exploration — no code changes

1. Current State: What Works

The real event stream (`event-stream.js` + `style.css` + `index.html`) has solid bones:

- **Universal log:** 234 callsites across 27 files feed it. It is the single source of truth for everything that happens.
- **Live streaming feedback:** `startStreaming` / `appendStream` / `finishStreaming` gives "it's alive" token-by-token output.
- **Filtering infrastructure:** Checkboxes for thoughts/speech/actions/system/raw-LLM + actor dropdown + tick toggle. All wired to config persistence.
- **Theming:** Dark / Disco Elysium / Linear / Terminal — already plumbed through `data-style`.
- **State backbone:** `_characterState` and `_areaEventLog` are consumed by prompts and inspectors. The visual bubbles are a *projection* of state.
- **Persistence:** IndexedDB restore of last 500 bubbles survives refresh.
- **Actor names prepended:** Character names appear at the start of bubbles (`jake halloway think observing` , LLM → `qwen3.5-2b...`), so you can always see who did what.
- **Turn-based execution:** Agents act one at a time in sequence, so the stream is naturally readable without simultaneity concerns.

2. Current State: What Hurts

2.1 Flat density — no visual hierarchy

Every event is a single `msg-bubble` row with `[Tick X | HH:MM] icon actor text`. In a long session, the stream becomes an undifferentiated wall. You cannot quickly distinguish thoughts from actions from results.

2.2 Actor filter is broken

The actor dropdown (`#stream-agent_filter`) exists in the HTML and has an `onchange` handler, but `updateAgentFilterDropdown()` is **never called anywhere**. The dropdown stays empty. This is a dead feature that needs to be wired up.

2.3 No area-based filtering

`_areaEventLog` is populated and consumed by the inspector's area view (`inspector/area-view.js:68`), but the event stream has no way to filter or highlight events by area. If you click an area in the inspector, the stream keeps showing everything.

2.4 LLM raw output order is wrong

In the real export, the raw LLM request/response appears *before* the resolved action/result:

```
🔥 LLM request (system prompt)
📄 LLM response (JSON action)
📺 examine Panel 17 ← actual outcome
```

The LLM bloat itself is **not the problem** — you have the `filterRawLLM` checkbox and can toggle it off. The problem is that when raw LLM is shown, the ordering buries the outcome. You want to see what happened first, then how the agent decided it.

2.5 Narration is invisible

`narration-ui.js` substitutes AI/player narration for area descriptions and action results, but the stream shows no indicator. You see the result text but not that it was *narrated* vs raw engine output.

2.6 Memory/reflection events are not styled

🧠 `reflected on N memories` exists in the category map but has no dedicated `.msg-reflection` style. It falls through as a generic system message.

2.7 Phase markers are visible but subtle

`logPhase` emits `observe`, `think`, `decide`, `act`, `react` as emoji-prefixed bubbles. They appear in the stream but blend in with other rows. They could be more visually distinct (background tint, separator line, etc.).

2.8 World results could be clearer

Action results are rendered as plain text in `.msg-result`. There's no visual distinction between:

- A successful action ("You pick up the matches.")
- A failed skill check ("You don't notice anything else.")
- A system constraint ("You can't do that while grappled.")
- A trigger effect (the door speaks to you)

These are all the same color and weight in the stream.

2.9 No compact/dense mode toggle

The current stream is fixed-density. Turn cards exist in the code (`turn-card` classes in `style.css`) but are not actively used in the flat bubble mode. There's no toggle between a compact single-line mode and a expanded card mode.

2.10 XSS risk

`logThought` and `_addBubble` interpolate LLM text into `innerHTML` without escaping. Only `rawLLM` paths escape. This is a security hole.

3. Demo vs. Real: Where They Diverge

Aspect	Demo (event-stream-demo.html)	Real (event-stream.js)
LLM payload	Compact 3-line mockups	200-500 line system prompts repeated per phase
Grouping	Turn cards with semantic rows	Flat bubbles, no grouping
Actor identity	Color-coded borders + names	Monochrome, same font for all
Phase visibility	Not modeled (demo is post-hoc)	Visible but subtle emoji prefixes
Skill checks	Beautiful <code>.skill-badge</code> chips	String-matched, rendered inline
Flat events	Separate section, clearly labeled	Mixed into the same flat stream
Narration	Not present	Invisible substitution
Memory events	Not present	<code>reflected on N memories</code>
Area filtering	Not present	<code>_areaEventLog</code> exists but not exposed to stream
Actor filter	Not present	Dropdown exists but is broken (never populated)

The demo solves the *narrative* problem well. The real stream has more data but less structure.

4. What's Missing / What to Add

4.1 Area filter — click area, see only that area's events

This is the #1 requested feature. The data already exists in `_areaEventLog`. The inspector already shows it. The stream just needs a filter mode:

- Add an area dropdown/filter bar (or make the stream react to inspector selection)
- When area "foyer" is selected, only show events where `data-area="foyer"` or where the actor is currently in "foyer"
- Highlight area-change events (entering/leaving) so you can see transitions

UI pattern: When you click an area in the inspector, the stream gets a "scoped to: foyer" banner and filters. Click "clear" to show all again.

4.2 Fix the actor filter

`updateAgentFilterDropdown()` exists but is **never called**. Wire it up:

- Call it after every new actor is seen (in `_addBubble`, `logPhase`, etc.)
- Call it on stream restore
- The dropdown then actually works for filtering by character

4.3 Reorder: outcome before LLM detail

When raw LLM is visible, change the order within a turn:

```
jake halloway think observing
examine matches
↳ You pick up a small box of matches...
LLM → qwen3.5-2b-polaris-highiq-instruct-i1 [collapsed]
LLM Response [collapsed]
```

This way the *result* is always visible first. The LLM detail is secondary context, not the primary content.

4.4 More filters

Add filter toggles for:

- **Area** (dropdown, "all areas" default)
- **Result type** (success / failure / neutral — inferred from text or structured data)
- **Narration** (show/hide AI-narrated events separately)
- **Phase** (show/hide observe/think/decide/act/react markers)
- **Skill checks** (show/hide skill badges separately from actions)

Keep the existing ones (thoughts, speech, actions, system, raw-LLM, tick, actor).

4.5 Clearer world results

Style action results differently based on outcome:

- **Success**: green left-border or subtle green tint
- **Failure**: red left-border or subtle red tint
- **Neutral/system**: no tint, default text
- **Trigger/effect**: purple left-border (existing `.msg-trigger` style)

Add a result type badge: ✓ success, ✗ failure, ⚠ partial, ⓘ info — small, inline, before the result text.

4.6 Narration indicator

When `narration-ui.js` substitutes narration, emit a small badge:

```
[AI Narration] The oak door groans as it opens...
```

Or add a `data-narrated="true"` attribute and style it with a subtle gold left-border.

4.7 Memory/reflection styling

Add `.msg-reflection`:

```
jake halloway reflected on 3 memories (tags: fear, escape)
```

Style it distinct from system messages — maybe purple tint, similar to `.msg-trigger`.

4.8 Phase marker styling

Make `logPhase` visually distinct even in flat mode:

- Background tint (subtle, matching phase color)
- Phase label in a small badge/pill shape
- Left border color matching phase (observe=blue, think=purple, decide=orange, act=green, react=teal)

4.9 Compact mode toggle

A toggle between:

- **Card mode** (demo style): turn cards with semantic rows, more vertical space, easier to read
- **Compact mode** (current style): single-line bubbles, max density, fast scanning

Default to compact (current behavior) since that's what the user is used to.

4.10 XSS fix

Escape LLM-derived text in `logThought` + `_addBubble` via existing `_escapeHtml()`. Must-do regardless of other changes.

5. What's in the Wrong Order

5.1 LLM raw output appears before the resolved action

In the real export:

```
🔥 LLM request (system prompt)
🔥 LLM response (JSON action)
🔵 examine matches
↳ You pick up the matches.
```

Fix: Reorder so action + result appear first, then LLM detail:

```
🔵 examine matches
↳ You pick up the matches.
🔥 LLM • think-decide [expandable]
🔥 LLM Response [expandable]
```

5.2 Turn cards close on system messages

In `_closeTurnCard()`, any non-agent event immediately closes the current turn card. A ⚠️ `replanned` system message mid-turn breaks card grouping. System messages that fire during an agent's turn should stay *inside* the turn card, not close it.

6. Brainstorming: Ideas to Consider

6.1 Result significance levels

Not all results are equal:

- **Critical:** death, scenario end, condition change, combat hit
- **Normal:** picked up item, moved to room, opened door
- **Minor:** "You don't notice anything else", "Nothing unusual happened"

Style them with different weights or left-border colors. Critical results get a thicker border or bold text.

6.2 Event provenance tags

When a trigger fires, show the source node:

```
⚡ [front_door] The massive oak front door groans...
```

Clickable → opens inspector on that node. This bridges the stream to the graph editor.

6.3 Stream scope indicator

When the stream is filtered by area or actor, show a persistent banner:

```
📌 Scoped to: foyer (jake halloway) [x clear]
```

So you never forget you're in a filtered view.

6.4 Action verb highlighting

Color the action verb differently from the target:

```
🔵 examine matches
```

`examine` in accent color, `matches` in default text. Makes actions scannable at a glance.

6.5 Tick dividers for major events

Keep the existing tick markers but add *area transition* dividers:

```
— foyer → library —
```

When an agent moves between areas, show a transition line. This makes spatial movement visible in the stream.

7. Recommended Implementation Order

Phase 1: Fix broken things + reorder (quick wins)

1. **Fix actor filter** — call `updateAgentFilterDropdown()` so the dropdown populates
2. **XSS fix** — escape `logThought` + `_addBubble`
3. **Reorder**: action/result before LLM detail within a turn
4. **Fix turn card closing** — system messages during a turn shouldn't close the card

Phase 2: Filtering + clarity (medium effort)

5. **Area filter** — dropdown + scoped view, uses existing `_areaEventLog`
6. **More filters** — narration, phase, skill checks, result type
7. **Clearer world results** — success/failure styling, result badges
8. **Narration indicator** — badge or border for AI/player-narrated events

Phase 3: Visual polish

9. **Memory/reflection styling** — `.msg-reflection` distinct from system
10. **Phase marker styling** — tinted background, pill badges, color-coded borders
11. **Action verb highlighting** — accent color on verb, default on target
12. **Compact mode toggle** — card vs dense mode

Phase 4: Nice-to-haves

13. **Stream scope indicator** — banner when filtered
14. **Area transition dividers** — show movement between rooms
15. **Event provenance tags** — clickable node IDs on triggers
16. **Result significance levels** — critical/normal/minor visual weight

8. Decisions Needed

#	Decision	Options
1	LLM ordering : reorder so action/result comes before raw LLM?	Yes / No
2	Turn cards : keep current flat mode, add card mode toggle, or switch default?	Flat-only / Toggle / Card-default
3	Area filter : dropdown in stream header, or auto-scope when area clicked in inspector?	Dropdown / Inspector-linked / Both
4	Result styling : color-code by success/failure/neutral?	Yes / No
5	Narration : inline badge or separate filter toggle?	Badge / Filter / Both
6	Phase markers : subtle styling overhaul, or keep as-is?	Overhaul / Keep

9. What the Demo Does Well (to preserve)

- Turn card layout with actor color border
- Semantic message rows (thought, speech, action, result, trigger, heard, env)
- Skill badges as visual chips
- LLM request/response chips with expand
- Tick dividers for temporal structure
- Flat non-agent events (user commands, graph edits, saves) separated from agent turns
- Hover states and clickable actor names
- Density mode concept (cards vs compact)

10. Risk: The Stream is State, Not Just UI

Any redesign must keep these intact:

- `events.trackAction(charName, inner, speech, action, result)` → populates `_characterState.actionHistory`
- `events.logThought(charName, thought)` → populates `_characterState.lastThought`

- `events.logAreaEvent(area, charName, action, result)` → populates `_areaEventLog`
- `events.trackPhase(charName, phase, data)` → populates `detailedTimeline`
- `events.emit('log', {text, className})` → pub/sub subscribers exist
- `events.restoreLog()` / `_persistLog()` → IndexedDB round-trip

The visual redesign is a *projection layer* on top of this state. We can re-skin the DOM, add metadata attributes, and change ordering, but the state-population methods must keep working exactly as they do.

11. Verified Facts from Code + Logs

- **Actor filter dropdown:** defined in HTML (`#stream-agent_filter`), `updateAgentFilterDropdown()` exists in `event-stream.js:111` but is **never called** — dropdown stays empty.
- **Area event log:** `_areaEventLog` populated by `trackAction` → `logAreaEvent`, consumed by `inspector/area-view.js:68` and `prompt-builder/room-context.js:396`. Not exposed to stream.
- **Raw LLM filter:** `config.filterRawLLM` checkbox exists and is wired to `applyFilters()`. LLM bloat is toggleable.
- **Turn execution:** agents run sequentially (one at a time), not simultaneously — confirmed from `agent-engine.js` turn queue logic and user feedback.
- **Real log format:** `data/exports/Untitled-1.txt` and `data/scenarios/logs/event_stream_from_18082026.txt` show the actual stream output — flat bubbles, actor names prepended, raw LLM dumps inline.

End of design recommendation.

Front-end module index

Generated by `tools/js_module_index.py` — do not hand-edit.

Each row is the module's own `@module` / `@contributes` / `@powers` contract.

- Modules scanned: **133**
- Documented: **133**
- Awaiting a contract header: **0**

Documented modules

Module	Purpose	File	Contributes
<code>agent/action-normalizer</code>	LLM action → backend command translation	<code>static/js/agent/action-normalizer.js</code>	ActionNormalizer: field→command-string mapping plus verb valid
<code>agent/agent-state</code>	turn eligibility + abort machinery	<code>static/js/agent/agent-state.js</code>	AgentState: can-act checks (busy/unconscious/resting/simple_npc flags)
<code>agent/emote-picker</code>	reusable emote quick-pick	<code>static/js/agent/emote-picker.js</code>	EmotePicker.open/close/toggle + setCatalog (searchable chip grid)
<code>agent/human-turn-composer</code>	the panel you play a character from	<code>static/js/agent/human-turn-composer.js</code>	HumanTurnComposer: scene view, feed/digest, You strip, compose (do/say/emote/memory), phases
<code>agent/involuntary</code>	involuntary speech/emote flavour	<code>static/js/agent/involuntary.js</code>	Involuntary: hiccup/burp/yelp/stutter injection from conditions + baseline
<code>agent/memory-manager</code>	the agent memory write path	<code>static/js/agent/memory-manager.js</code>	AgentMemory.storeMemory() + reflect() (LLM summarization into
<code>agent/object-responder</code>	browser-side lines for <code>llm_respond</code>	<code>static/js/agent/object-responder.js</code>	ObjectResponder: watch <code>llm_pending</code> → generate a line via <code>llmClient/api/llm_respond</code>
<code>agent/plan-manager</code>	LLM plan generation	<code>static/js/agent/plan-manager.js</code>	PlanManager.generate() (multi-step plan via the LLM) + hasPlan()
<code>agent/plan-tracker</code>	plan state ownership	<code>static/js/agent/plan-tracker.js</code>	PlanTracker: plan array, step progress, failure map, replan eligibility

Module	Purpose	File	Contributes
prompt-builder/character-state	the "=== YOUR STATE ===" builders	static/js/agent/prompt-builder/character-state.js	emotion/relationship/insanity/encumbrance/trait/size/perceived/pdescribeVital(s)
prompt-builder/context-sections	the "=== YOUR STATE ===" fragment registry	static/js/agent/prompt-builder/context-sections.js	CONTEXT_SECTIONS (vitals, emotion, insanity, trait, size, ghost/debuildContextBlock(), buildPlanGuide()
prompt-builder/contextual-actions	the "=== AVAILABLE ACTIONS ===" block	static/js/agent/prompt-builder/contextual-actions.js	computeItemActions, buildAvailableActionsBlock, carriedItemNotd gating helpers
prompt-builder/conversation-context	speech salience + conversation instinct	static/js/agent/prompt-builder/conversation-context.js	classifySpeechType/salienceLabel/markSpeechLine, ownRecentSpebuildConversationInstinct
prompt-builder/helpers	leaf prompt utilities	static/js/agent/prompt-builder/helpers.js	lightToLevel, wayHandle, buildRelationMap, anonymousName, voisecondPersonDesc, frameSelfSpeech
agent/prompt-builder	the LLM prompt assembly subsystem	static/js/agent/prompt-builder/index.js	window.PromptBuilder: room/state/memory fragments, JSON sche prompts
prompt-builder/memory-context	memory retrieval + investigation notes	static/js/agent/prompt-builder/memory-context.js	buildMemoryContext: async recall scoring, keyword/semantic/rece notes
prompt-builder/room-context	the area/room context assembler	static/js/agent/prompt-builder/room-context.js	buildRoomContext/Parts, buildCharacterPreamble, viewerExits, buildNarratedRoomContext
prompt-builder/schema-fragments	shared prompt text + JSON schema fragments	static/js/agent/prompt-builder/schema-fragments.js	EMOTE_RULES_ / MEMORY_INSTRUCTION_ constants, JSON_FIELD
prompt-builder/system-prompt	the static character system prompt	static/js/agent/prompt-builder/system-prompt.js	buildCharacterSystemPrompt / buildReactSystemPrompt (+ sharec_loreHeader/_brevityRule helpers)
prompt-builder/turn-prompts	the per-phase prompt builders	static/js/agent/prompt-builder/turn-prompts.js	buildReaction/Observation/Decision/ResultReaction/DashFollowUp prompts
agent/rate-limiter	API throttle	static/js/agent/rate-limiter.js	RateLimiter: requests-per-minute window + waitMs()
agent/response-parser	LLM response parsing	static/js/agent/response-parser.js	ResponseParser: extract inner_monologue/action/speech/emote/nrepairJSON()
agent/threat-detector	room threat scan	static/js/agent/threat-detector.js	ThreatDetector: hostile-actor scan → formatted alert string (or null
agent/turn-feed	the human panel's feed and turn digest	static/js/agent/turn-feed.js	a viewer-scoped ring buffer of log rows, digest rendering, and auc
agent/turn-queue	turn ordering and advancement	static/js/agent/turn-queue.js	TurnQueue: sequential/random/initiative order, current character,
agent/turn-scene-view	scene-first chips for the turn panel	static/js/agent/turn-scene-view.js	TurnSceneView: GET /api/scene/ → clickable area/people/things/w filling menus
agent/turn-you-strip	the "You" strip	static/js/agent/turn-you-strip.js	TurnYouStrip: vitals bars, condition chips, carrying/wearing menus, memories
agent/vital-thresholds	the single source for vitals tier numbers	static/js/agent/vital-thresholds.js	CRITICAL/WARNING, BLADDER_, DRIVE_, SOCIAL_MILD, SANITY_S hoverText()
agent-engine	the character agent loop and turn management	static/js/agent-engine.js	AgentEngine: per-character step pipeline, phases, planning, reflect

Module	Purpose	File	Contributes
agent-lens	prompt-context preview	static/js/agent-lens.js	AgentLens: renders the exact area/agent prompt context an LLM v
api	the single HTTP client for every UI → engine call	static/js/api.js	ApiClient.* (actions, graph CRUD, items, saves, scenario ops) + run
config	user settings, saved API profiles, and toLLMConfig()	static/js/config.js	the config singleton: model/keys, thinking, graph physics, UI tog
context-window	context pruning and token budgeting	static/js/context-window.js	ContextWindowManager: token accounting and history pruning fc
event-bus	tiny synchronous pub/sub used across the UI	static/js/event-bus.js	AppEventBus (on / off / once / emit) and the appEvents single
event-stream	the event stream panel	static/js/event-stream.js	EventBus service: log/emit, row kinds, turn cards, filters, persistenc
graph/context-menu	right-click menus for graph nodes and edges	static/js/graph/context-menu.js	GraphContextMenu: per-type items (inspect/edit/duplicate/delete item/character/trigger)
graph/edge-inspector	the edge property editor	static/js/graph/edge-inspector.js	EdgeInspector: render/edit an edge's type and properties (add/de
graph/edge-types	the edge type registry	static/js/graph/edge-types.js	EdgeTypes: type names, colors, icons, valid source/target combos,
graph/event-handlers	graph click/context/manipulation handlers	static/js/graph/event-handlers.js	GraphEventHandlers: node/edge click, right-click context, addNod
graph/focus	search reveal + camera/physics focus	static/js/graph/focus.js	GraphFocus: freeze hidden nodes, cluster matches in a grid, keep-framing
graph/graph-background	the reference map overlay	static/js/graph/graph-background.js	the map image, its transform (rect/rotation/crop/opacity), and per persistence
graph/layout-engine	cardinal-based area placement	static/js/graph/layout-engine.js	window.GraphLayoutEngine.applyCardinalLayout(nodesObj)
graph/network-manager	vis.js Network construction and data loading	static/js/graph/network-manager.js	GraphNetwork: options, loadGraphData, applyVisibility, legend/tag
graph/node-badges	compact label indicators	static/js/graph/node-badges.js	NodeBadges: mechanic badges + tag-library icons in node labels, legendHtml()
graph/node-operations	node/edge CRUD for the graph	static/js/graph/node-operations.js	GraphNodeOps: create/delete/duplicate nodes and edges
graph/overlays	the ambient visual overlays	static/js/graph/overlays.js	GraphOverlays: light/heat/sound/trigger/cardinal recolouring + ca computeAmbientLight
graph/projector	the pure "what should be visible" model	static/js/graph/projector.js	GraphProjector.computeVisibleNodeIds() + applyVisibility() (no vis
graph/tooltips	rich hover tooltips	static/js/graph/tooltips.js	GraphTooltips: per-type tooltip text/HTML, tippy binding, edge-hc

Module	Purpose	File	Contributes
graph/tree-view	the left-panel world outline	static/js/graph/tree-view.js	GraphTreeView: interactive tree of rooms, exits, items, and players
graph-manager	graph view facade + shared graph state	static/js/graph-manager.js	graphManager : network handle, node map, filters, reveal/bulk/flo
inspector/agent/agent-header	the agent inspector's header block	static/js/inspector/agent/agent-header.js	InspectorAgentHeader: name/status row, emotion selector, vitals r
inspector/agent-view	the full character inspector	static/js/inspector/agent-view.js	InspectorAgentView: Inventory/Bio/Advanced tabs, paperdoll, trait memories
inspector/area-view	the area (room) inspector	static/js/inspector/area-view.js	InspectorAreaView: description/environment/light/noise editing, A
inspector/behaviors-view	the scripted-NPC behaviour editor	static/js/inspector/behaviors-view.js	InspectorBehaviors: behaviour list, action cards, trigger/interval/pr
inspector/helpers	shared inspector field/section builders	static/js/inspector/helpers.js	InspectorHelpers: field/row builders, tag editor, common sections
inspector/item-view	the item inspector	static/js/inspector/item-view.js	InspectorItemView: actions, parameters, tags, state, move, AI impr
inspector/known-by	character knowledge authoring	static/js/inspector/known-by.js	KnownBySection: the "Manage" modal (entities grouped by categ cleanup)
inspector/lore-view	world lore editor	static/js/inspector/lore-view.js	InspectorLore: list / add / edit / delete lore entries
inspector/memory-view	character memory management	static/js/inspector/memory-view.js	InspectorMemory: structured memories + flat world knowledge, e
inspector/panel	the single owner of #inspector-panel	static/js/inspector/panel.js	InspectorPanel.render(template): the only code allowed to write th
inspector/paperdoll-view	paperdoll + equipment menus	static/js/inspector/paperdoll-view.js	InspectorPaperdoll: body-region rendering, slot menus, equip/une
inspector/trigger-helpers	trigger display/removal for item & way inspectors	static/js/inspector/trigger-helpers.js	InspectorTriggers: list a node's trigger edges, remove one, open th
inspector/way-authoring	shared way/door authoring helpers	static/js/inspector/way-authoring.js	WayAuthoring: way naming, state, cardinal/direction helpers, vis e
inspector/way-view-connections	connection editing for the way inspector	static/js/inspector/way-view-connections.js	InspectorWayViewConnections: add/remove/reconnect the areas :
inspector/way-view-triggers	trigger extraction for the way inspector	static/js/inspector/way-view-triggers.js	InspectorWayViewTriggers: pull a way's trigger edges into the insp
inspector/way-view	the way (door/passage) inspector	static/js/inspector/way-view.js	InspectorWayView: showWay, state, cardinal/direction, connection
inspector	right-panel dispatcher for whatever you selected	static/js/inspector.js	the inspector singleton: showNode/showAgent/showRoom/sho routing, open/close
item-library/ai-generation	AI item generation and improvement	static/js/item-library/ai-generation.js	ItemLibraryAI (mixed into ItemLibrary via .call(this)): generate/impr parsing
item-library/consumable-triggers	heuristic trigger suggester	static/js/item-library/consumable-triggers.js	ItemLibraryTriggerSuggester.suggest() (offline, no AI) + suggestFo

Module	Purpose	File	Contributes
item-library/contents-editor	container contents editor	static/js/item-library/contents-editor.js	ItemLibraryContents (mixed into ItemLibrary): contents list + autoc items
item-library/placement	placing library items into the world	static/js/item-library/placement.js	ItemLibraryPlacement (mixed into ItemLibrary): place into rooms/c multi-spawn
item-library	the item library browser and editor	static/js/item-library.js	ItemLibrary: CRUD, AI generation, trigger/effect editing, container area placement
library-browser	the unified library browser	static/js/library-browser.js	LibraryBrowser: tabbed Items/Characters/Rooms/Traits/Conditions + editors
llm-client	the provider client (Chat Completions / Responses)	static/js/llm-client.js	LLMClient.chat/chatWithTools: retries, streaming, JSON repair, thin controls, raw-capture hook
main	bootstrap that wires every front-end module together	static/js/main.js	the VW namespace, legacy global aliases for HTML onclick, init on
narration-ui	the narration mode selector	static/js/narration-ui.js	NarrationUI: the none / player / ai toggle and its wiring
nl-editor/agent-loop	the NL editor's multi-turn ReAct loop	static/js/nl-editor/agent-loop.js	NLEditorAgent: tool-calling turns, library-first system prompts, cla
nl-editor/ghosts	live preview of staged operations	static/js/nl-editor/ghosts.js	NLEditorGhosts: dashed ghost nodes/edges for create/update/del auto-pan
nl-editor/index	the NL editor controller	static/js/nl-editor/index.js	window.NLEditor: wires StagingBuffer + ToolRouter + AgentLoop
nl-editor/staging	the staged-ops buffer	static/js/nl-editor/staging.js	NLEditorStaging: buffer graph ops until explicit Apply; approve/rej
nl-editor/tools	the 24-tool catalog + overlay graph view	static/js/nl-editor/tools.js	NLEditorTools + OverlayGraphView: merge live worldState with un
nl-editor/ui	the NL editor's UI components	static/js/nl-editor/ui.js	NLEditorUI: chat stream, staged-ops tray, clarification buttons, Cm
prompt-docs	shared prompt text for generation features	static/js/prompt-docs.js	VW.PromptDocs: the ITEM_GENERATION_SYSTEM prompt (scheme instructions)
shared/ai-generator	the shared LLM call + parse pattern	static/js/shared/ai-generator.js	AIGenerator.generate(userPrompt, systemMessage, opts) →
shared/dataset-collector	dataset capture + raw exchange store	static/js/shared/dataset-collector.js	DatasetCollector.capture / captureRaw / getAll / getAllRaw / clear panel
shared/diff-modal	world→library conflict resolution	static/js/shared/diff-modal.js	DiffModal.show(current, incoming, sections, options) → update / c per-entry selection
shared/dom-utils	small DOM helpers	static/js/shared/dom-utils.js	escaping, element creation, and event helper utilities
shared/embedding-client	embeddings for semantic memory	static/js/shared/embedding-client.js	window.EmbeddingClient (OpenAI-compatible /embeddings); retu
shared/emotion-mapper	emotion label → affect dimension	static/js/shared/emotion-mapper.js	labelToDimension: curated keyword map + embedding fallback → null
shared/env-presets	named environment presets + zone apply	static/js/shared/env-presets.js	EnvPresets: saved presets (localStorage) and apply to area / neigh
shared/json-schemas	structured-output schemas for LLM calls	static/js/shared/json-schemas.js	StructuredFormats: strict json_schema shapes + a plain json_objec

Module	Purpose	File	Contributes
shared/json-utils	JSON extraction and repair	static/js/shared/json-utils.js	code-fence stripping, JSON extraction, repairJSON()
shared/lit-bootstrap	the deferred lit-html bootstrap	static/js/shared/lit-bootstrap.js	stamps html / render / directives onto window.Lit for classic script
shared/search-select	searchable single-value select	static/js/shared/search-select.js	SearchSelect(container, { options, value, inputClass, onChange, all
shared/tag-multiselect	searchable tag picker	static/js/shared/tag-multiselect.js	TagMultiselect(container, { tags, appliesTo, allowNew, onChange })
shared/template-sync	the "Library Template" footer pattern	static/js/shared/template-sync.js	InspectorTemplateSync: template selector + Refresh-from-Library
shared/trigger-editor	the add/edit trigger modal	static/js/shared/trigger-editor.js	TriggerEditor.show({ mode, initialData, onSave, ... }) + TRIGGER_SN
shared/trigger-graph	the node-graph editor for triggers/behaviours	static/js/shared/trigger-graph.js	TriggerGraph.show(): trigger/condition/effect (or behaviour/action wires + onSave
shared/trigger-suggest-ai	AI trigger suggestions	static/js/shared/trigger-suggest-ai.js	TriggerSuggestAI.suggest(fields, kind) → cleaned trigger objects, o
shared/trigger-suggest-diff	review/merge UI for suggested triggers	static/js/shared/trigger-suggest-diff.js	TriggerSuggestDiff: classify keep / add / conflict, then decide per r
shared/trigger-types	trigger dropdown vocabulary	static/js/shared/trigger-types.js	TriggerTypes: trigger / condition / effect type lists for the editors
shared/vital-color	polarity-aware vital bar colours	static/js/shared/vital-color.js	VitalColor: colour + percent for a vital (drives invert, Temperature I
sky-scape	the engine-driven Sky Clock UI	static/js/sky-scape.js	SkyScape: top-bar time/date/moon/weather line + the World Sky
storage	the IndexedDB persistence layer (with a localStorage fallback)	static/js/storage.js	StorageProvider: get/set/getAll/clear per named object store
stream/control-mode	character control mode cycling	static/js/stream/stream-control-mode.js	StreamControlMode: cycle avatar / agent / observer control for a c
stream/stream-filters	filtering, scoping, and search	static/js/stream/stream-filters.js	StreamFilters: kind filters, actor/area scoping, stream search, persis
stream/stream-persistence	IndexedDB round-trip for the stream	static/js/stream/stream-persistence.js	save/restore of the event stream (cap 2000) + the persisted area fi
stream/stream-raw-llm	collapsed LLM payload chips	static/js/stream/stream-raw-llm.js	collapsed request/response chips with token meters, parse-error s
stream/stream-scrubber	timeline minimap	static/js/stream/stream-scrubber.js	StreamScrubber: per-entry colour segments, click-to-jump, scroll I
stream/stream-turn-cards	turn card grouping	static/js/stream/stream-turn-cards.js	StreamTurnCards: group log entries into per-character turn cards
ui/changes-panel	changes-since-source panel	static/js/ui/changes-panel.js	ChangesPanel: live-vs-source diff grouped by kind, with per-group
ui/command-palette	Ctrl+K fuzzy command palette	static/js/ui/command-palette.js	CommandPalette: fuzzy search over graph nodes, menu actions, ai

Module	Purpose	File	Contributes
ui/create-modal	create room/item/connection modal	static/js/ui/create-modal.js	CreateModal + the item vocabularies (actions, equip slots, states, i
ui/edit-feed	"World edited" feed with per-edit undo	static/js/ui/edit-feed.js	EditFeed: session-local feed (cap 8) of world_changed events, each
ui/engine-config-view	the Engine Config settings tab	static/js/ui/engine-config-view.js	EngineConfigView: renders and saves the tunable engine constant
ui/help-center	help, coach tips, and guided tours	static/js/ui/help-center.js	HelpCenter: tip registry, smart triggers, spotlight, tours, and the H
ui/llm-inspector	the raw LLM exchange inspector	static/js/ui/llm-inspector.js	the 🐞 floating panel: expand per entry, usage line, copy request/
ui/recent-edits	the "where was I" rail	static/js/ui/recent-edits.js	RecentEdits: session-local history of GUI edits + the floating 🌀 ju
ui/room-template-palette	New Room from Template	static/js/ui/room-template-palette.js	RoomTemplatePalette: list library areas, create a room from one (+ items)
ui/saveload-view	save/load game UI	static/js/ui/saveload-view.js	SaveLoadView: save list, save/load/delete, named saves
ui/scenario-health	scenario health dashboard	static/js/ui/scenario-health.js	ScenarioHealth: per-file health scan (parse, trigger edges, dangling + stats
ui/scenario-manager	the Scenario Manager modal	static/js/ui/scenario-manager.js	ScenarioManager: list / Open / Audit / Duplicate / Rename / Delet
ui/scenario-status	source status chip + one-click Commit	static/js/ui/scenario-status.js	ScenarioStatus: dirty indicator (edit_seq vs commit_seq) + 📄 Com
ui/scenario-wizard	"Scenario from Text" wizard	static/js/ui/scenario-wizard.js	ScenarioWizard: premise → LLM-drafted world in the template for review, apply
ui/settings-view	the Settings / profile UI	static/js/ui/settings-view.js	SettingsView: populate/save the settings form, profile switch/save, test
ui/setup-checklist	new-scenario onboarding checklist	static/js/ui/setup-checklist.js	SetupChecklist: the five setup stages with links to each tool, per-sc
ui/undo-history	visible undo history dropdown	static/js/ui/undo-history.js	UndoHistory: list labeled snapshots (newest first) and restore to ar
ui/world-export	export, print, and clipboard utilities	static/js/ui/world-export.js	WorldExport: save-file dialog/downloads, world JSON export, pron
ui-controller	the left-hand roster panel	static/js/ui-controller.js	UIController: agent list, per-agent vitals bars, alerts, turn info, form
ui-helpers	shared UI utilities	static/js/ui-helpers.js	toast/notify helpers (Notyf) plus tooltip and select-enhancement h
validator-panel	the World Issues triage panel	static/js/validator-panel.js	ValidatorPanel: group by node or code, jump / quick-fix / remove , progress
world-state	reactive cache of the server's /api/state payload	static/js/world-state.js	the worldState singleton: data, fetch/poll, getNode/players/area
world-sync	the "Sync World → Library" list	static/js/world-sync.js	WorldSync: per-entity status (new/diff/synced) + routing into the s

Awaiting a contract header

These predate the contract; the guard ignores them until they are documented.

Long-Horizon Simulation — Design & Progress

Last updated: 2026-09-19

Goal: run the Kraktooth Goblin Camp (and any scenario) for weeks of in-game time with **every** character a real agent — not 7 minds and 16 mannequins — while keeping LLM cost proportional to *novelty and attention*, not to time or population. Then let the user watch it, follow a character, and drop in to interact.

Related: [[reversibility-contract]], [[trace-format]], tasks 397 (scopes), 399 (background simulation), 403 (unified memory).

1. The constraint that drives everything

The goblin scenario pins `time_per_tick_minutes = 1`. A week is 10,080 ticks. With ~23 characters, LLM-per-character-per-tick is ~80,000 calls/week — dead on arrival. So the design decouples three things that the current engine fuses:

1. **World time** — a cheap 1-minute tick that advances regardless of the player.
2. **Cognition** — LLM calls anchored to events/sequence-boundaries, not ticks.
3. **Fidelity** — only the attended set (anchors + graph-hop radius) runs full cognition; everything else runs deterministic background rules.

Non-negotiable decisions:

- **One state model, two decision policies.** Same `Player`, same graph, same clock; only the decision source changes. See the reversibility contract.
- **Trace with reasons.** Background actions write an objective fact + why-tag; the LLM summarizes the trace into memory (never the reverse).
- **Shared state, not two models.** Promotion is "start asking the character instead of their rulebook."

Attention model (borrowed from Star Citizen Quantum's virtualize/physicalize, with Dwarf Fortress / RimWorld identity continuity): anchors (observed character, human player, pinned location/item) + a graph-hop radius + a global cap with deterministic eviction + hysteresis, promoting on approach.

2. Phase 0 — Survival (done)

The world could not survive a day: session-scaled drains killed everyone in ~2h.

- `vital_rates.py` — single source of truth for per-minute rates (1 tick = 1 in-game minute). From a full meter: Hunger ~3 weeks, Thirst ~3 days, Energy ~16h; Social/Hygiene/Entertainment ~1–2 days; Sanity slow (~14 days). `change(player, stat, per_minute)` is the fractional accumulator so sub-1 rates actually accrue.
- `engine/tick_manager.py` — baseline loop + all environmental, temperature, social, sanity, bladder, sleep and HP-regen effects routed through `vital_rates`. Hunger/Thirst grace+drain retuned so time-to-death lands near the 3-week / 3-day targets.
- `engine/activities.py` — activity regen rescaled to per-minute.
- **Scenario migration** — `tools/migrate_decay_rates.py` re-bakes per-player `decay_rates` (baked rates *override* engine defaults, so calibration was invisible without this). Applied to `world_template.json` + the goblin scenario.
- **Goblin weather fixed** — the scenario shipped a perpetual blizzard (`temperature_mod` to -15) on a green, temperate map, inherited as authored content. Replaced with a clear → overcast → rain day cycle.
- `data/library/traits/high_metabolism.json` — goblin fast metabolism (Hunger ×2, Thirst ×1.5, Energy ×1.3).

Findings (with evidence)

- Environmental drains (stale air 1/tick, humid Hygiene 1/tick, dark Sanity 1/tick, cold Energy/HP per tick) dominated and killed in ~2h.

- **The mystery HP drain was temperature, not sanity.** Sanity breakdown conditions never drain HP by design; cold does (`core_temp < 33` → HP `-0.15/min`). Root cause: an authored **snowy gale forecast re-applies every tick**, driving effective temperature below 5°C and core temp down. Proved by isolating weather (zero HP drops with weather cleared).
- The soak's "neutral environment" was initially invalid because weather re-writes area env each tick; the tool now clears forecast too.

Tooling — `tools/soak_sim.py`

Headless fast-forward with **live progress** (bar, %, game time, ticks/s, elapsed, ETA, alive/dead, new deaths + causes), projected wall-clock for day/week/month, survival + cause breakdown, death times, growth (log/turn_events/graph/memories/trace), survivor vitals, `--debug-hp` (HP drops with conditions/temp/area/env), `--neutral-environment`, `--engine-decay`, `--override`, `--set`, `--apply-trait`, `--report`.

Throughput: ~6–9 ticks/s with 23 characters → **1 game week ≈ 18–29 min, 1 month ≈ 1.5–2h.**

3. Phase 1 — Time engine (partially exists)

Run-N-turns already works, and with no human in the roster nothing blocks the round. Remaining: a server-side batch `advance` and a non-blocking human (no-human roster, or a turn timeout). See the recap for the verified mechanics (2s/step hardcoded, one `tick_turn` per queue wrap).

4. Phase 2 — Background tier (in progress)

Done: the objective trace

- **engine/trace.py** — `record / recent / since / summarize_window / rollup / load / to_list`. Bounded at 200 entries, salient-first retention, run collapsing. No LLM.
- **Integrated** — `Player.trace_log`, `Player.to_dict`, and `engine/serialization._deserialize_player` round-trip it.
- **Wired** — need tier crossings (`why="needs:<vital>"`), deaths (`salient`), and resolved actions in `apply_action`.
- **Tests** — `tests/test_trace.py` (7). Soak reports trace volume.

This is task-399's "append-only facts log."

Done (v1): background survival runner

`engine/background_simulation.py` (`BackgroundSimulation.process_due`), wired into `tick_turn()` after `process_simple_npcs`. Characters with `simulation_mode == "background"` are considered only when due (`next_due_tick`, deterministic jittered interval), and make one coarse decision against shared state:

- **drink** when Thirst ≥ 55 — either from a water-tagged **area** (natural water is an area tag, not an item) or a drink item, else travel toward one;
- **eat** when Hunger ≥ 55 from a reachable food item;
- **sleep** when Energy ≤ 30 , or ≤ 15 regardless (critical exhaustion wins);
- **travel** one hop toward the nearest reachable resource area.

Every decision writes a trace entry with a reason tag. `simulation_mode` and `next_due_tick` are serialized. A 2-day, 23-character soak went from **3/23 alive** → **22/23** after the priority fix. Tests: `tests/test_background_simulation.py`.

Next: schedules, promotion/demotion, and scenario hygiene

These are filed as tasks — see §9 for the current map.

1. **Background schedules / work / coarse social** (task-409) so background characters do more than survive.

- 2. **Promotion/demotion** (task-412) — `trace.summarize_window` builds the LLM catch-up summary on promotion; demotion rolls the foreground span into the trace (reversibility contract).
- 3. **Scenario hygiene** (task-408) — reachability is now fixed (30/30, see §7); what remains is de-duplicating the 46→23 character nodes, canonical way/area ids at authoring time, and giving the scenario a `name`.
- 4. **Food renewal** (task-410) so a month does not starve the camp.

5. Files touched this pass

File	Change
<code>vital_rates.py</code>	new — canonical per-minute rates + <code>change()</code>
<code>player.py</code>	decay defaults from <code>vital_rates</code> ; <code>trace_log</code> + serialization
<code>virtual_world_engine.py</code>	<code>baseline_decay</code> from <code>vital_rates</code>
<code>engine/tick_manager.py</code>	all drains → per-minute; trace on needs/death/act
<code>engine/activities.py</code>	per-minute activity regen
<code>engine/trace.py</code>	new — trace store
<code>engine/serialization.py</code>	restore <code>trace</code>
<code>data/library/traits/high_metabolism.json</code>	new goblin metabolism trait
<code>data/scenarios/kraktooth_goblin_camp.json</code>	temperate forecast; <code>decay_rates</code>
<code>world_template.json</code>	<code>decay_rates</code> migrated
<code>tools/soak_sim.py</code>	new/expanded — reporting, debug, isolation
<code>tools/migrate_decay_rates.py</code>	new — rate migration
<code>tests/test_trace.py</code>	new ; <code>test_activities.py</code> , <code>test_social_company.py</code> updated to per-minute
<code>docs/design/reversibility-contract.md</code> , <code>docs/design/trace-format.md</code>	new contracts

6. Test status

Full suite excluding `test_mcp_*` (pre-existing 'function' object has no attribute failures, unrelated): **2879 passing, 0 failures**. The combat suite has a pre-existing flaky RNG test that can fail intermittently.

The exact total moves with `tests/test_data_no_mojibake.py`, which is parametrized over every JSON under `data/` — adding or deleting a scenario, template, or tag file shifts it by one. Compare failures, not the total.

7. Open issues

- **Scenario graph ids are still inconsistent.** `tools/repair_way_links.py` symmetrized way links and connected Side Tunnels ↔ Water Source, so all **30/30 areas are reachable** (verified via `include_hidden=True`); the earlier "6 stranded areas" is resolved. Remaining: way/area endpoints should be canonical at authoring time so strict-id pathfinding needs no normalized-name compensator.
- **Duplicate character nodes.** The canonical goblin scenario has **46 character nodes for 23 players**. 22 are bare-named orphans (0 edges, referenced nowhere); the extra is a `player_human_explorer` / `player_player_human_explorer` artifact. Raw JSON node **keys do not match their id fields**, so dedupe must go through `WorldGraph` with an orphan proof, not raw JSON surgery (task-408).
- **The camp's 11 authored triggers are dead data.** They are `logic_trigger` → `area` edges with `event` on the node; the runtime matches `trigger_type` on the edge with the owner as source, and nothing flips them at load. Folder-authoring must emit the modern shape (task-408).
- **Food is at the edge.** One-week soak survivors average Hunger **84.7** (max 100 — starvation end of the drive scale). The camp cannot feed itself over a month (task-410).

- Background characters currently only **survive** (eat/drink/sleep); they have no schedules, work, or social behavior yet. Soak shows Social 0.1, Sanity 0, Hygiene 0, Entertainment 0 (task-409).
- **Cross-zone roaming** remains the hardest seam (see reversibility contract).
- **MCP test failures** are pre-existing and unrelated to this work.

8. Performance pass (2026-09-19) — tasks 406, 407

The blocker found by profiling was **not** the LLM: it was trigger/exit plumbing. Per tick the engine swept all ~106 nodes calling `_execute_triggers`, which built a full template context *before* checking for a match, and that context read the legacy `current_area` property — rebuilding an `Area` and its exits on every access, by scanning all edges and lowercasing every endpoint (~876 exit rebuilds and ~1M `str.lower()` per tick). The camp has **zero** tick/time triggers, so all of it was waste.

Task 406 — trigger dispatch

- `graph.get_trigger_sources(type)` : event index over `triggers` edges that carry a `trigger_type` . `_fire_turn_triggers` / `_fire_time_triggers` now iterate only trigger owners, of any node type.
- Standing items (not carried, not lit) now get `on_tick` , once, without double-firing the carried/lit paths — the prerequisite for growth triggers.
- `_execute_triggers` fetches edges and type-filters before building context, and reads the area by id instead of the `current_area` property.

Task 407 — graph lookups

- Edges indexed by lowercased source/target; lookups are O(degree) with no per-edge `.lower()` . `_revision` drives cache invalidation; a length check lazily rebuilds if external code mutates `edges` directly.
- `build_exits_for_area(include_hidden=True)` cached by revision (authoring path only — the game view depends on player discovery state).
- Lighting scan collapsed from two passes to one.

Measured

- One-week (10,080-tick) background soak: **9m49s** → **17.1 ticks/s, 23/23 alive, 0 deaths** (was ~6–9 t/s; a week used to be unreachable interactively at ~128h of browser-driven wall clock). Trace 4,459; memories 17.
- Full suite: **2831 passing**, 0 failures (the 4-file delta is the mojibake parametrization, not coverage).
- Post-fix profile hot spots: `lighting.get_ambient_light` (~33%, now trimmed), the call volume of `get_edges_for_target` , background `move_to_area` , and `Player.state` . The old trigger/exit cost is gone from the hot path.

What this does and does not change for playing

- *Simulating/observing* the world: materially better — weeks are now minutes headless, triggers cost nothing when unused, and objects can act over time.
- *Interactive play*: essentially unchanged. The browser still drives one character at a time with a hardcoded ~2s step and one `tick_turn` per full roster wrap. The 2s sleep is UI pacing, not a rate limit (that is `RateLimiter`); it should be configurable and skipped when a step already took longer. Making the world playable at speed is **task-414**.

Second pass (same day) — lighting and edge moves.

- `get_ambient_light` now uses `max(base, best_item)` instead of `min(base + sum_items, max(base, best_item))` . The sum was dead arithmetic the ceiling discarded: four torches never out-shone one. Effective light is the brightest source, full stop.
- Every area's light is computed **once per tick** (`recompute_area_lights`) and read from a stamp, instead of rescanning the room plus neighbours on every access. The stamp is keyed to graph revision, and `toggleable_items` / burn-out still compute fresh on their own paths.
- `graph.remove_edge` / `remove_edges_for_node` now unindex just the removed edges instead of rebuilding every index (trigger edges still rebuild, since the trigger index is a set of sources).
- `graph.retarget_edge` added: unequip moves its edge in place (`equipped` → `carrying`) instead of remove + add. Take/drop still remove + add, but its

direct `graph.edges.remove(...)` bypasses were replaced with graph calls.

Follow-up: retarget take/drop placement edges too (task-407 note).

Measured after the second pass: one-week soak **56s** → **181 ticks/s** (was 9m49s / 17.1 t/s after the first pass), 23/23 alive, and **identical survivor vitals and trace count** — behaviour unchanged. Test suite runtime fell 78s → ~25s.

Third pass — the accumulating cost was the turn-event buffer.

Profiling early vs late ticks (200 vs 9,500) showed `get_edges_for_target`, `condition_has_condition`, `get_state`, lighting and temperature all flat, but `GameLogger.record_turn_event` went 0.086s → 0.987s for the *same* call count. Cause: it rebuilt the whole `turn_events` list on every append, and a headless run never calls `clear_turn_events`, so the buffer grew to ~17,000 — $O(n^2)$ across the run. It now prunes only when the turn changes and caps the buffer at 2,000. After the fix, call counts are flat early vs late (~2% drift) and `record_turn_event` is out of the hot list. The residual is periodic hourly work plus host-load variance.

Also in this pass: take/drop capture their placement edge and retarget it (player on take, room on drop) instead of remove + add, and the capacity/hand checks now run **before** any mutation, so a failed take can no longer orphan the item.

9. What's left (task map)

#	Area	What	Status
406	triggers	event index, standing-item ticks, lazy context	review
407	graph	edge indexes, exits cache, lighting pass	review
408	world	consolidate scenario, dedupe nodes, canonical ids, folders→JSON	todo
409	characters	background schedules + daily reflection + coarse social	todo
410	gameplay	plant growth → food renewal, week supply	todo
411	world	attention budget / fidelity tiers	todo
412	characters	promotion/demotion + trace→memory consolidation	todo
413	testing	tick-perf baseline + regression guard	review (<code>tests/test_perf_guards.py</code>)
414	gameplay	server-side <code>advance(N)</code> + non-blocking human	todo
415	ui	long-horizon observer mode	cancelled (deferred)

Pending confirmations

Decisions and findings from the `static/js` documentation sweep. Each item records the **decision** and **what was done**. Anything still open is listed at the bottom.

Created 2026-09-20.

A. Prompt / vitals — resolved

#	Item	Decision	Status
A1	Unreachable <code>Social</code> tier in <code>character-state.js</code>	Keep the fix	✅ done — <code>SOCIAL_MILD = 65</code> , the <code>[social_need: moderate]</code> cue now fires at 50–65
A2	<code>voiceLabel()</code> ignored the <code>known</code> registry	"if you know someone you know their voice" → use their name	✅ done — shared <code>isKnownToViewer()</code> now backs both <code>anonymousName</code> and <code>voiceLabel</code>
A3	<code>includeMemory</code> accepted but never wired	Wire it as optional , not removed — memories now come at turn <i>start</i>	✅ done — <code>config.endOfTurnMemory</code> (Settings → 🟡 End-of-turn Memory), off by default
A4	<code>MEMORY_INSTRUCTION_REACTION</code> vs <code>_REACT</code> drift	Combine them	✅ done — one <code>MEMORY_INSTRUCTION_REACT</code>

#	Item	Decision	Status
A5	Tier numbers hardcoded outside <code>vital-thresholds.js</code>	<i>see explanation below</i>	🕒 open
A6	<code>decide</code> recomputed <code>relationshipNL</code>	"unification thing" — but see below	✅ done differently — it was dead code , removed

A5 — what it actually means

`agent/vital-thresholds.js` declares itself "the ONE source for vitals tier boundaries". The *numbers* for Hunger/Thirst/Energy/Sanity/Social/Bladder all live there. But `character-state.js` still hardcodes other tier numbers inline:

- **Entertainment** prose tiers: `10` , `25` , `50` (lines ~395–398)
- **Sanity** has one stray `< 75` alongside the named tiers
- **Temperature** prose: `33 / 35 / 36` and `38 / 40 / 42` (these are *now* band-driven for the species work — see B3 — but the warm-blooded defaults are still written inline at the call site)

The risk is drift: change the "bored" threshold in one file and the other file still uses the old number, so the UI and the prompt disagree.

Options: (a) move them into `vital-thresholds.js` (`ENTERTAINMENT_*` , `SANITY_STRAINED`) and read them in `character-state.js` ; (b) accept that prose-local numbers are fine and fix the file's header to say so.

My recommendation: (a) for Entertainment + Sanity (cheap, removes the drift); (b) is defensible for the temperature prose now that it is band-driven.

A6 — resolved as "delete the dead code"

`buildObservationPrompt` and `buildDecisionPrompt` are documented dead exports (never called) in three places — `Agent Engine.md` , `task-160` , and `task-350` , which states the real turn-start call is `buildReactionPrompt` . Rather than "unify the calling convention" of code nothing calls, both builders and their exports were **deleted** (82 lines). `git log` has them if the phase split is ever revived.

B. Carried over

B1 — Graph map + layout — partly fixed, still needs your visual pass

Right-click empty canvas → 🖱️ Add background image; drag / corner-resize / top-dot rotate / crop. Resize is scale-about-centre.

Search was wrecking hand-made layouts — fixed. Two causes:

1. `focus.js _clusterResults()` called `network.moveNode()` on every match, ignoring that the node was frozen (`physics: false`). It now excludes frozen nodes from the cluster grid.
2. `focus.js _kickClusterPhysics()` switched **global** physics on and ran `stabilize(60)` even when physics was off — re-settling the whole layout. It now returns immediately when `graphManager._physicsEnabled === false` .

Node positions now persist to the world. 🖱️ → 💾 **Save layout to world** (and locking) writes every node's `properties.x` / `properties.y` through `ApiClient.batchGraph` — one atomic request, one undo step. `buildNodeConfig()` seeds `x / y` from those properties on load, so a layout survives reloads, travels with the scenario file, and can be committed. The browser-local IndexedDB copy remains as a fallback.

Backgrounds are per-scenario (fixed). Two bugs made a map leak between worlds: the module never reset when a scenario loaded (the previous world's map stayed on screen), and the IndexedDB fallback keyed unnamed scenarios to one shared `default` slot. The **world is now authoritative**: every `state:updated` re-derives the background from the loaded world, and a world with no background of its own clears the overlay. The IndexedDB fallback is consulted only for a **named** scenario, and the physics freeze is tracked so a new scenario doesn't inherit the previous one's locked layout.

The background map travels with the file too. It is uploaded as a **file**

(`POST /api/graph/background/image` → `static/images/backgrounds/...`, mirroring the existing node-image flow) and the scenario keeps only the **path** plus the transform in a new top-level `graph_background` block — serialized exactly like `world_lore`. Base64-in-the-JSON was rejected: a 2 MB map would add ~2.7 MB of text to every commit.

Caveats: positions only *hold* while physics is off/locked, and saving touches ~130 nodes, so the scenario JSON grows slightly and the next Save/Commit includes them.

x / y are excluded from library sync (done): `handle_library_create_or_update` strips presentation-only properties via `_strip_presentation_properties`, so a laid-out world can't leak its coordinates into an archetype. The reverse is safe by construction — the template-refresh paths build explicit key whitelists, so they never clear a node's saved position.
`tests/test_library_presentation_keys.py` covers the helper and the route.

B2 — Scenario name field — done

The top-left chip already looked editable ("Click to rename scenario") but the handler only wrote `document.body.dataset.scenarioName` — it **never called the server**, so the name was cosmetic, vanished on reload, and even fed the graph-background cache key.

Now:

- `POST /api/scenario/name` sets `world._scenario_name` and — when the current source has a different basename — **repoints the save target** to `data/scenarios/<name>.json`, so a world booted from the boot template commits into its own file instead of overwriting `world_template.json`. An existing file is **never** clobbered (the previous target is kept and a warning is returned). Names are sanitised.
- The click-to-edit handler POSTs the new name, updates the chip, the body dataset and `worldState.data._scenario_name`, and **reverts on failure** (the server is the source of truth).
- `tests/test_scenario_name.py` (5 tests): name set, target repointed, unsafe characters sanitised, existing target untouched, missing name rejected.

B3 — Cold/hot-blooded species — engine + prompt/alert done

You asked for **cold-blooded (reptilians)**, **hot-blooded (dragonoids)**, and **warm-blooded as the default humanoid**.

Done:

- `data/library/traits/cold_blooded.json`, `hot_blooded.json`
(`effects.temperature_band`); absence ⇒ warm-blooded.
- `engine/traits.py`: `TEMPERATURE_BANDS` + `TraitSystem.get_temperature_band()`
(a trait may name a band **or** override individual keys).
- `engine/tick_manager.py`: drift, cold/heat damage bands, and death-cause attribution all read the band. Warm-blooded defaults reproduce the old numbers exactly.
- `character-state.js` (temperature prose) + `ui-controller.js` (the "Critical body temp" alert) are band-aware, so **Croak-Mother at 20 °C is no longer flagged critical**.
- `Croak-Mother` now carries `cold_blooded`.
- `tests/test_temperature_bands.py` (5 tests).

Still to do: `shared/vital-color.js` bar/percent colours still use the warm scale (they receive only `vitals`, so they need the band threaded through their call sites).

B4 — what it actually means

The scenario file contains **46 character nodes for 23 players**. The extras are zero-edge orphans (node id equals the display name, e.g. a stray `Arix` beside the real `player_Arix`), plus a `player_human_explorer` / `player_player_human_explorer` naming artifact. Two wrinkles make it unsafe to

fix with a text edit:

1. the JSON object **keys don't match the id fields**, and
2. there is no `name / meta.title` on the scenario (B2).

So the dedupe has to load the file through `WorldGraph`, prove each candidate has zero edges and no references, and only then remove it — which is why it is queued as a tool rather than a find-and-replace. Practical effect today: mostly invisible (orphans have no edges), but it doubles some per-node work and can surface duplicate names in lists.

Still open

1. **A5** — move Entertainment/Sanity tier numbers into `vital-thresholds.js`.
2. **B1** — your visual pass on the graph map.
3. **B2** — editable scenario name field (top-left).
4. **B3 remainder** — band-aware colours in `shared/vital-color.js`.

Presence Gap Analysis — Chat RP vs VirtualWorld

Date: 2026-08-23

Purpose: Evidence-based list of what the agent experience already does well, what genuinely hurts the "characters feel present" feeling, and candidate fixes — so we can re-verify each item and decide what to build. Nothing here is implemented yet.

Method: compared exported engine logs against a hand-played multi-character chat RP session (character card + live play). The chat session is the benchmark: two humans, one LLM character, dense banter, ~25 exchanges.

Logs reviewed:

- `data/exports/event_log_2026-08-10T21-45-19.txt` (Task 18 door puzzle, Jane two, old format)
- `data/exports/lyrie_28082026.txt` (Frozen Thicket solo run, Lyrie, new format, 2026-08-18)
- `data/exports/Untitled-1.txt` (Task 17 conditions test, partial)

Caveat: the Aug 10 log predates the Aug 16–23 work (contextual actions, speech salience, conversation instinct, social gating, emotional states). Several findings were already partially addressed between logs; each item below says whether it still needs verification in a current-build run.

Part 1 — Already working (verified in logs)

These were previously suspected missing and are NOT; do not re-litigate without new evidence.

1. **Inner monologue is first-class.** `inner_monologue` is a schema field, rendered as  events (Aug 10 log: 2,267 monologue vs 41 speech events). Voice consistency across a session is strong (Lyrie's "little everflame friend" register never breaks).
2. **Memory writes are experiential.** React-pass memories are first-person, tagged, importance-rated: *"I accidentally put out my everflame ember... I should be more careful next time"* (`clumsiness/magic/loss` , Lyrie log L2245). Backstory memories surface in recall (Great Baking Incident block, Lyrie log L53–58).
- *Correction (Tommy, 2026-08-23):* recall is keyword-based by design; `[just now]` stamps are turn-fresh writes pending tick→time conversion. Re-verify stamp aging in current build.
3. **POV filtering exists.** `=== WITNESSED ===` separates `[the stranger] said:` from `[Heard] a voice said:` (Aug 10 L40994–40997). Not all-knowing by default.
4. **Contextual actions replaced the verb table.** New-format prompts list concrete `=== AVAILABLE ACTIONS ===` per turn (Lyrie log L30–41).
5. **Emotional state injection.** `=== YOUR STATE ===` includes affect lines ("quite anxious and hopeful", Lyrie log L48).
6. **Simulation produces story beats.** Body temperature drove the whole Lyrie run: shivering → seek warm hollow → summon flame → extinguish accident → distress → lesson.
7. **Micro-arc continuity works.** The ember mistake produced a memory that was *referenced under pressure two turns later* ("if I don't put it out this time", Lyrie log L~113).

That is the same mechanism that makes chat RP characters feel continuous.

Part 2 — Confirmed issues

Each: evidence → impact → proposed action → how to verify.

G1. Emote pronoun stitching bug (rendering)

- **Evidence:** "Lyrie hugs **yourself**" (L229), "hugs **your knees**" (L440), double periods sigh. . (L2046, L2258, L3206), Lyrie log.
- **Impact:** constant low-grade uncanny; breaks immersion every few turns.
- **Action:** normalize second-person pronouns to third when prefixing an emote with the actor's name; collapse duplicate trailing punctuation. Tiny renderer fix.
- **Verify:** grep any new export for `<name> (yourself|your |you )` pattern = zero hits.

G2. Engine outcome lines are stubs next to model prose

- **Evidence:** model side "It felt nice and safe, like a little friend." vs engine side "You use the create flame." / "You extinguish the everflame ember." (Lyrie log).
- **Impact:** the world reads like a debug print while characters read like literature; tonal whiplash every action resolution.
- **Action options:** (a) outcome prose templates per action verb with slot-filled variety, (b) optional light narration pass weaving ✨ results into a sentence, (c) leave engine lines terse but make the UI typography de-emphasize them.
- **Verify:** blind read of an export: outcome lines shouldn't be identifiable as "the machine ones".

G3. Prompt boilerplate ratio still high

- **Evidence:** rules/examples blocks re-sent on every decision call (~100 lines even in new format); situational content ~30 lines (both logs).
- **Impact:** model attention budget spent on API documentation; form-following voice.
- **Action:** hoist static rules into the system prompt (provider-side prompt caching where available); slim per-call context to situation + delta.
- **Verify:** token accounting per call; qualitative voice shift test.

G4. No pacing authority (director vacuum)

- **Evidence:** solo runs loop (hollow→thicket ping-pong, returning to abandoned areas); Aug 10 run: 430 ticks, one puzzle loop, nothing escalates. Chat benchmark: the human director called every beat change ("that's about when the date turns bad").
- **Impact:** scenes drift toward need-satisfying mundanity; competent but flat.
- **Action candidates:** lightweight director pressure (scene tension value, beat goals), instigator behaviors for NPCs, or explicit human-director controls. Director-studio's Focus→Monologue→Action weave is a working prototype of the shape.
- **Verify:** 30-turn multi-agent run with zero human input should produce at least one escalation (argument, revelation, arrival) not caused by random wander.

G5. Emotional accumulation is thin in practice

- **Evidence:** Aug 10 log repeats `You feel fine – no pressing needs right now.` every turn; emotional state line exists (new format) but appears as a single adjective line. Contrast benchmark: one rejection colored 20+ exchanges.
- **Impact:** characters reset to neutral between beats unless something big just happened.
- **Action:** verify current emotional-states persistence across turns; if it decays too fast, add mood inertia (events set a baseline that tints N subsequent prompts).
- **Verify:** after a distressing event, check the next 3–5 decision prompts carry tint.

G6. Memory reliability — false positives

- **Evidence:** Aug 10 L41014: *"I have nothing on me — no keycard"* while ground truth was `Carrying: keycard`; self-corrected two turns later (L21602).
- **Impact:** confident wrong memories are worse than gaps; they cause ghost-correction loops.
- **Action:** keyword recall can retrieve stale contradicted entries; consider recency + contradiction marking (later observation about same entity_ids downgrades earlier).
- **Verify:** inventory-state mismatch scenarios in a test run; count false-memory events.

G7. Presentation is telemetry, not scene (umbrella issue)

- **Evidence:** stream renders  ...  ...  act go Door 1 /  go Door 1 /  ... fragments (both logs) vs benchmark's continuous authored prose.
- **Impact:** this IS most of the felt gap. Content is present; typesetting isn't.
- **Action:** optional "scene view" renderer composing a turn's events into paragraphs (monologue italic, speech quoted, emotes woven, engine results as connective tissue). Pure presentation layer — no engine change required.
- **Verify:** read a scene-view export next to the chat benchmark; ask "who wrote which?".

G8. Social density unverified in current build

- **Evidence:** neither reviewed log exercises multi-agent conversation (solo puzzle runs); salience/conversation-instinct/social-gating landed Aug 21–23, post-logs.
- **Impact:** unknown whether banter emerges or agents wait politely for turn structure.
- **Action:** run the mansion scenario (or `taco_bell_date.json`) with 2+ autonomous agents + human, export, evaluate against Part 1 checklist + G4/G5.
- **Verify:** count unsolicited addressations between agents; check RESPONSE PRIORITY rule actually fires on witnessed speech.

Part 3 — Ideas ranked (impact ÷ effort)

#	Idea	Addresses	Effort
1	Emote pronoun/punctuation normalizer	G1	XS
2	Scene-view renderer (presentation only)	G7	M
3	Hoist static rules to cached system prompt	G3	S
4	Mood inertia for emotional states	G5	S
5	Outcome prose templates per verb	G2	M
6	Multi-agent social soak test + eval harness	G8, G4	M
7	Memory contradiction/recency weighting	G6	M
8	Director pressure prototype (tension/beat goals)	G4	L

Part 4 — Open questions

- Does tick→time memory stamp conversion exist in the current build? (Tommy says yes/planned — verify.)
- Do provider-side cached system prompts work with our OpenAI-compatible providers?
- Should scene-view live in the event stream UI or as an export filter first?

Part 5 — Verification round 1: mansion multi-agent run (2026-08-23)

Log: `data/exports/mansion_event_log_2026-08-23T15-04-54.txt` — 7 characters
(elena vance, kyrie johansen, jake halloway, sammy lopez, miki, the butcher, kayla jenkins), ~240 ticks, 29 LLM calls.

Resolved by this run

Item	Result	Evidence
G1 pronoun stitching	FIXED — 0 leaks in rendered emotes	line scan over all  /  renders
Memory stamp aging	LIVE — recalls show "1 minute ago"/"2 minutes ago"; <code>[just now]</code> only for same-turn writes	22/43 recall blocks aged
G3 prompt boilerplate	MOSTLY SOLVED — static rules/schema ≈ 7–8% of call-body chars (~82k of ~1.05M)	boundary scan ACTIONS→first dynamic section
G8 social emergence	PASSING — collaborative deduction chain across 5 agents: diary/skull rule relayed room-to-room → Blackwood calling cards → loose tile under rug → love letters w/ "m." initial → key taped in vase. Direct addressation, distinct voices, building on findings	speech stream ticks 10–246

Item	Result	Evidence
Conversation instinct	live — "You feel sociable right now — inclined to speak up." drives speech	=== CONVERSATION === ×51
Anti-repeat guard	live — ANTI-REPEAT rule references agent's own prior lines	decision prompts
Cross-area hearing	attributed via ways: "[Heard from the library_door_back] a woman's voice said:"	WITNESSED blocks, 23/29 non-empty
Threat autonomy	butcher runs independent horror logic off-screen ("Intruders in my house. The Entity demands sacrifice.")	butcher decide pass

Still open after this run

Item	Result	Evidence
G5 emotional accumulation	CONFIRMED OPEN — 43/43 state blocks read "You feel fine — no pressing needs right now."; zero dynamic affect despite horror stimuli (stalking butcher, haunted-house tension). The Lyrie-run emotion line did not appear on this code path	all === YOUR STATE === bodies
G2 engine outcome prose	unchanged (terse 🌀 lines); less jarring at this event density but still stubs vs model prose	outcome lines throughout
G7 scene-view rendering	untested (presentation layer not built yet)	—
G4 pacing authority	partially exercised: butcher acts as an organic instigator/threat, which may be enough pressure in horror scenarios; calm scenarios remain untested	butcher behavior

Round 2: taco_bell_date first load (2026-08-23, later)

Two-agent run of the authored date scenario (Miki + Jake both autonomous).

Item	Result	Evidence
G5 emotional states	RESOLVED (rendering) — "miki doki is quite anxious." renders when the character has an emotion set; mansion run was empty because every character was neutral/healthy. Vitals-derived inner life also fires: Sanity 70 → === YOUR MIND === unease text; Social 40 → "You feel isolated."	decision prompts, turn 0
Authored-beat reproduction	The earring arc replayed itself: seeded memory ("missing... it matters") + hidden item under table → Miki crawls under the booth mid-date, nearly cries, writes an importance-8 memory (<i>"I told jake 'i'm fine' while absolutely not being fine"</i> , tags relief/grief/hope). No director involved — graph props + memories were enough	turns 0–1
Vitals → characterful behavior	Jake's hunger 10 triggered CRITICAL NEEDS plan-first rules, but he satisfied them in voice: steals the burrito as "date tax" — the same beat hand-written in the source RP	jake decide turn 1
POV stranger labels	bare [Heard] a woman's voice said: pre-introduction — correct; names withheld until introduced	witnessed blocks

Bugs found & fixed in taco_bell_date.json :

1. World lore rendered [general] undefined: — prompt builder expects entry.title (system-prompt.js:112), scenario used name . Added title.

2. Second-person converter verb artifact: "grinning like he knows..." → "like you knows".

Reworded description to avoid pronoun+verb constructions (converter family, G1).

Open follow-ups:

- Does anything *update* emotion during play (events/mood engine), or does it stay at its seed value? Watch across a longer run.
- Ambient dread flavor ("the shadows seem to watch you") is horror-generic; fine for sanity<75 anywhere but worth a scene-appropriate variant pool eventually.
- Steering prefixes before JSON in assistant turns ("Take the earring.", "React silently, emotional.") — confirm these are the intended mode-steering mechanism.

New observations

- Recall quality is planning-grade and relationship-aware: *"miki's following me in. good. i don't want her wandering off alone"* (elena, decision prompt).
- Witnessed-empty blocks (6/29) occur only when a character is genuinely alone — correct.
- Next highest-leverage move: wire emotional states into the multiplayer decision-prompt path (they rendered in the solo Lyrie run but not here), then re-export and re-check.

Relationship interpretation - prompt mockup (task-350)

What this shows: the same moment from a real run, before and after the proposed change.
The core rule: the recipient decides how a line landed - the speaker intent is irrelevant.
The model outputs a discrete bucket (no raw numbers); the engine owns the number via an authorable table.

1. Today - the recipient just gets the raw lines, no interpretation

```
[Tick 21 | 19:20]
- a human turn for miki doki -

=== WITNESSED ===
jake halloway says (to the room): you have got this, please stay
tyler says (to miki): hey, can you move - you are blocking the counter

=== YOUR STATE ===
You are shaking and your stomach is gnawing at you. You feel uneasy.

What do you think, say, and do?
```

Problem: nothing asks how it landed on you. The engine would apply a flat +2 for any speech (or nothing), so a nasty line and a sweet line move the relationship identically. And a speaker self-declaring a tone is wrong - a creep calling himself flirty must not charm the recipient.

2. Proposed - inject a WHAT YOU HEARD block into the recipient inner monologue

```
[Tick 21 | 19:20]
- miki doki receives lines from jake and tyler -

=== WHAT YOU HEARD ===
- jake halloway said: you have got this, please stay
- tyler said: hey, can you move - you are blocking the counter

For EACH line, decide how it landed on YOU (not how it was meant).
Pick ONE bucket per person: big_negative | minor_negative | no_change | minor_positive | big_positive.
The engine applies the delta - you do NOT pick a number.

=== YOUR STATE ===
You are shaking and your stomach is gnawing at you. You feel uneasy.

What do you think, say, and do? Respond in your voice, then give your read.
```

The recipient response - words plus one tiny structured field

```
{
  inner_monologue: jake actually thinks I can do this, that almost made me feel like a real person for a second. Tyler is just annoyed &
  emote: hunches smaller, hugs the tray to her chest

  interpretation: {
    jake halloway: minor_positive,
    tyler: minor_negative
  },
  reason: jake believing in me actually helped; tyler snapping just made the pit in my stomach worse.
}
```

So the recipient feelings, not the speaker claim, drive the change. miki is already uneasy and fragile, so tyler snap is minor_negative even though he probably did not mean it harshly. And jake kindness lands minor_positive because she is starved for it.

3. The engine maps bucket to number (authorable, you tune it)

bucket	delta
big_negative	-6
minor_negative	-2
no_change	0
minor_positive	+2
big_positive	+4

The model never emits these numbers. It picks the bucket; the engine applies the mapped value and clamps.
A single line cannot spike anything - buckets are small and the delta is clamped. Missing/invalid bucket -> no_change (0).

4. The creepy-to-kid case - why subjectivity matters

```
creep leans in: hey cutie, you look like you need a ride home
-> jessica (13, scared, already on edge) reads it as: big_negative -> -6
(her word for it: i felt sick and i wanted the floor to swallow me)

the same words, said by a friend she trusts, might read: no_change / minor_positive
(her word for it: ok that is weird but it is just jimmy being jimmy)
```

The difference is entirely the recipient. The same sentence moves closeness differently depending on who said it and how the recipient feels about them - exactly what a recipient-POV read captures, and what a speaker-declared tone could never do.

5. Memory tie-in (your note: it is really a memory thing)

The reason plus the felt emotion write an emotion-tagged memory (task-96 emotional residue), so how that made me feel persists. Later recall of that person re-surfaces the feeling, not just the fact.

```
memory: { text: tyler snapped at me for blocking the counter and my stomach dropped - i felt small,
           importance: 4, tags: [tyler, shame, anxiety], emotion: {label: sad, intensity: 5} }
```

Reversibility Contract

How a backgrounded character can be *promoted* to full cognition, and *demoted* back, without ever becoming a different person. This is the rule that makes "same characters, two fidelities" safe instead of a source of retcons.

Related: [[trace-format]], and tasks 397 (scopes), 399 (background sim), 403 (unified memory).

Principle

One state model, two decision policies. There is exactly one `Player`, one graph, one clock, one set of vitals. Background and foreground differ *only* in where decisions come from:

- **Background:** deterministic/probabilistic rules over schedule + needs + traits. No LLM.
- **Foreground:** the LLM proposes goals/plans; the deterministic layer validates and executes.

Promotion is "start asking the character instead of their rulebook."
Demotion is the reverse. Nothing is materialized, re-rolled, or recreated —
so nothing can contradict.

What must stay continuously true (both fidelities)

While a character is backgrounded, the rule layer must keep updating the same fields the foreground layer reads. If a field is frozen during background, it is a reversibility violation.

Field	Why it must stay live
<code>current_area</code> + position	Promotion at a stale location is teleportation
<code>vitals</code> (all)	Hunger/thirst/energy/HP must be the real, decayed values
<code>conditions</code>	A backgrounded character can be wounded, ill, asleep
<code>inventory</code> / <code>equipped</code>	Objects must not duplicate or vanish across the seam
<code>relationships</code>	Coarse deltas are enough, but they must accrue
<code>memories</code> / trace	The history of what they did must survive
<code>activity</code> / <code>state</code>	"Asleep in the Sleeping Halls" is real state
<code>traits</code>	Acquired slowly; must not reset
<code>goals</code> / current plan	Promotion resumes the plan, it does not invent a new one

Allowed approximation (background only)

Background may be *coarse*, but must be **reconcilable**:

- Action timing may be scheduled in blocks rather than tick-exact.
- Paths may be resolved as "travel between areas with an ETA" rather than step-by-step, **provided the character is placed at a real area at the right tick** (no teleporting).
- Probabilistic outcomes are allowed **only** if seeded, so a span is reproducible.
- Numbers may move in whole units via the `vital_rates.change()` accumulator.

Background may never invent outcomes the foreground could not have produced (e.g. an item appearing in an inventory that was never taken).

Promotion handoff

On promotion the character must receive:

- Their **current state** (already real — no update needed).
- A **catch-up summary** built from the trace since they were last foreground (`engine/trace.summarize_window`), so the LLM knows what happened to them and can act as someone who lived it.
- Their **standing goal/plan**, if any.

The LLM's first foreground output may revise the plan; it may not rewrite the past.

Demotion handoff

On demotion the foreground span is rolled into the trace: the character keeps their state, and the *reasons* and *facts* of what they just did are recorded (see trace format). No memory is deleted; live memories stay in the memory store and the trace covers the mechanical span.

Invariants that must hold across any promote/demote

- Position continuity** — the character is in a real area before and after.
- Object conservation** — items are neither duplicated nor lost.
- Vital continuity** — no jumps in vitals, only real accumulated change.
- Relationship symmetry** — A's view of B and B's view of A stay consistent.
- Identity stability** — the opaque id (`task-316`) never changes on promotion.

- 6. **Determinism** — a background span replays identically from its seed.
- 7. **No orphans** — promotion never creates a second node for an existing character (guards against the duplicate-character-node class of bug).

Non-goals

- Pixel/tick-perfect motion while backgrounded.
- Identical LLM and rule outcomes — they will differ; they must not *contradict*.
- Making background characters as smart as foreground ones.

Open problem

Cross-zone roaming. A character travelling between zones belongs to no single zone. Background must model the transition as a scheduled event with an ETA and promote on approach, so arrivals are coherent rather than spawned.

Sky Widget — Reconciled Design (A)

Reference note reconciling the **qwen/GLM sky-stage mockups** (docs/design/sky-clock-mockup-v1.html , sky-clock-mockup-v2-real-moon.html) with the **roadmap 3.3** spec (docs/virtualWorld/dev_tasks/environment-roadmap.md) and the **implementation** in static/js/sky-scape.js + engine/weather_forecast.py .

Source materials

File	Source	What
sky-clock-mockup-v1.html	Qwen assistant	Full animated sky stage + slider (demo, no engine wiring)
sky-clock-mockup-v2-real-moon.html	GLM assistant	v1 + realistic moon physics (rise/set, elongation glare, phase terminator)
environment-roadmap.md §3.3	Tommy	Compact top-bar widget + World Sky editor panel (stage + forecast/override/date controls)
Time & Weather.md (canonical spec)	Team	Forecast engine, moon 30-day cycle, wind/humidity tables, trigger integration
sky-scape.js (this session)	Implementation	Port of the stages into an engine-driven module; top-bar + modal

What was adopted

From the mockups (display layer)

- **Sky stage gradient** (skyAt()): the v1 gradient keyframes ported directly (hour—color pair).
- **Sun arc**: position/animation logic from v1, with dimming via weather.
- **Moon arc**: v2's age = day/30 + moonrise = sunrise + age × 24h + moonset = rise + dayLen — correct real-lunar behavior. Used in the stage and the readout.
- **Season palettes**: v1's SEASON_STYLE map (spring/summer/autumn/winter hill colors, sunrise/sunset times).
- **Weather layers**: clouds, rain streaks, snow flakes, fog overlay, dimming — all ported from the mockup's WEATHER_LAYER table.
- **Reduced-motion** respect: low-OP computing; kept the transition: opacity .4s on the stage.
- **Moon phase chips**: MOON_CHIP emoji map from v2.

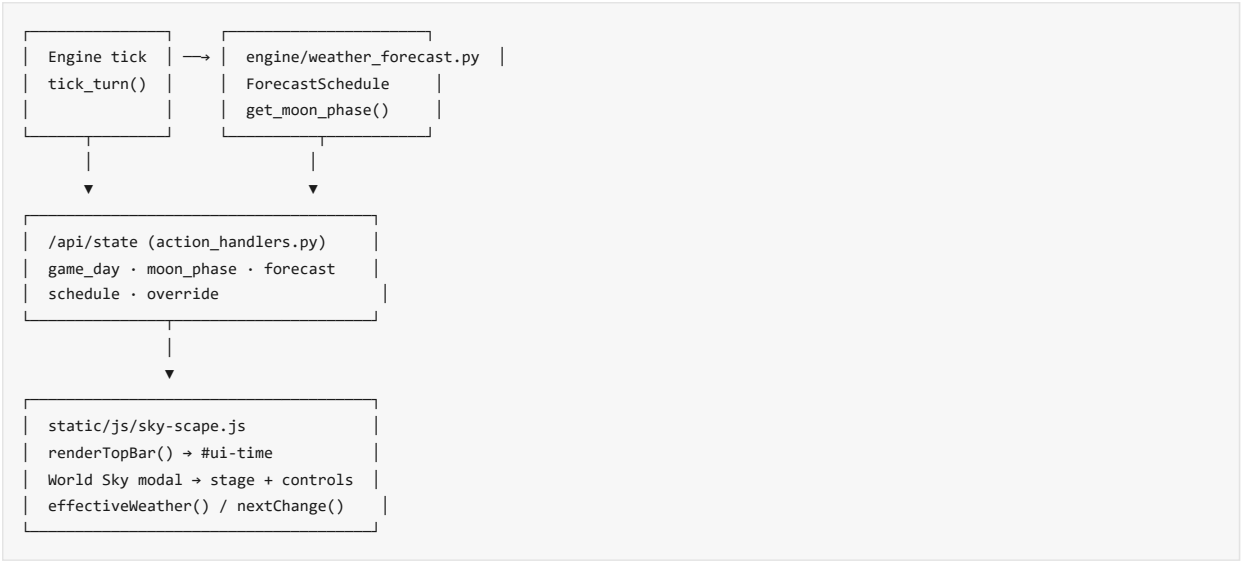
From the roadmap (product layer)

- **Top-bar widget**: renderTopBar(el, state) replaces the bare #ui-time clock with a compact line: 🕒 08:37 · Day 3 of 27 (Aug 2084) · 🌙 waning · 🌞 clear · 🌧 next change in 2h .
- **World Sky panel**: modal with the stage as the centerpiece, time skips (+15m / +1h / +1 day), weather override dropdown + duration + Set/Clear buttons, moon phase readout, forecast-next-change indicator, GM override indicator.
- **Engine drive**: EVERYTHING is read from /api/state : game_time , game_day/month/year , moon_phase , forecast_schedule , forecast_override . The mockup's independent chips are replaced by read-only derived displays (season from month, weather from effective forecast, moon from get_moon_phase()).
- **Forecast next-change** helper: nextForecastChange(state) computes the next entry offset from the current time, returning human-readable delta + weather.

What was left out (v1 scope cuts)

Mockup feature	Reason
Flow (animated hour-per-0.5s rAF)	Demo affordance; real panel uses clickable time skips that call engine APIs
Lightning flashes	Stormy weather layer only — lightning is a visual treat but adds no authoring value in v1
Tree silhouettes	Seasonal tree styling in the hills — low ROI; ground+hill is enough
Realistic moon terminator	v2's shadow disc per phase was nice but the emoji + readout is sufficient
Drag-to-set-time slider	v1's time slider; replaced by discrete skip buttons (+15m/+1h/+1 day)
Per-area override	<code>forecast_override</code> is global in v1; per-area is a future extension

Implementation architecture



Files changed / created (this session)

Engine

- `engine/weather_forecast.py` — NEW: `ForecastSchedule`, `get_moon_phase`, wind/humidity tables
- `virtual_world_engine.py` — `calendar_config`, `forecast_*` fields, `game_day/month/year` properties, `_forecast_tick()`, `_fire_time_triggers()`, `set_game_time()`, `set_game_date()`, `current_moon_phase()` (blood_moon aware)
- `engine/tick_manager.py` — `forecast_tick` hook, wind extinguish, wind energy drain, `on_turn_start/end` hooks, humidity social
- `engine/lighting.py` — moon_provider + moon night-light bonus (weather-gated)
- `engine/equipment_bonuses.py` — effective_temperature wind/humidity kwargs, aggregate wind_resistance/water_resistance
- `engine/environment_propagation.py` — wind multiplier on heat propagation
- `engine/effects.py` — import `WEATHER_HANDLERS`
- `engine/effect_handlers/environment.py` — weather/wind/humidity/transparent keys + adjust cycling
- `engine/effect_handlers/weather.py` — NEW: `set_time`, `set_date`, `set_weather`, `forecast_override`, `adjust_forecast`
- `engine/runtime_config.py` — `forecast.apply_scope` key
- `engine/serialization.py` — save/load `calendar_config`, `forecast_schedule`, `forecast_override`

Routes

- `routes/action_handlers.py` — `/api/state` exposes `game_day/month/year`, `moon_phase`, `forecast_schedule`, `forecast_override`
- `routes/settings.py` — `GET/POST /api/settings/forecast`, `POST /api/settings/forecast-override`

Frontend

- `static/js/sky-scape.js` — NEW: engine-driven sky widget (top-bar + modal stage + controls)
- `templates/index.html` — script include for `sky-scape.js`

Tests

- `tests/test_graph_batch.py` — 6 batch tests (from earlier session)
- `tests/test_engine_config.py` — baseline includes `forecast.apply_scope`

Trace Format (objective, code-written history)

The **trace** is a compact, append-only, per-character record of what actually happened, written by code — never by the LLM. It is the substrate for:

- first-person **memory** (the LLM summarizes a trace window into memory),
- **"what have you been up to?"** when you click a background character,
- **reversibility** on promote/demote (see [[reversibility-contract]]),
- and debugging long runs (an auditable objective record).

It is deliberately *not* the memory store. Memories are subjective, sparse, and LLM-authored. The trace is objective, mechanical, and free.

Entry schema

Each entry is a plain dict (JSON-friendly) on `player.trace_log`:

```
{
  "t": 4120,           // world tick
  "kind": "need",      // see kinds below
  "what": "became very hungry",
  "why": "needs:hunger", // reason tag, from the deciding layer
  "area": "Food Storage",
  "tags": ["need"],    // short labels for filtering
  "salient": false,    // survives rollup when true
  "delta": null        // optional small numeric payload, e.g. {"HP": -1}
}
```

Field rules:

- `t` — required, integer world tick (`world.time_ticks`).
- `kind` — required, one of the enumerated kinds below.
- `what` — required, short third-person objective phrase ("took 2 rations").
- `why` — a reason tag naming *why this happened* (see reason tags). Optional but strongly preferred; this is what makes background activity explainable.
- `area` — the area name at the time, when relevant.
- `tags` — 0–3 single-word labels (`need` , `combat` , `trade` , `travel`).
- `salient` — set true for things worth remembering (deaths, discoveries, conflict, relationship change, first encounters, completed plans).
- `delta` — optional; only for things a reader may want numerically (HP loss).

Kinds

kind	meaning	typical why
move	changed area / arrived	goal:travel , need:shelter
act	performed an action verb	goal:* , plan:*
need	a vital crossed a tier	needs:hunger
condition	condition gained/lost	react:* , env:*
encounter	another character involved	social:* , threat:*
observe	notable percept	sense:*
plan	plan/sequence started or completed	plan:*
death	character died (salient)	cause:*
promote / demote	fidelity boundary crossed	sim:fidelity

Reason tags

`why` names the layer that decided the action, so the reason survives even when the LLM that chose it is long gone:

- `needs:<vital>` — driven by a pressing need (hunger, thirst, rest).
- `goal:<name>` — serving a chosen goal/plan.
- `plan:<name>` — executing a standing plan (background rule tier or LLM).

- `social:<name>` — prompted by another character.
- `threat:<source>` — reactive to danger.
- `order:<character>` — following someone's instruction.
- `env:<factor>` — forced by environment/weather/area status.
- `llm:<reason>` — an LLM deliberation chose this (foreground only).

Granularity

The trace is not an action log for its own sake. Write at the level a person would actually remember or a reader would care about:

- Routine mechanical steps are **collapsed** ("crossed the camp to the store", not every way-node traversed).
- Needs are recorded only on **tier crossings** (e.g. crossed into "very hungry"), not every tick.
- Salient events are recorded **individually** and marked `salient`.

Retention and rollout

`player.trace_log` is capped (default 200 entries, mirroring the memory store). `engine.trace.rollup()` runs periodically and:

1. Drops the oldest non-salient entries first.
2. Compresses long runs of the same `(kind, why, area)` into one summary entry ("spent the day working in the Workshop", `kind: "plan"`).
3. Always keeps `salient` entries (bounded separately).

Trace → memory

Code writes the trace; the LLM writes memory. The bridge is a window:

```
engine.trace.summarize_window(player, since_tick) -> text lines
```

Used at natural boundaries (plan/sequence completion, promotion, end of day), the LLM turns that window into 1–3 first-person memories in the existing memory store. This is why objective trace must come first: LLM memory alone drifts (a recorded run had a character "remember" a waxwork man who was never there).

Serialization

`player.trace_log` is a list of plain dicts, written by `Player.to_dict()` and restored in `engine.serialization._deserialize_player`. Bounded, so it does not blow up save size.

TypeScript in the front end

Status: adopted. Toolchain wired, first module converted (`static/js/agent/rate-limiter.ts`). Migration is **incremental** — `.js` and `.ts` coexist, and the app keeps working at every step.

Why it can be incremental

The front end loads **classic** `<script>` **tags** and shares state through globals (`config`, `worldState`, `VW.PromptBuilder`, ...). TypeScript supports that directly: a `.ts` file with **no** `import / export` compiles to a plain script with the same top-level declarations. So one file can become TypeScript without touching its neighbours or the HTML.

The two configs

Command	Config	Purpose
<code>npm run build:ts</code>	<code>tsconfig.json</code>	Compiles <code>static/js/**/*.ts</code> → <code>.js</code> next to the source (same path the HTML already loads). <code>strict</code> , <code>noEmitOnError</code> , comments preserved.
<code>npm run typecheck</code>	<code>tsconfig.check.json</code>	Parses all JS + TS with <code>noEmit</code> . JavaScript is <i>not</i> type-checked unless a file opts in with <code>// @ts-check</code> , so the codebase can be tightened file by file.

`typecheck` passing today means every file is at least parseable and self-consistent; it will start reporting as files opt into `@ts-check`.

The load-order constraint (do not break it)

Converted files must **not** use `import / export`. Cross-file symbols come from ambient declarations in `static/js/types/*.d.ts` (currently `globals.d.ts`).

Two options land in `tsconfig.json` because TypeScript 7 removed them:

- `module: "esnext"` — `"none"` no longer exists. A file with no imports/exports still emits as a classic script, so this is a non-issue in practice.
- `alwaysStrict` was removed entirely. **Emitted files therefore start with `"use strict";`** — converted files get strict semantics automatically. That is a benefit, but it is a real behaviour change for the file, so convert deliberately (no sloppy-mode-only constructs).

Converting a file

1. Copy `foo.js` → `foo.ts` (same directory, same name).
2. Add types; widen globals in `static/js/types/globals.d.ts` if needed.
3. Keep the `@module/@contributes/@powers/@relates/@docs` header **in the .ts** — it is emitted into the `.js`, so the module index keeps working.
4. Delete `foo.js` (the build regenerates it).
5. `npm run build:ts`, then `npm run typecheck`, `node --check <file>`, `npm run lint`.

The emitted `.js` carries a header note that it is generated. **Never hand-edit a .js that has a sibling .ts.**

For symbols other code reaches through `window`, keep the assignment:

```
(window as unknown as { RateLimiter: typeof RateLimiter }).RateLimiter = RateLimiter;
```

Naming convention (enforced by review)

Cryptic single-letter names are not acceptable for domain values. Prefer the thing's name:

Don't	Do
<code>const v = Number(vitals[key])</code>	<code>const value = Number(vitals[key])</code>
<code>const T = window.VitalThresholds</code>	<code>const thresholds = window.VitalThresholds</code>
<code>v < T.WARNING</code>	<code>value < thresholds.WARNING</code>
<code>const n = Number(...)</code>	<code>const value = Number(...)</code>

A sweep renamed **163** such locals across 11 files (e.g. the prompt builder's `describeVital`, which is now readable). Short names are fine only where they are idiomatic and local — loop indices (`i`, `j`), coordinates (`x`, `y`).

Suggested conversion order

Leaf → core → views (views are last: lit-html templates are the hardest to type):

1. Leaves with no globals or one: `shared/dom-utils.js`, `shared/json-utils.js`, `shared/vital-color.js`, `agent/rate-limiter.js` **✓ done**
2. Logic modules: `agent/action-normalizer.js`, `agent/response-parser.js`, `agent/plan-tracker.js`, `agent/rate-limiter.js` peers

- 3. Services: `config.js`, `storage.js`, `world-state.js`, `api.js`, `llm-client.js`
- 4. Views/last: `inspector/*`, `ui/*` (they read `window.Lit`)

Add `// @ts-check` to a `.js` file once its ambient globals are typed enough — that starts type-checking it without a rename.

Guarding it

- `npm run typecheck` — add to CI alongside `npm run lint`.
- `python tools/js_module_index.py --check` — the module contract guard; it steps over a leading `"use strict"`; so emitted scripts still count as documented.

Goblin Scenario + Generation Scripts — Verification (2026-09-18)

Server: `http://127.0.0.1:4444` (verified listening, `/api/health` → running).

Verdict

Artifact	Works?	Notes
<code>tools/validate_scenario.py</code> on goblin scenario	✅ pass	Shallow check only — it did not catch the runtime bugs below.
<code>tools/build_scenario.py</code> (component build)	⚠️ after fix	Two real code bugs found and fixed; still blocked by an incomplete component set .
<code>tools/assemble_scenario.py</code>	⚠️ after fix	Was dumping the entire library; now gated by constraints.
Goblin scenario in the running engine	❌ broken	Toolless/scene runtime shows duplicate characters and the player as a stranger.

The validator passing is misleading: the scenario loads (`{ "status": "success" }`) but is functionally wrong at runtime.

Bug 1 — `player.current_area` must be the area *display name*, not the node id

- Goblin scenario originally stored `"current_area": "area_camp_entrance_trail"`.
- Engine resolves areas by **name** first (`engine/room_perception.py:resolve_area_node`).
- Result before fix: `GET /api/scene/player_human_explorer` returned `name: "area_camp_entrance_trail"`, `desc: ""`, `ways: []`, and `in` edges pointed at non-existent `area_area_camp_entrance_trail`.
- Known-good scenarios store names: `pin.es.json` uses `"apartment 2e"`, not `area_*`.
- Severity:** total — the starting scene was empty (no description, no exits, no items).
- Status: the canonical file has since been updated to display names (all 23 resolve).

Bug 2 — Character nodes are duplicated (still present)

- Scenario authors 23 `character_*` nodes **and** a `players` block with 23 entries.
- The engine treats `players` as authoritative and creates its own `player_<key>` anchors.
- On load there are **46 character nodes** (23 `character_*` + 23 `player_*`).
- Runtime evidence (current canonical, no-persist load) — starting area contains:
`id=player_human_explorer` <- the active player, shown as a separate stranger
`id=Kiala id=player_Kiala` <- duplicate
`id=Belne id=player_Belne` <- duplicate
- Canonical convention (`pin.es.json`): exactly one `player_<key>` node per player key, node name == player key, **no** `character_*` nodes.
- Extra failure: the human explorer's node name is `"Human Explorer"` but the player key is `player_human_explorer`, so the scene's self-exclusion fails.

Bug 3 — `build_scenario.py` derived IDs from names → punctuation drift

- `_slugify("Chief's Pit")` → `area_chief_s_pit`, but components reference `area_chiefs_pit`.
- Crashed with `Character character_thrazz` references missing area `'area_chiefs_pit'`.
- Fix applied:** component filename stem is now the authoritative node id (`load_components_with_ids()` in `tools/build_scenario.py`).

Bug 4 — `build_scenario.py` normalized triggers twice, first with an empty node set

- `triggers = [normalize_trigger(t, set()) ...]` ran before all node ids existed, so any trigger with a `target` raised `targets missing` node.

- **Fix applied:** single normalization pass after `all_node_ids` is computed.

Bug 5 — `assemble_scenario.py` dumped the whole library

- With `--constraints '{}'` it imported **456 items and 45 characters** into the seed, producing 474 items / 68 characters / 570 edges and 4 validation errors (characters with no description).
- **Fix applied:** item/character/way imports are now gated on explicit `items / characters / ways` constraint targets and stop at the target.
- Re-run result: seed preserved exactly (30 areas, 29 ways, 18 items, 23 chars, 114 edges).

Still open — component set is incomplete

`tools/build_scenario.py` now fails only because `tools/scenario_components/` is missing areas that its ways/triggers reference:

```
Way way_entrance_to_pit references unknown area(s): area_camp_entrance, area_chiefs_pit
Trigger logic_trigger_examine_camp_trail targets missing node 'area_camp_entrance_trail'
```

Only `area_chiefs_pit.json` is present, but the way needs `area_camp_entrance` and the trigger needs `area_camp_entrance_trail`. Either add those area components or remove the dangling way/trigger components.

Remaining remediation for the scenario to actually work

1. Delete the 23 authored `character_*` nodes (keep the `players` block authoritative), or rewrite them to `player_<player_key>` matching the player keys with node name == key.
2. Align the human player: player key and character node name must match (`player_human_explorer`), otherwise the scene shows the player as a stranger.
3. Re-verify with `GET /api/scene/player_human_explorer` (expect: 1 explorer excluded, no duplicates) and `GET /api/scene/Thrazz` for the camp interior.
4. Note `"persist": true` in the scenario: loading it writes scenario files to disk, and a peer session (`main`) is active in this project — coordinate before bulk edits.

Files changed by this verification

- `tools/build_scenario.py` — filename-as-id; single trigger normalization pass.
- `tools/assemble_scenario.py` — constraint-gated library imports.
- Report: this file.

Goblin Scenario Work Session — 2026-09-18

Session summary

- Reviewed goblin scenario plan and existing scenario JSON.
- Ran validation on `data/scenarios/kraktooth_goblin_camp.json`.
- Fixed all reported validation issues and rewrote the scenario file.

Files changed

- `data/scenarios/kraktooth_goblin_camp.json`
- `PATCH_NOTES.md`

Validation fixes applied

- Added `pass_message` to 29 ways.
- Added `max_weight_capacity` to `item_water_skin`.
- Added `equip_slots` to `item_broken_shield`.

Validation result

```
python tools/validate_scenario.py --input data/scenarios/kraktooth_goblin_camp.json passed.
```

Current scenario counts

- Areas: 30
- Ways: 29
- Characters: 23
- Items: 18
- Triggers: 11
- Edges: 114

Open follow-ups

- Expand item placement beyond the current 18 items.
- Add opening discovery/area-entry triggers.
- Wire goblin schedules/behaviors.
- Load in-engine and smoke-test movement/turn flow.

Kraktooth Goblin Camp — Sandbox Scenario Plan (v3)

1. Premise

A living, breathing goblin camp embedded in a contested borderland. The session is not a story with an ending; it is an ongoing D&D-style sandbox slice of life centered on the KrakoTooth tribe and the individuals orbiting them.

The tone is grounded fantasy with respectable worldbuilding — survival hardship, small politics, resource scarcity, personal grudges, rituals, petty theft, traps, bad food, and occasional violence. Nothing is railroading toward a finale; events emerge from the environment, relationships, and immediate needs.

The player is a human in the forest, free to move anywhere in the territory and watch the world evolve.

2. Core Themes

- Survival vs. comfort
- Tribe loyalty vs. personal ambition
- Usefulness vs. status
- Scarcity and improvisation
- Attention as currency
- Territory, routes, and secrets
- Civilization as opportunity, not safety

3. Viwo Scenario Shape

Per `ScenarioCreationGuide.md`, the engine reads:

- `graph.nodes` for areas, ways, items, characters, logic triggers
- `graph.edges` for connections, `in`, `triggers`, `equipped`, `carrying`
- `players` as authoritative character state
- `world_lore` injected into every agent prompt
- Runtime state: `active_player`, `time_ticks`, `clock_start_*`, `turn_number`, `narration_mode`, `ghost_mode`

This plan is therefore organized around: areas/ways graph, character roster, item/trigger tables, lore blocks, and runtime config.

4. Time and Weather

- Timescale: **1 minute per turn**
- Initial simulation: **1 week** (10,080 turns)
- `time_per_tick_minutes` : 1
- `clock_start_hour` / `clock_start_minute` : TBD
- Days and nights cycle
- Weather shifts: rain, cold, heat, fog
- Seasons change resource availability
- Some areas become inaccessible during bad weather

- Goblin activity patterns shift with light and weather

5. Map and Graph Plan

5.1 Camp Areas (nodes)

These become `area_*` nodes.

1. `area_camp_entrance` — main entrance, traps, smells of smoke
2. `area_chiefs_pit` — central fire, meetings, feasting
3. `area_chiefs_den` — leader room, trophies, war plans
4. `area_sleeping_halls` — hammocks, furs, personal belongings
5. `area_nursery` — young goblins, caretakers, basic supplies
6. `area_cooking_area` — fires, meat racks, pots, spices
7. `area_food_storage` — dried meat, mushrooms, barrels, spice shelves
8. `area_workshop` — repairs, crafting, tinkering
9. `area_scrap_pile` — junk, stolen goods, parts, valuables in trash
10. `area_training_pit` — sparring, weapon practice, cages
11. `area_prison_pens` — slaves, captives, trade goods
12. `area_shaman_lair` — rituals, herbs, strange artifacts
13. `area_healing_area` — basic medicine, bandages, potions
14. `area_scouting_rooms` — maps, reports, tracking info
15. `area_mine_access` — tunnels to lower caves, stone, ore
16. `area_storage_caves` — wood, stone, clay, spare supplies
17. `area_animal_pens` — wolves, captured beasts, livestock
18. `area_water_source` — underground lake, fishing, water storage
19. `area_waste_disposal` — refuse, bones, latrine downstream
20. `area_side_tunnels` — expansion, exploration, unknown routes

5.2 Surrounding Areas

21. `area_deep_forest`
22. `area_raven_river`
23. `area_murk_lake`
24. `area_blackmarsh`
25. `area_old_dwarven_ruins`
26. `area_abandoned_farm`
27. `area_eldenford_village`
28. `area_human_road`
29. `area_northern_hills`
30. `area_camp_entrance_trail` — forest approach to camp

5.3 Ways (edges)

Each connection is a bidirectional `way_*` node with `connection` edges.

Examples:

- `way_camp_entrance_to_forest` between entrance and trail
- `way_forest_to_raven_river`
- `way_forest_to_blackmarsh`
- `way_forest_to_old_dwarven_ruins`
- `way_forest_to_abandoned_farm`
- `way_forest_to_eldenford_edge`
- `way_raven_river_to_camp_tunnel`
- `way_side_tunnels_to_mine_access`
- Internal camp ways between numbered areas

Some ways should have:

- `hidden` : true for secret routes
- `needs_open.enabled` : true for blocked passages
- `requires` : climb, crawl, jump
- `cost.time` and `cost.energy` : travel cost in minutes and energy

6. Runtime Config

- `active_player` : human explorer
- `clock_start_hour` : TBD
- `clock_start_minute` : TBD
- `time_per_tick_minutes` : 1
- `turn_number` : 0
- `ghost_mode` : false
- `narration_mode` : none
- `current_area` : starting forest area
- `game_time` : derived from clock start

7. Character Roster

All characters go in `players` . Each needs:

- `personality`
- `description` / `base_description`
- `stats`
- `vitals`
- `skills`
- `traits`
- `tags`
- `emotion`
- `current_area`
- `autonomy` , `simple_npc` , `npc_behavior` , `npc_state` , `npc_action_interval`
- `relationships`
- `memories`
- `decay_rates`
- `interest_tags`

7.1 Human Player

- `simple_npc` : false
- `autonomy` : false
- `current_area` : `area_camp_entrance_trail`
- Starting vitals, skills, equipment TBD

7.2 LLM-Agent Goblins

These should be `simple_npc: false` to use the full agent engine:

- Thrazz
- Zikka
- Vekka
- Mikka
- Gribba
- Rikka
- Krikka

7.3 Additional Goblin Characters

These can be `simple_npc: true` or LLM agents depending on desired simulation cost:

- Arix
- Kiala
- Belne
- Leslie (optional)

7.4 Human Villagers (Eldenford)

- Village Elder
- Blacksmith
- Merchant
- Road Guard Captain
- Farmer

7.5 Animals / Creatures

These should almost certainly be `simple_npc: true` with `npc_behavior` patterns:

- Rag-Tail (wolf pack alpha)
- Old Iron-Back (bear)
- Tusker (boar)
- Shadow-Pelt (worg leader)
- Silver-Talon (raven)
- Croak-Mother (frog)

`simple_npc: true` characters skip LLM calls and act via backend `tick_turn`, which is appropriate for animals and background villagers.

8. Items and Triggers

Items are `item_*` nodes with properties including:

- `actions`
- `description`
- `current_state`
- `uses`
- `weight`
- `hidden`
- `tags`
- `action_costs`
- `skill_check`
- `contents` (for containers)
- `equip_slots`
- `light_level`
- `damage_dice`
- `damage_type`
- `defense`
- `heating_rate`
- `insulation`
- `target_temperature`
- `temp_range`

8.1 Item Placement

Items are placed via `in` edges to areas or other items:

- `item_rusty_hatchet` → `area_training_pit`
- `item_bread` → `area_cooking_area`
- `item_backpack` → `area_sleeping_halls`
- `item_healing_herbs` → `area_healing_area`

8.2 Triggers

Triggers are `logic_trigger` nodes with `triggers` edges from items, areas, or ways.

Key trigger types to use:

- `on_examine` / `on_search` for discovery
- `on_take` / `on_drop` for inventory events
- `on_eat` / `on_drink` for consumables
- `on_use` / `on_activate` for magic items and tools
- `on_light` / `on_toggle_off` for light sources
- `on_enter` for area events
- `on_tick` for periodic effects
- `on_open` / `on_close` for containers and ways
- `on_speech` for social reactions

8.3 Example Items to Build First

- Basic weapons: rusty hatchet, spear, club, bow
- Tools: hammer, saw, pry bar, torch
- Food: dried meat, bread, mushrooms, berries
- Drink: water skin, mushroom tea
- Light: torch, lantern

- Containers: backpack, chest, basket
- Keys and locked containers
- Scrap items for the workshop
- Shamanic items: herbs, bones, charms
- Traps

9. Magic System (Viwo-shaped)

In Viwo, magic is modeled as:

- Items with `magic` tag
- `on_activate` / `on_use` triggers that consume `Mana` or limited `uses`
- `adjust_vital` effects for costs
- `damage`, `apply_condition`, `heal`, `teleport`, etc. for effects
- Spells can be learned from scrolls via `on_read` → `spawn_item`

Goblin magic in this scenario:

- **Create Flame**: spell-item with `on_activate` `damage/fire` + `light`
- **Spark**: cheaper ignition variant
- **Snuff**: extinguishes `light` sources in area
- **Find Water**: reveals hidden water source or adjusts environment
- **Warn**: applies `frightened` condition or adds sensory message
- **Blight**: withers plants/food in area
- **Grow**: accelerates plant growth
- **Fool**: minor illusion — message-based sensory effect
- **Stick**: applies `stuck` condition to target
- **Break**: weakens item/way, sets state to `jammed` or `broken`

10. World Lore

`world_lore` is injected into every agent prompt. Key lore blocks:

10.1 Tribe Lore

- Kraktooth tribe history and reputation
- Current leadership structure
- Recent events affecting the camp

10.2 Region Lore

- Geography: forest, river, marsh, hills, ruins
- Human village: Eldenford attitudes and defenses
- Dwarven ruins: what remains and what dangers
- Trade routes and patrol patterns

10.3 Magic Lore

- Goblin shamanic practices
- Known magical sites or residual magic
- Human attitudes toward goblin magic

10.4 Threat Lore

- Known predators and their patterns
- Human patrol schedules and routes
- Rival tribes or dangers

11. Starting State

11.1 Time

- Start at dawn: `clock_start_hour: 6`, `clock_start_minute: 0`
- `game_time`: "06:00:00"
- `turn_number`: 0
- `time_ticks`: 0

11.2 Player Spawn

- `current_area`: `area_camp_entrance_trail`
- Basic survival gear: torch, knife, water skin, bread
- knows general region, not specific locations

11.3 Camp State

- Normal operations
- Food reserves: moderate
- Tools: functional but worn
- No immediate threats
- Weather: clear

11.4 Relationships

- Tribal relationships at baseline
- Outsiders have default attitudes
- Player unknown to most

12. What Makes This Sandbox Work in Viwo

- No main quest, no victory condition
- LLM agents have personal drives, not plot hooks
- Simple NPCs provide ambient life via backend behaviors
- The camp is a graph system with interdependent needs
- Triggers create emergent events from item use, area entry, and time
- World lore keeps all agents grounded in shared context
- Outside pressure changes priorities
- Small events compound via triggers and relationships
- The player can intervene, observe, or ignore
- Characters respond to behavior through relationship shifts and memories

13. Open Questions for Viwo Implementation

1. **Agent vs Simple NPC split:** Which goblins should be full LLM agents, and which are `simple_npc: true`? I suggest the 7 core goblins as LLM agents, others/animals/villagers as simple NPCs. Agree?
2. **Player spawn area:** Is `area_camp_entrance_trail` the right start, or somewhere else in the forest?
3. **Starting clock:** Dawn at 06:00, or something else?
4. **Starting inventory:** Basic explorer kit, or something more specific?
5. **Initial relationships:** Any pre-existing relationships among NPCs at spawn, or all neutral/unknown?
6. **Trigger density:** Should the opening area have many discovery triggers, or sparse and revealed through play?
7. **Simple NPC behavior:** Should villagers have `wander` / `guard` / `patrol` patterns, or mostly `stationary`?
8. **Animal behaviors:** Which animals should `hunt`, `flee`, or `wander`?
9. **Magic availability:** Are there spell items in the world at start, or only improvised magic via shamanic items?
10. **Content boundaries:** Any content the scenario should avoid or tone down?

14. Implementation Order

1. Finalize agent/simple-NPC split and starting clock
2. Build graph skeleton: all area nodes and way edges
3. Define starting areas and player spawn
4. Create player character JSON block
5. Create all NPC/animal character blocks
6. Write `world_lore` blocks
7. Place core items in starting areas
8. Write discovery triggers for starting area
9. Add 5–10 opening triggers for emergent first-turn events
10. Validate against `ScenarioCreationGuide.md` shape
11. Load in engine and verify area connections, turn queue, and basic movement

Enhanced Item Creation: Multi-Target Placement, Trigger Generation, Prompt Preview

Filed: 2026-07-16

Summary

The "Create New Item" modal currently only places items in rooms and its AI generator doesn't parse triggers. This spec extends the modal to support placing items in rooms, containers, or character inventories; fixes trigger generation; adds an editable prompt preview; and updates all LLM generator prompts with comprehensive trigger documentation.

1. Target Type Selector

Current: Single "Target Area" dropdown in the create modal.

New: Replace with a radio group for target type, then a dynamic select list.

Radio	Select shows
 Area	All rooms from <code>worldState.rooms</code>
 Container Item	All item nodes (excluding current) from graph
 Character	All nodes of type <code>character</code> from graph

When the user picks a type, the select list updates to show matching targets.

AI Context

The target's description is passed to the LLM system prompt:

- **Area:** `worldState.rooms[areaName].description` (already implemented)
- **Container:** `worldState.getNode(containerId).properties.description`
- **Character:** `worldState.getNode(charId).properties.description || worldState.getNode(charId).name`

The prompt line added: `\nThis item will be placed in/inside/carried by "${targetName}": ${description}`

Backend (`/api/build/item`)

The `POST /api/build/item` endpoint currently creates a `location` edge to a room. Extend to:

- `container` field → create `contains` edge (item → container)
- `character` field → create `carried_by` edge (item → character)
- If `room` provided → existing `location` edge behavior (unchanged)
- Only one target type is allowed per request

2. Trigger Generation Fix

Problem: `main.js:generateWithAI('item')` includes triggers in the format hint (line 335) but the parse step (lines 386-389) discards them.

Fix: Add trigger parsing to the item branch:

```
// After setting basic fields
const triggersField = document.getElementById('item-triggers-json');
if (triggersField && data.triggers) {
  triggersField.value = JSON.stringify(data.triggers);
}
```

System prompt update (`main.js`, line 318): Replace the generic "Respond with raw JSON" with the same detailed trigger/container documentation that `item-library.js` already has in its `improveWithAI` prompt.

3. Prompt Preview/Editor

UX: A small button  next to the "Generate" button in the create modal and item library.

On click, opens a modal showing the full assembled prompt in an editable textarea:

```
[System]
You are a procedural content generator...
[Context]
...
[User prompt]
...
```

The user can edit the text before sending. When they click "Send", the edited text replaces what would have been sent.

Implementation:

- Assemble the prompt (system + user messages) the same way `generateWithAI` does
- Display in a textarea
- On "Send", parse the textarea to extract the user message part (everything after the last `[User prompt]\n` line, or use a separator)
- Pass the edited user message + original system message to `llmClient.chat()`
- If the user edited the system message too, use the modified version

Simple approach: Prepend `[System]\n{systemMsg}\n\n[Context]\n{context}\n\n[User prompt]\n{prompt}` in a full-screen textarea modal. On send, extract the portion after `[User prompt]\n` as the user message. The system message stays as originally assembled (user edits only the user portion). The modal has:

- The editable textarea
- "Send" button → uses the edited user message
- "Cancel" button → closes modal, no change
- "Copy" button → copies full prompt to clipboard

4. LLM Generator Prompt Audit

All AI generation points need comprehensive, consistent trigger documentation:

Location	Function	Has triggers?	Has full docs?
<code>main.js</code>	<code>generateWithAI('item')</code>	Format hint only	No
<code>item-library.js</code>	<code>generateWithAI()</code>	Format hint + basic	No (lacks full docs)
<code>item-library.js</code>	<code>improveWithAI()</code>	Full docs + examples	Yes

Action: Extract the trigger/effect/condition documentation from `item-library.js:improveWithAI` (lines ~930-987) into a shared file `static/js/prompt-docs.js`:

```
// prompt-docs.js — registered before main.js and item-library.js
window.VW = window.VW || {};
VW.PromptDocs = {
  ITEM_GENERATION_SYSTEM: `...trigger docs...`
};
```

Insert in load order between `api.js` and `agent-engine.js` in the HTML template.

Both `main.js:generateWithAI` and `item-library.js:generateWithAI` reference `VW.PromptDocs.ITEM_GENERATION_SYSTEM` instead of duplicating the prompt text.

5. Files Affected

- `static/js/main.js` — trigger parsing, system prompt docs, target type selector HTML, prompt preview, `generateWithAI` refactoring
- `static/js/item-library.js` — updated system prompt in `generateWithAI`, prompt preview
- `app.py` — extend `/api/build/item` for container/character targets
- `static/js/api.js` — update `createItem()` to send new fields
- `docs/superpowers/specs/2026-07-16-enhanced-item-creation.md` — this spec

6. Error Handling

- If no target selected → alert "Select a target"
- If container/character doesn't exist in graph → backend returns 404

- Backend validates only one target type per request

7. Testing

- Manual: create items targeted at room, container, character via both create modal and library
- Manual: AI generate with triggers → verify triggers field is populated
- Manual: prompt preview → edit and verify modified prompt is sent

Entity-Indexed Memory System

⚠️ **SUPERSEDED (task-178, 2026-08-04)**. The EntityIndex / MemoryStore / world_knowledge described in this doc were **deleted** in the memory unification. Memory is now ONE store, Player.memories[], on the backend; the agent writes one LLM-generated reaction memory per turn via the react prompt's memory field. Spatial recall (the "KNOWN ROUTES" block) now comes from engine/spatial_memory.py (BFS over visited_areas + the real graph). See docs/virtualWorld/AI & Narration/Memory System.md and docs/virtualWorld/dev_tasks/done/characters/task-178-unify-memory-systems.md. This doc is kept as a historical record only.

Problem

Characters repeatedly "re-discover" the same information (the fireplace is cold, the painting shows the Valerius family) because the memory system stores raw action logs instead of entity-linked subjective takeaways. There is no persistence of *known facts* — every turn the LLM sees the room from scratch.

Architecture: Two-Tier Memory

Tier 1: Entity Index (persistent knowledge)

Every room, item, exit, and notable entity the character encounters gets an entry in CharacterEntityIndex. Entries do **not** decay.

```
EntityIndexEntry {
  entity_id: string,      // e.g. "room:living_area", "item:fireplace"
  type: "room" | "item" | "door" | "player" | "concept",
  properties: {
    exits?: string[],     // room: ["kitchen via swinging door"]
    items?: string[],     // room: ["fireplace", "painting"]
    states?: string[],    // entity-level: ["cold and dark", "has ash"]
    interactions?: string[], // item: ["examined", "tried to light"]
    location?: string,    // item: "Living Area"
    last_seen_tick: number,
    visited_count: number, // areas only
  },
  memory_ids: string[],  // references into episodic store
}
```

Storage

The entity index lives on the **frontend** (in-memory, persisted to localStorage) and on the **backend** (serialized on the Player object, saved/loaded with game state).

On every action result:

1. Parse the action and result text for entity names (areas, items, exits)
2. Extract state information ("cold", "dark", "contains X", "leads to Y")
3. Create or update entity index entries
4. Link new episodic memories to entity IDs

Tier 2: Episodic Memory Store

Each action produces **two** memory records:

1. **Raw memory** — the action + result text, entity-linked, importance based on action type
2. **Subjective memory** — the character's inner monologue from the reaction phase, entity-linked, importance +1

```

EpisodicMemory {
  id: string,
  text: string,          // the subjective interpretation or raw log
  subjective: boolean,   // true if from reaction-phase inner monologue
  type: "discovery" | "observation" | "action" | "conversation" | "reflection",
  tick: number,
  room: string,
  entity_ids: string[],  // ["room:living_area", "item:fireplace"]
  importance: number,    // 1-10
  embedding: number[],   // 768-dim vector (client-side only)
  decay: number,         // 0 = fresh, incremented over time
}

```

Memory Decay

- Importance drifts down by ~1 per 50 ticks for episodic memories
- Below importance 2: only retrievable with very high vector similarity
- Entity index: **no decay** (known facts persist)
- Frequently retrieved memories decay slower (reinforcement via recency boost)

Spatial Reasoning

The entity index builds a local map from areas the character has visited:

```

guest_bedroom → upstairs hallway
upstairs_hallway → guest bedroom, living room
living_area → kitchen, study, upstairs hallway, front door
kitchen → living room, cellar

```

When the LLM needs to decide where to go, BFS over the character's known connections computes reachable destinations. The result is injected as:

```

=== KNOWN ROUTES FROM HERE ===
Kitchen (has water, kindling, bread): go Upstairs Hallway → downstairs to Living Area → through swinging door

```

If the character has no goal, only adjacent areas with notable properties are shown.

Retrieval & Prompt Injection

The decision prompt gets three memory sections:

1. **KNOWN FROM HERE** — entity index properties for current room + BFS routes to interesting places
2. **I REMEMBER** — top 3-5 episodic memories via hybrid search:
 - Vector similarity (semantic match to current context)
 - Entity overlap (memories linked to entities in current room / recent thought)
 - Importance × recency boost
3. **RECENT** — the last 2 raw actions (for immediate context only)

Memory context is injected into **observation** (spatial only) and **decision** (spatial + episodic) prompts. Not in reaction or system prompts.

Data Flow

```

Action → Engine returns result
  → Store raw memory (entity-linked, importance=3-6 based on action type)
  → Extract entities from action + result → update entity index
  → Reaction phase: LLM generates inner monologue
  → Store subjective memory (entity-linked, importance = raw_importance + 1)
  → Next turn: build context from entity index + episodic RAG → LLM prompt

```

Implementation Order

1. **EntityIndex class** (client-side, memory-store.js or new entity-index.js)
 - Schema, CRUD, persistence to localStorage + server sync
 - Entity extraction from action text
 - BFS route finding

2. **Backend entity index** (Player model + engine)
 - Add `knowledge` dict to Player
 - Add API endpoints for CRUD
 - Serialize/deserialize in `to_dict/load_from_dict`
3. **Modified memory storage in agent-engine.js**
 - Store raw + subjective memories with entity links
 - Create entity entries on action results
4. **Modified `_buildMemoryContext`**
 - Replace flat "RECENT EVENTS" with entity-indexed spatial + episodic sections
 - Hybrid retrieval scoring
5. **Memory decay** (background tick processing)
6. **Cleanup** — remove old redundant systems (`world_knowledge`, legacy `_memory`, duplicate `MemoryStore` storage)

Save/Load Architecture

Date: 2026-07-16
Status: Draft

Summary

Separate runtime saves from scenario templates. Two save types:

1. **Save Game** — full runtime snapshot to `saves/` directory (resume progress)
2. **Save Scenario** — authorial content back to the source scenario file (preserve world-building)

Data Model

Save Game (`saves/<name>_<timestamp>.json`)

Full `VirtualWorld.to_dict()` output (minus the removed `player` duplicate field).
Contains everything: graph, players with all runtime state, lore, time, logs.

Save Scenario (source scenario file)

Authorial content only. Same as `to_dict()` but strips clearly transient artifacts:

Preserved (author may set these)	Stripped (play artifacts only)
graph (areas, items, ways, edges)	game_log
world_lore	turn_events
player vitals (custom starting HP etc.)	recent_hearing (per player)
player emotion	speech_log
player memories (starting lore dumps)	
player inventory (start with items)	
player state (start unconscious)	
player stats/skills/traits/personality	
time_ticks, turn_number (start at dawn)	
clock settings, ways, item_registry	
ghost_mode, narration_mode	

Files

Backend

File	Change
<code>virtual_world_engine.py</code>	Add <code>_scenario_source</code> field. Add <code>to_scenario_dict()</code> method that returns <code>to_dict()</code> minus transient fields.
<code>app.py</code>	Rename <code>_save_world_template</code> -> <code>_save_scenario</code> . Lore endpoints call <code>_save_scenario</code> . Add save game CRUD endpoints. Add save-scenario endpoint. Auto-detect source on startup.

Frontend

File	Change
static/js/api.js	Add <code>saveGame()</code> , <code>loadGame()</code> , <code>listSaveGames()</code> , <code>deleteSaveGame()</code> , <code>saveScenario()</code>
static/js/main.js	Wire toolbar buttons for save/load scenario and save/load game
static/js/inspector.js or new widget	Load Game modal — list saves, click to load, delete

API Endpoints

Save Game CRUD

GET /api/save-games	→ [{name, scenario, timestamp, tick, turn, player}, ...]
POST /api/save-game	body: {name?: string} → saves to saves/<name>_<ts>.json
POST /api/load-game/<filename>	loads save file, replaces world state
DELETE /api/save-game/<filename>	deletes a save file

Scenario Save

POST /api/save-scenario	writes clean scenario data to source file
-------------------------	---

Implementation Order

1. Rename `_save_world_template` -> `_save_scenario` in app.py; update lore endpoints
2. Add `to_scenario_dict()` to engine
3. Add `_scenario_source` tracking to engine + app startup
4. Add save game CRUD backend endpoints
5. Add frontend API methods
6. Add toolbar buttons + Load Game modal

Vitals/Needs System Redesign

Date: 2026-07-16

Status: Draft

Summary

Overhaul the vitals/needs system to make each vital mechanically meaningful, add new interaction loops, and introduce progressive LLM insanity prompts at low Sanity.

Change List

1. Energy = 0 → Unconscious

Before: Energy=0 → HP -1/turn

After: Energy=0 → player state flips to "unconscious", `state_timer` = 5

- While unconscious: no vitals decay (body preserves), Energy recovers at +3/turn (same rate as sleeping)
- State timer ticks down each turn; when it hits 0, player wakes with Energy = 10-15
- Other NPCs can wake the player early with a "revive" action
- Remove the HP-drain-at-zero from Energy entirely
- Narrative consequence is incapacitation (collapse from exhaustion), not death

2. Hunger/Thirst = 0 → HP Drain (adjusted numbers)

Before: Hunger=0 → HP -2/turn, Thirst=0 → HP -5/turn

After:

- Hunger=0 → HP -1/turn (~20 hours to die from full HP at 5 min/turn)
- Thirst=0 → HP -2/turn (~10 hours from full HP)
- Both=0 → HP -3/turn (~6.6 hours from full HP, compounding)

This puts starvation as a multi-day threat and dehydration as more urgent.

3. Sanity → HP Drain + LLM Insanity Tiers

Before: Sanity=0 → HP loss, death labeled "madness"

After: Keep death path. Add progressive LLM prompt injection:

Sanity Range	Label	LLM Prompt Injection
75-100	Sound	Normal. Emotion unaffected.
50-75	Unsettled	"Something feels wrong. You can't shake the feeling you're being watched." Emotion drifts toward fear/paranoia .
25-50	Strained	Mid paranoia: "You hear faint whispers... was that shadow always there?" Emotion drifts toward irritability/anger .
10-25	Fractured	Strong delusions: "Reality feels thin." + sensory hallucinations. Emotion drifts toward rage/aggression .
0-10	Broken	"You no longer trust your own mind." Random false memories, garbled room data (wrong items, wrong exits). Emotion locked to frenzy/panic .

LLM context: New `_buildInsanityContext(sanity)` function in `agent-engine.js` generates tier-appropriate injection text. Appended to the prompt before each `step()`. At < 25, the room description sent to the LLM may be randomly garbled (missing items, phantom items, wrong exit names).

4. Bladder ↔ Thirst → Hygiene Cycle

Before: Bladder decays at -1/turn regardless, Hygiene at -1/turn regardless

After:

Bladder decay rate scales with Thirst:

- Thirst > 75: Bladder -3/turn
- Thirst 50-75: Bladder -2/turn
- Thirst 25-50: Bladder -1/turn
- Thirst < 25: Bladder -0/turn

When Bladder hits 0 → Hygiene -30 (accident).

Relieve action:

- Use a bathroom/latrine/toilet item → Bladder resets to 100, clean
- Relieve in place → spawns `item_puddle` (or similar) in current room with `tags: ["filth", "stench"]` and environmental effect `stench: "urine"` → ongoing Hygiene decay for anyone in the room
- Puddle can be cleaned (use a rag/mop) or dries after N ticks

5. Hygiene → NPC Perception

Before: No social consequence

After:

- NPC reaction modifier: $-\text{floor}((100 - \text{hygiene}) / 20)$ → -0 at 100, -5 at 0
- LLM prompt injection when interacting with a low-hygiene character: "You notice {name} smells terrible / looks unkempt" at Hygiene < 50

6. Social → Conversation Gain

Before: Baseline -1/turn, environment alone -1, with others +1, perfume +1

After:

- Talking to another character (including NPC social actions): Social +2 per turn of conversation
- Talking to self (alone, muttering): Social +1 (slower but prevents isolation despair)
- Remove the current "alone = extra -1" environmental penalty (covered by Entertainment model instead)
- Baseline decay stays -1/turn

7. Entertainment → Examine/Use Gain

Before: Only baseline -1/turn and alone penalty

After:

- Successfully examining a new item: Entertainment +2
- Using an item: Entertainment +1
- Discovering a new room: Entertainment +5 (first-time bonus, tracked by a per-character set of visited areas)
- Baseline decay stays -1/turn

8. Social + Entertainment → Sanity Link

Before: Being alone → Sanity -1

After:

- Social < 50: Sanity -1/turn
- Social < 25: Sanity -2/turn
- Entertainment < 50: Sanity -1/turn
- Entertainment < 25: Sanity -2/turn
- Both < 50 at same time: Sanity -3/turn (compounding)
- This replaces the simple "alone = -1 Sanity" environmental effect

Implementation Order

- Backend — `tick_turn()` in `virtual_world_engine.py` :**
 - Remove Energy=0 HP drain, add unconscious state transition
 - Adjust Hunger/Thirst HP drain rates
 - Add Bladder decay scaling with Thirst
 - Add Hygiene penalty when Bladder hits 0
 - Add Social gain from conversation (Social → Sanity link)
 - Add Entertainment gain from examine/use (Entertainment → Sanity link)
 - Add puddle item spawning on relieve-in-place
 - Add Hygiene NPC reaction modifier
- Backend — `app.py` :**
 - Add `POST /api/action/relieve` endpoint (or integrate into existing action system)
 - Add puddle cleaning action
- Frontend — `agent-engine.js` :**
 - Add `_buildInsanityContext(sanity)` function
 - Inject insanity context into LLM prompt before each `step()`
 - Garble room data for LLM at < 25 Sanity
 - Track visited areas per character for Entertainment first-bonus
 - Inject Hygiene description into character context for NPC interactions
- Frontend — `ui-controller.js` / `inspector.js` :**
 - Update vital bars display if needed

Files Changed

File	Change
<code>virtual_world_engine.py</code>	<code>tick_turn()</code> major rewrite; new <code>_spawn_puddle()</code> ; new relieve action
<code>app.py</code>	Add relieve/clean actions if needed
<code>agent-engine.js</code>	Add <code>_buildInsanityContext()</code> ; modify <code>_buildRoomContext()</code> for garbling; modify <code>step()</code> for injection
<code>world_template.json</code>	May need puddle item template

Character Inspector Enhancements

Problem

Two related gaps in character data persistence and inspection:

- Description not round-tripped through scenario save/load.** The `description` field (set per-character in files like `jake.json`) is accepted by the inspector's "Save" button, but the backend's `api_update_player` never stores it, `to_dict()` never serializes it, and `load_from_dict` never restores it. So any character description edits are lost on scenario save, and the descriptions from imported character files never make it into the scenario file at all.
- Stats and skills not visible or editable.** The inspector panel shows vitals, personality, appearance, emotions, inventory, and relationships — but not the character's core D&D-style stats (STR/DEX/CON/INT/WIS/CHA) or skills (Athletics, Perception, etc.). There is no way to view or edit them through the UI. The backend also has no handler to update them.

Scope

Three files, small surgical changes:

File	Change
<code>app.py</code>	Add <code>description</code> , <code>stats</code> , <code>skills</code> to <code>api_update_player</code> 's generic field handler

File	Change
virtual_world_engine.py	Add description to <code>to_dict()</code> player serialization and <code>load_from_dict()</code> player restoration
static/js/inspector.js	Add editable stats grid and editable skills list to <code>showAgent()</code>

Detailed Design

1. Backend — `app.py:api_update_player`

Insert three handlers in the generic field block (after line 784):

```
if "description" in data:
    p.description = data["description"]
if "stats" in data:
    p.stats = data["stats"]
if "skills" in data:
    p.skills = data["skills"]
```

This is a straightforward extension of the existing pattern — every other character field follows this same structure.

2. Serialization — `virtual_world_engine.py`

`to_dict()` (line 3078): Add `"description": getattr(p, 'description', '')` to the player serialization dict, alongside the existing `"personality"` field.

`load_from_dict()` (line 3184): Add `p.description = pdata.get("description", "")` to restore it on load.

3. Frontend — `inspector.js:showAgent()`

Add two new collapsible/inline sections between the vitals display and the Personality section:

Stats section: A compact 2x3 or 3x2 grid of number inputs, one per stat (STR, DEX, CON, INT, WIS, CHA). Each input fires `ApiClient.updateCharacter(charName, { stats: updatedStats })` on change, sending the full stats object.

Skills section: A list of editable skill rows (name + rank number). Each row has a number input for the rank and a remove button. An "Add Skill" dropdown/input at the bottom allows adding new skills. Skills also auto-save via `ApiClient.updateCharacter(charName, { skills: updatedSkills })` on any change.

Both sections use the existing `ApiClient.updateCharacter()` pattern already established for personality, description, and emotion.

Files Not Changed

All other inspector sections (vitals, personality, appearance, inventory, relationships, timeline, memories, behaviors, nudges, manual commands, import/export) remain untouched.

Trigger Enhancements — `adjust_vital`, Multi-Effect, and UX Improvements

Problem

The trigger system is functional but limited: only one effect and one condition per trigger, no way to adjust arbitrary vitals (Bladder, Hunger, Sanity, etc.), no template variable substitution, and the trigger editor for items lacks success/fail message fields and a searchable condition value picker. AI-generated items also don't include triggers.

Scope

Six focused changes across three files. No new files needed.

Feature	Files	Complexity
1. <code>adjust_vital</code> effect	<code>virtual_world_engine.py</code> , <code>item-library.js</code>	Small
2. Multi-effect/multi-condition arrays	<code>virtual_world_engine.py</code> , <code>item-library.js</code>	Large
3. Split message → success/fail in editor	<code>item-library.js</code>	Small
4. <code>has_item</code> dropdown as searchable input	<code>item-library.js</code>	Small
5. <code>_render_template()</code> + context expansion	<code>virtual_world_engine.py</code>	Medium

Feature	Files	Complexity
6. AI trigger generation	main.js	Small

Detailed Design

1. `adjust_vital` effect type

New effect in `_execute_triggers()` :

```
effect_type: "adjust_vital"
effect_params: {
  stat: "Bladder" | any vital name,
  amount: -30 | positive or negative integer,
  target: "self" | character name,
  message: "You feel relieved."
}
```

- Adds `amount` to the specified vital (supports negative values for reduction)
- Clamps to 0-100 (HP uses `Max_HP` as upper bound)
- Target "self" uses active player, other targets look up by player name
- Added to `EFFECT_TYPES` in the engine and to `ItemLibrary.EFFECT_TYPES` in the frontend, with `stat` (text) + `amount` (number) + `target` (dropdown) params in the trigger editor

2. Multi-effect / multi-condition arrays

Backward-compatible format change. `_execute_triggers()` checks for arrays first, falls back to single values:

```
effects = trigger_edge.properties.get("effects", None)
if effects is None:
    effects = [{ "type": effect_type, "params": effect_params }]
conditions = trigger_edge.properties.get("conditions", None)
if conditions is None:
    conditions = [condition] if condition else []
```

Execution logic:

- ALL conditions must pass (AND logic)
- If any condition fails and `failure_message` exists, show it and skip effects
- If all pass, execute ALL effects in order

Frontend trigger editor:

- Single effect row becomes list with "Add Effect" button
- Single condition row becomes list with "Add Condition" button
- Each row independently configurable (type dropdown + params)
- "Failure Message" text field shown when conditions > 0

3. Success/Fail messages

Engine already supports `success_message` and `fail_message` in `effect_params`. Frontend change only: split the single "Message" input into two fields ("✅ Success Message" / "❌ Fail Message") that appear when a condition is set. Both stored as `effect_params.success_message` and `effect_params.fail_message`.

4. `has_item` dropdown as searchable input

Replace the `<select id="trigger-cond-extra">` in `item-library.js` with `<input list="...">` + `<datalist>` populated with all graph items and library items. Same pattern already used for `on_use_on` target selection in the trigger editor.

5. `_render_template()` and context expansion

New method on `VirtualWorld` :

```
def _render_template(self, text, context):
    """Replace {variable} placeholders with values from context dict."""
    if not text or "{param:" in text:
        # Custom params via item properties
        item_params = context.get("item_params", {})
        text = re.sub(r"\{param:(\w+)\}", lambda m: str(item_params.get(m.group(1), m.group(0))), text)
    for key, value in context.items():
```

```
text = text.replace("{ " + key + "}", str(value))
return text
```

Context built in `_execute_triggers()` and passed through to message rendering calls. Context includes: `game_time`, `time_ticks`, `turn_number`, `player_name`, `area_name`, `item_name`, `item_state`, `player_hp`, `player_energy`, `room_light`, `room_temp`, `room_smell`, plus custom `item_params`.

6. AI-generated triggers

In `main.js:generateWithAI()`, extend the format hint:

```
const formatHint = `{"name":"...", "description":"...", "actions":"examine,take,use", "uses":1, "weight":0.5, "hidden":false, "triggers":[{"tr
```

After AI response, parse the `triggers` array and populate the triggers JSON textarea (`item-triggers-json`).

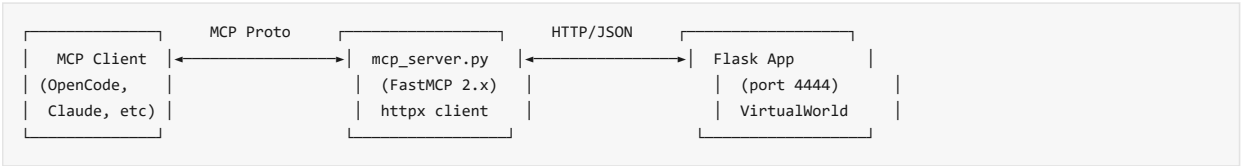
Files Not Changed

- Way inspector trigger UI is already present (`_buildTriggersHtml` is called in `_showDoor`)
- `on_state_enter` / `on_state_exit` trigger types already implemented in engine and frontend
- Skill check condition type already implemented
- `app.py` — no changes needed

Virtual World MCP Server — Design Spec

Architecture

A FastMCP server that proxies commands to an already-running VirtualWorld Flask server via HTTP.



Files

- `mcp_server.py` — single file, entry point, ~600-800 lines
- `.opencode/mcp.json` (or user's MCP client config) — registration snippet

Configuration

Env Var	Default	Description
VIRTUAL_WORLD_URL	http://localhost:4444	Flask server base URL
VIRTUAL_WORLD_TIMEOUT	30	HTTP request timeout in seconds

Tool Reference

1. Core Commands (Action Wrappers)

All call `POST /api/action` with `{"command": "<cmd>"}`.

Tool	Params	Command
<code>look()</code>	none	<code>look</code>
<code>go(direction)</code>	<code>direction: str</code>	<code>go {direction}</code>
<code>take(item_name)</code>	<code>item_name: str</code>	<code>take {item_name}</code>
<code>drop(item_name)</code>	<code>item_name: str</code>	<code>drop {item_name}</code>
<code>use(item_name, target?)</code>	<code>item_name: str, target: str = ""</code>	<code>use {item_name} [on {target}]</code>
<code>examine(target)</code>	<code>target: str</code>	<code>examine {target}</code>

Tool	Params	Command
inventory()	none	i
stats()	none	stats
speak(text)	text: str	say {text}
attack(target)	target: str	attack {target}
rest(minutes)	minutes: int = 10	rest {minutes}
open_door(door)	door: str	open {door}
close_door(door)	door: str	close {door}
toggle(item_name)	item_name: str	toggle {item_name}

2. World State

Tool	Params	Endpoint
get_state()	none	GET /api/state
get_graph_nodes()	none	GET /api/graph/nodes
get_graph_edges()	none	GET /api/graph/edges
list_players()	none	GET /api/players
get_game_time()	none	GET /api/state (extract game_time)
get_area_description()	none	GET /api/room/description
find_path(from_room, to_room)	from_room: str, to_room: str	POST /api/path
get_debug_state()	none	GET /api/debug/state

3. Player Management

Tool	Params	Endpoint
create_player(name, ...)	name, stats?, vitals?, skills?, traits?	POST /api/players
set_active_player(name)	name: str	POST /api/players/active
update_player(name, ...)	name, new_name?, state?, room?, stats?, skills?, traits?, behaviors?, personality?, description?, emotion?, relationships?	POST /api/players/{name}
delete_player(name)	name: str	DELETE /api/players/{name}
kill_player(name)	name: str	POST /api/players/{name}/kill
move_player(name, room)	name: str, room: str	POST /api/players/{name}/move
player_speak(name, text, room?)	name, text, room?	POST /api/players/{name}/speak
import_player(data)	data: dict	POST /api/players/import
import_character(char_id, active?)	char_id: str, active: bool = True	POST /api/registry/characters/import
get_player_knowledge(name)	name: str	GET /api/players/{name}/knowledge
set_player_knowledge(name, data)	name: str, data: dict	PUT /api/players/{name}/knowledge

4. Memory System

Tool	Params	Endpoint
get_player_memories(name)	name: str	GET /api/players/{name}/memories
set_player_memories(name, memories)	name: str, memories: list	PUT /api/players/{name}/memories
add_player_memory(name, text, ...)	name, text, type?, importance?, tick?	POST /api/players/{name}/memories/entry
update_player_memory(name, entry_id, ...)	name, entry_id, text?, type?, importance?	POST /api/players/{name}/memories/entry/{id}
delete_player_memory(name, entry_id)	name, entry_id	DELETE /api/players/{name}/memories/entry/{id}

Tool	Params	Endpoint
<code>clear_player_memories(name)</code>	<code>name: str</code>	POST <code>/api/players/{name}/memories/clear</code>

5. World Editing — Graph Nodes & Edges

Tool	Params	Endpoint
<code>create_node(type, name, properties?)</code>	<code>type, name, properties?</code>	POST <code>/api/graph/node</code>
<code>update_node(node_id, properties?)</code>	<code>node_id, properties?, name?</code>	PATCH <code>/api/graph/node/{id}</code>
<code>rename_node(node_id, new_id)</code>	<code>node_id, new_id</code>	POST <code>/api/graph/node/{id}/rename</code>
<code>delete_node(node_id)</code>	<code>node_id: str</code>	DELETE <code>/api/graph/node/{id}</code>
<code>create_edge(source, target, type, properties?)</code>	<code>source, target, type, properties?</code>	POST <code>/api/graph/edge</code>
<code>update_edge(source, target, old_type, ...)</code>	<code>source, target, old_type, new_type?, properties?</code>	POST <code>/api/graph/edge/update</code>
<code>delete_edge(source, target, type)</code>	<code>source, target, type</code>	DELETE <code>/api/graph/edge</code>

6. World Editing — Higher-Level Builders

Tool	Params	Endpoint
<code>build_room(name, description, ...)</code>	<code>name, description, light?, temp?, air?, smell?, noise?</code>	POST <code>/api/build/room</code>
<code>build_item(name, room?, description?, ...)</code>	<code>name, room?, description?, actions?, uses?, weight?, hidden?, contents?, ...</code>	POST <code>/api/build/item</code>
<code>connect_rooms(room1, room2, dir1, dir2, ...)</code>	<code>room1, room2, dir1, dir2, state?, description?, cost?, way_id?</code>	POST <code>/api/build/connect</code>
<code>reconnect_door(way_id, room_a, room_b, ...)</code>	<code>way_id, room_a, room_b, dir_a?, dir_b?</code>	POST <code>/api/graph/door/reconnect</code>
<code>move_item(node_id, room?, container?)</code>	<code>node_id, room?, container?</code>	POST <code>/api/graph/item/{id}/move</code>
<code>build_item_from_library(room, item_id)</code>	<code>room: str, item_id: str</code>	POST <code>/api/build/item-from-library</code>

7. Item Registry (Library)

Tool	Params	Endpoint
<code>list_library_items()</code>	<code>none</code>	GET <code>/api/registry/items</code>
<code>add_to_library(item_id, data)</code>	<code>item_id: str, data: dict</code>	POST <code>/api/registry/items</code>
<code>remove_from_library(item_id)</code>	<code>item_id: str</code>	DELETE <code>/api/registry/items/{id}</code>

8. Character & Trait Registry

Tool	Params	Endpoint
<code>list_registry_traits()</code>	<code>none</code>	GET <code>/api/registry/traits</code>
<code>add_registry_trait(trait_id, data)</code>	<code>trait_id: str, data: dict</code>	POST <code>/api/registry/traits</code>
<code>list_registry_characters()</code>	<code>none</code>	GET <code>/api/registry/characters</code>
<code>add_registry_character(char_id, data)</code>	<code>char_id: str, data: dict</code>	POST <code>/api/registry/characters</code>

9. World Lore

Tool	Params	Endpoint
<code>get_world_lore()</code>	<code>none</code>	GET <code>/api/world/lore</code>
<code>set_world_lore(entries)</code>	<code>entries: list</code>	POST <code>/api/world/lore</code>
<code>add_lore_entry(category, content, title?)</code>	<code>category, content, title?</code>	POST <code>/api/world/lore/entry</code>
<code>update_lore_entry(entry_id, ...)</code>	<code>entry_id, category?, content?, title?</code>	POST <code>/api/world/lore/entry/{id}</code>
<code>delete_lore_entry(entry_id)</code>	<code>entry_id: str</code>	DELETE <code>/api/world/lore/entry/{id}</code>

10. Save / Load / Reset

Tool	Params	Endpoint
<code>list_saves()</code>	none	GET <code>/api/save-games</code>
<code>save_game(filename)</code>	<code>filename: str</code>	POST <code>/api/save-game</code>
<code>load_game(filename)</code>	<code>filename: str</code>	POST <code>/api/load-game/{filename}</code>
<code>delete_save(filename)</code>	<code>filename: str</code>	DELETE <code>/api/save-game/{filename}</code>
<code>save_scenario(filename?)</code>	<code>filename?: str</code>	POST <code>/api/save-scenario</code>
<code>export_world()</code>	none	GET <code>/api/save</code> (returns full dict)
<code>import_world(data)</code>	<code>data: dict</code>	POST <code>/api/load</code>
<code>reset_world()</code>	none	POST <code>/api/reset</code>

11. Settings

Tool	Params	Endpoint
<code>get_ghost_mode()</code>	none	GET <code>/api/settings/ghost_mode</code>
<code>set_ghost_mode(enabled)</code>	<code>enabled: bool</code>	POST <code>/api/settings/ghost_mode</code>
<code>get_narration_mode()</code>	none	GET <code>/api/settings/narration</code>
<code>set_narration_mode(mode)</code>	<code>mode: 'none' 'player' 'ai'</code>	POST <code>/api/settings/narration</code>

12. NPC Behavior & Editing

NPC behavior is edited through `update_player(name, ...)` which supports:

- `behaviors: list[str]` — behavior tags (e.g. `["wander", "hostile"]`)
- `npc_state: str` — e.g. `'idle'`, `'patrolling'`
- `npc_behavior: str` — e.g. `'wander'`, `'guard'`
- `npc_action_interval: int` — ticks between NPC actions
- `personality: str` — personality description
- `state: str` — game state (`awake`, `sleeping`, etc.)
- `current_area: str` — NPC location

Way properties (for editing visual direction, pass message, state) are on door nodes:

- `update_node(way_id, properties={...})` — edit `current_state`, `description`, `pass_message`, `cost`, etc.
- `reconnect_door(...)` — change which rooms a door connects to

Item editing (state, name, properties):

- `update_node(item_id, name=..., properties={...})` — edit `current_state`, `description`, `actions`, `uses`, `hidden`, `weight`, etc.

Trigger editing:

- Triggers are `logic_trigger` nodes with edges of type `triggers`
- Create: `create_node("logic_trigger", name, properties={trigger_type, effect_type, effect_params, ...})` + `create_edge(item_id, trigger_id, "triggers")`
- Edit: `update_node(trigger_id, properties={...})`
- Delete: `delete_node(trigger_id)` (also deletes incident edges)

Error Handling

- HTTP errors (4xx/5xx) return the Flask JSON error body as the MCP tool error message
- Connection errors (`httpx.ConnectError`) return "Cannot reach Virtual World server at {url}"
- Timeouts return "Request timed out after {timeout}s"
- The `_api(method, path, body?)` helper centralizes all HTTP interaction

MCP Resources (optional)

Could expose:

- `virtualworld://state` → full world state
- `virtualworld://players` → player list
- `virtualworld://nodes` → graph nodes
- `virtualworld://time` → game clock

Example MCP Client Config

```
{
  "mcpServers": {
    "virtual-world": {
      "command": "python",
      "args": ["mcp_server.py"],
      "env": {
        "VIRTUAL_WORLD_URL": "http://localhost:4444"
      }
    }
  }
}
```

Non-Goals

- Starting/stopping the Flask server — that remains separate
- Authentication/authorization — runs on localhost only
- WebSocket streaming — all tools are request/response

Narrative Actions (Emote / do System) — Design

Date: 2026-07-20

Summary

Add support for narrative roleplay actions — actions a character performs that have no game-mechanical effect but add flavor, emotion, and social depth. These are the equivalent of a D&D player saying "I lean against the wall and light my pipe" — no dice, no mechanics, just character.

The system adds an optional `emote` field alongside `action` and `speech` in agent responses, a `do` verb for human players, and posts all emotes as room events so other characters witness and react to them naturally.

Approaches Considered

Approach 1 (Recommended): Separate `emote` / `do` field

Agent returns `{"action": "go north", "emote": "kisses Alice gently"}` — mechanical and narrative in one turn. Human players type `do kisses Alice gently`. Unrecognized commands also auto-fallthrough to `emote`.

Pros: Clean separation; both action types in one turn; full creative freedom.

Cons: More API calls; agent prompt changes needed.

Approach 2: Unrecognized verbs fall through

No special agent field. `{"action": "kiss Alice"}` goes unrecognized → becomes narrative.

Pros: Simplest; no new fields.

Cons: Agent must choose mechanical vs narrative each turn; risk of accidental triggers.

Approach 3: Dedicated `emote` verb

Add `emote` as a recognized verb. Agent writes `{"action": "emote kisses Alice"}`.

Pros: Explicit intent; consistent with existing command structure.

Cons: Clunky; agent can't do mechanical + narrative in same turn.

Phase 1: Quoted Parameter Parsing

Problem

Commands like `put brass key in backpack` fail because the parser splits on spaces — "brass" and "key" become separate tokens. Multi-word item names (`front door` , `brass key` , `wooden chest`) are unreliable.

Solution

Replace simple `command.split()` with a tokenizer that respects double-quoted strings:

```
"go front door"      → tokens: ["go", "front door"]
"take 'brass key'"   → tokens: ["take", "brass key"]
"put 'brass key' in 'wooden chest'" → tokens: ["put", "brass key", "in", "wooden chest"]
"use key on chest"   → tokens: ["use", "key", "on", "chest"]
```

Rules:

- Single quotes `'...'` and double quotes `"..."` both create a single token.
- Unquoted tokens split on spaces as before (backwards compatible).
- This lives in a helper function `tokenize_command(cmd)` in `app.py`.

Impact

Every `if/elif` branch in `take_action()` that parses positional arguments gets updated to use the new tokenizer. Existing commands continue to work identically for single-word arguments.

Phase 2: `do` Verb (Human Players)

Syntax

```
do kisses Alice gently
do stands by the window, watching the rain
do "leans against the wall and lights a pipe"
```

Processing

1. `take_action()` recognizes `do` as a verb token.
2. Everything after `do` is the emote text.
3. Calls `process_emote(actor, emote_text)`.

Auto-fallthrough

If a command goes through all existing parsing layers and none match (including LLM reinterpretation), it becomes an emote instead of returning "Unknown command."

Priority order (unchanged from existing, except last):

```
verb aliases → movement → door open/close → eat/drink → relieve →
use X [on Y] → examine → take → drop → inventory → rest/sleep →
fumble → look → stats → manifest/vanish → toggle → attack → speak →
LLM reinterpretation → EMOTE (new)
```

This ensures `kiss Alice` (no recognized verb) becomes an emote, while `take sword` still executes `take`.

Phase 3: Agent `emote` Field

Agent Response Format

Extend the existing JSON response:

```
{
  "inner_monologue": "I want to be affectionate...",
  "speech": "You look lovely today.",
  "emote": "gently strokes Alice's cheek with a soft smile",
  "action": "look"
}
```

System Prompt Updates

Add to the character's system prompt (both `_buildCharacterSystemPrompt` and `getSystemPrompt`):

```
EMOTE (optional): Describe a roleplay action your character performs.
This is pure flavor – it does NOT change the game state.
Examples: "kisses Alice's forehead", "paces nervously",
"fiddles with a ring", "winks at Bob", "curtsies deeply"

You may include an emote alongside your regular action in the same turn.
The emote is visible to everyone in the room.
```

Execution Flow

In `AgentEngine._executeAction()` (reactive mode) or the combined flow (legacy mode):

1. Execute mechanical action: `POST /api/action { command: action }`
2. If emote is present: `POST /api/emote { actor, emote }`
3. Combine both results into the reaction phase context

Action Validation

The `_validateAction()` method adds `emote` as a valid top-level field (not a verb).
The `action` field still gets validated against the known verb list as before.

Phase 4: Backend `process_emote()` Engine Method

Location

New method in `virtual_world_engine.py`:

```
def process_emote(self, actor_name, emote_text):
```

Processing Steps

1. **LLM parsing** — Send emote text to LLM to extract:
 - `description`: vivid 1-3 sentence narrative of the action
 - `targets`: character names directly involved
 - `context`: room name, other characters present
2. **Area event logging** — Store the emote as a room event:

```
{
  "type": "emote",
  "actor": "Alice",
  "description": "Alice leans in and gently kisses Bob on the cheek.",
  "targets": ["Bob"],
  "turn": current_turn
}
```

3. **Return** description text for display.

API Endpoint

```
POST /api/emote
{ "actor": "Alice", "emote": "kisses Bob gently" }
```

```
→ { "description": "...", "targets": ["Bob"] }
```

LLM Prompt for Emote Parsing

```
You are parsing a narrative roleplay action in a text-based virtual world.

Action text: "{emote_text}"
Performed by: {actor_name}
Current room: {area_name}
Characters present: {character_list}

Return JSON with:
- "description": A 1-3 sentence narrative of the action as seen by an observer.
  Write in present tense, third person. Make it vivid but concise.
  Do NOT invent outcomes for other characters – only describe the actor's actions.
  Example: "Alice steps close to Bob, rises on her toes, and gently kisses his cheek."
- "targets": An array of character names directly involved in the action.
  Empty array if no specific target.

Return ONLY valid JSON, no other text.
```

Display & Event Flow

Client-side (event-stream.js)

New message type `msg-emote` with distinct styling:

- Icon: 🗨️ (or text marker)
- Label: `[Emote]` prefix
- Filter toggle: bundled with actions or standalone

Area Event Integration

Emotes are stored alongside other room events in the event log. When agents observe their surroundings, they see emotes as witnessed events. The agent naturally decides how to feel/react on their next turn.

No immediate reactions — the receiving character's LLM processes the emote on their next turn, deciding their emotional response and relationship effects organically. This avoids the complexity of auto-generating reactions.

File Changes Summary

File	Change
app.py	Add <code>tokenize_command()</code> , do verb handling, auto-fallthrough to emote, <code>POST /api/emote</code> endpoint
virtual_world_engine.py	Add <code>process_emote()</code> method, LLM emote parsing integration
agent-engine.js	Add <code>emote</code> field to response format, execution flow, prompt updates
event-stream.js	Add <code>msg-emote</code> display type, filter toggle, room event integration
mcp_server.py	Add <code>emote(text)</code> tool

Trait System Redesign — Data-Driven Traits with Engine Primitives

Problem

Trait definitions are hardcoded in `engine/traits.py` as a `TRAIT_DEFINITIONS` dict with 22 entries. Adding a new trait requires a code change. The engine also references trait names directly via `TraitSystem.has_effect(player, "dark_vision")` — coupling engine code to specific trait names.

Solution: Three Runtime Stores + JSON Traits

Traits become pure JSON, adding a new one never requires engine changes.

Player Runtime State (what the engine reads)

```
player.capabilities → set[str]      - permanent/modal binary things you CAN do or are
player.conditions  → set[str]      - temporary status effects with engine consequences
player.tags        → set[str]      - identity/classification for triggers to check
player (vitals,stats) - adjustable by triggers (HP, STR, Athletics, Energy, etc.)
```

Engine code never references trait names. It only checks these fields.

Trait JSON Schema

```
{
  "id": "dark_vision",
  "name": "Darkvision",
  "description": "Can see in complete darkness.",
  "category": "physical",
  "params": null,
  "modifiers": {
    "capabilities": {"add": ["darkvision"]},
    "conditions": {},
    "tags": {}
  },
  "triggers": []
}
```

Engine Primitive Fields

When a trait is gained, its `modifiers` are applied to the player's runtime stores. When lost, they are reversed.

```
{
  "modifiers": {
    "capabilities": {"add": ["darkvision", "resistance:fire"], "remove": []},
    "tags": {"add": ["undead"], "remove": []},
    "speed": {"replace": 30},
    "size": {"replace": "medium"},
    "hp_per_level": 1
  }
}
```

- **Set fields** (capabilities, tags, resistances, immunities): additive — `add` inserts, `remove` deletes. Multiple traits contribute independently.
- **Scalar fields** (speed, size, vision): `replace` — takes the highest-priority value on gain, restores the previous value on loss. If two traits set speed, the highest wins.
- **Additive mods** (hp_per_level, skill bonuses): summed across all active traits.

Trigger-Driven Behaviors

For everything beyond simple modifiers — active abilities, tick effects, reactions — traits define `triggers`:

```
{
  "id": "north_born",
  "name": "North-Born",
  "modifiers": {
    "capabilities": {"add": ["resistance:cold"]}
  },
  "triggers": [
    {
      "trigger_type": "on_tick",
      "conditions": [{"type": "temperature_below", "value": 0}],
      "effects": [{"type": "adjust_vital", "params": {"vital": "Warmth", "amount": 5}}]
    }
  ]
}
```

These triggers use the existing trigger system's condition/effect format and are evaluated when the player possesses the trait.

Rate Multipliers (Glutton, Fast Healer, etc.)

Some traits modify *decay rates* or *regen rates* rather than values (glutton: hunger decays $\times 2$, slow_healer: HP regen $\times 0.5$). These use additive modifiers that multiply the baseline rate:

```
{
  "modifiers": {
    "rate_multipliers": {"Hunger": 2.0, "Thirst": 1.0},
    "hp_regen_multiplier": 0.5
  }
}
```

Stored on `player.rate_multipliers` dict. The tick system reads it during decay processing. Multiple traits multiply their contributions: glutton ($\times 2$) + another trait ($\times 1.5$) = $\times 3$ total.

Equipment → Trait Pipeline

Items grant traits through the existing trigger system — no change needed:

```
{
  "triggers": [
    {"trigger_type": "on_equip", "effects": [{"type": "apply_trait", "params": {"trait": "hardy"}}},
    {"trigger_type": "on_unequip", "effects": [{"type": "remove_trait", "params": {"trait": "hardy"}}]}
  ]
}
```

Migration Path

1. Add `modifiers` and `triggers` to trait JSON schema
2. Update `TraitSystem` to load from JSON, drop hardcoded `TRAIT_DEFINITIONS`
3. Add `player.capabilities` runtime store
4. Refactor engine code: `TraitSystem.has_effect(player, "dark_vision")` → `"dark_vision" in player.capabilities`
5. Add effect types: `add_capability`, `remove_capability`
6. Add `player.rate_multipliers` dict + `set_rate` / `clear_rate` effects
7. Migrate all 22 existing traits to new JSON format
8. Wire `on_trait_gain` / `on_trait_loss` to apply/clear modifiers

Live World Editing via MCP + Real-Time GUI Updates

Yes — the repo already ships an MCP server

`mcp_server.py` is a FastMCP servers that proxies an agent's tool calls to the running VirtualWorld Flask app (<http://localhost:4444>). It is registered as a regular MCP server in Claude Code / opencode / kilo / any MCP client. It exposes ~60 tools covering:

- Core commands: look, go, take, drop, use, examine, inventory, stats, speak.
- World state: `get_state`, graph nodes/edges, players, pathfind, game time.
- Player management: create/set-active/update/delete/kill/move/import.
- Memory system: `get/set/add/update/delete/clear` player memories.
- Graph CRUD: create/update/rename/delete nodes and edges.
- High-level builders: `build_area`, `build_item`, `connect_areas`, `reconnect_way`, `move_item`, `build_item_from_library`.
- Library registries: items, traits, characters.
- World lore, save/load/reset, settings, turn management.

So an external agent ALREADY can live-edit, connect, and update the loaded world:

```
connect_areas('Elm Street', 'Oak Street', 'east', 'west')
build_item_from_library('Oak Street', 'monster_energy_drink')
update_player('Miki', current_area='Elm Street')
add_player_memory('Jake', 'Met Miki at the corner.', importance=7)
```

To run it, register `mcp_server.py` in your MCP client config (see the design spec <docs/superpowers/specs/2026-07-18-virtual-world-mcp-server-design.md>).

The gap this work closes: real-time visibility

The GUI's event stream was client-generated from agent actions; edits made by an external agent through the API would not appear until a manual refresh. We added a server->client push channel + a broadcast hook so edits (yours OR an agent's) show up live:

1. World event hub — engine/world_events.py

A process-global fan-out bus (WorldEventHub) with a rolling buffer. publish() broadcasts a dict to every subscriber; subscribe()/unsubscribe() manage SSE clients; recent(n) returns the last n events for snapshot reads.

2. Broadcast hook — app.py after_request

For every mutating /api/ call (POST/PATCH/DELETE/PUT) the server publishes:

```
{ type: 'world_changed', method, path, editor }
```

editor comes from the X-WV-Editor request header, which the MCP server sets to its VIRTUAL_WORLD_EDITOR env value (default 'mcp-agent'). That's how you see WHO made the edit.

3. SSE endpoint — routes/events.py

GET /api/events streams the live events (text/event-stream).

GET /api/events/recent?count=50 returns the buffer as a snapshot.

4. GUI live handler — static/js/event-stream.js

On load it opens EventSource('/api/events'); when a world_changed arrives it refetches world state (worldState.fetch(), debounced) and logs a thin 'World edited by — POST /api/...' line when a non-local editor acted.

5. MCP tools — mcp_server.py

- get_recent_world_events(count): snapshot of recent edit events.
- resource virtualworld://events: readable snapshot of the same buffer.

Result

1. Point an agent at the MCP server and set its editor name:
env VIRTUAL_WORLD_EDITOR=deepseek-agent (or your agent's name).
2. The agent creates a room in Elm Street, places Miki, wires the Monster Crossing trigger. Every API call broadcasts a world_changed event.
3. The GUI you're watching refetches state in real time and the stream shows 'World edited by deepseek-agent — POST /api/build/item'.

Note on the running server

The Python changes (app.py hook, routes/events.py, engine/world_events.py, library_ops.py per-entry apply) require a Flask restart to take effect. The Static JS changes (diff-modal.js, library-browser.js, template-sync.js, event-stream.js, api.js, agent-view.js) load fresh on a browser refresh.

Tests

tests/test_events_and_perentry.py covers the hub, the broadcast hook, the recent endpoint, and the per-entry refresh apply. Run with:

```
python -m pytest tests/test_events_and_perentry.py -q
```

VirtualWorld Wiki

This is the Obsidian vault for **VirtualWorld** — a Flask + JS text-based game engine for AI beings. This wiki documents every system, how it works, how it's wired, and where the code lives.

- Repo: this vault lives inside the game repo, under docs/virtualWorld/.
- Code conventions: AGENTS.md is referenced by the project config but is not present in this checkout.
- Design specs: docs/superpowers/specs/
- Code map: docs/design/js-module-index.md — what each front-end module contributes (generated).

[[World Building/Rooms & Areas|🏠 World Building]]

Doc	What it covers
[[World Building/Rooms & Areas Rooms & Areas]]	Area nodes, environment properties, descriptions per light level, area concept
[[World Building/Doors & Connections Doors & Connections]]	Way nodes, 6 states, connections, hidden doors, unlocking, auto-close, pass_message
[[World Building/Graph System Graph System]]	WorldGraph, Node/Edge dataclasses, 5 node types, 7 edge types, serialization

[[Characters/Characters Overview|🧑 Characters]]

Doc	What it covers
[[Characters/Characters Overview Characters Overview]]	Player class, 3 character types, import/export, registry, library format
[[Characters/Traits System Traits System]]	Trait definitions, library format, how traits modify gameplay
[[Characters/Skills System Skills System]]	Skill checks, progression, action resolution, combat integration
[[Characters/Vitals System Vitals System]]	HP, energy, hunger, thirst, sanity, decay per tick, death, ghost mode
[[Characters/Equipment Loadouts Equipment Loadouts]]	Per-character generated equipment lists by slot
[[Characters/NPC Behavior System NPC Behavior System]]	Simple NPCs, behavior types, action intervals, LLM agent vs scripted
[[Characters/Relationships System Relationships System]]	Closeness model, what moves it (speak/give/combat), labels, guidance, grapple modifier
[[Characters/Emotion & Affect System Emotion & Affect System]]	Multi-dimensional affect map, semantic emotion mapping, mental-vital coupling, relationship valence, self- & social-recall re-feel

[[Items & Inventory/Items Overview|📦 Items & Inventory]]

Doc	What it covers
[[Items & Inventory/Items Overview Items Overview]]	Item class, states, actions, placement, containers, matching, locked_with
[[Items & Inventory/Inventory Inventory]]	Edge model, take/drop, context menus, API client, weight system
[[Items & Inventory/Equipment & Paperdoll Equipment & Paperdoll]]	13 equipment slots, paperdoll grid, equipping, LLM appearance gen
[[Items & Inventory/Item States & Toggleables Item States & Toggleables]]	Toggleable items, active effects, room modification, drain per tick

[[Environment/Light System|🌡️ Environment]]

Doc	What it covers
[[Environment/Light System Light System]]	0-100 scale, light levels, restrictions per level, sanity decay, ghost immunity
[[Environment/Temperature System Temperature System]]	Area temperature, heat propagation (status), effects on vitals
↳ [[Environment/Temperature/Body Temperature Body Temperature]]	Per-player core temp vital, separate from room temp
↳ [[Environment/Temperature/Environment Temperature Environment Temperature]]	Room environment.temperature property, defaults

Doc	What it covers
L [[Environment/Temperature/Equipment & Temperature Equipment & Temperature]]	<code>insulation</code> property, how gear shifts effective ambient temp
L [[Environment/Temperature/Trigger Integration Trigger Integration]]	<code>temperature_below</code> / <code>temperature_above</code> trigger conditions
L [[Environment/Temperature/UI & Display UI & Display]]	Temp vital bar, display ranges and formatting
[[Environment/Time & Weather Time & Weather]]	Tick advancement, in-game clock, day/night cycle status, weather status

[[Rules Engine/Triggers & Effects| ⚙️ Rules Engine]]

Doc	What it covers
[[Rules Engine/Triggers & Effects Triggers & Effects]]	Trigger types, effect types, trigger editor, conditions, multi-effect
[[Rules Engine/Combat System Combat System]]	Turn-based combat, damage, initiative, weapon system, death
[[Rules Engine/Conditions System Conditions System]]	Conditions, application/removal/ticking, library, UI badges

[[Library System/Library System Overview| 📁 Library System]]

Doc	What it covers
[[Library System/Library System Overview Library System Overview]]	Directory structure, per-file format, load/save helpers, import mechanics, API
[[Library System/Tags System Tags System]]	Tag library, what tags do (container/furniture/heat/light), stranger labels from character tags, tag API

[[AI & Narration/LLM Providers| 🤖 AI & Narration]]

Doc	What it covers
[[AI & Narration/LLM Providers LLM Providers]]	Provider configs, API keys, rate limiting, retry logic, fallback models
[[AI & Narration/Agent Engine Agent Engine]]	Agent loop, prompt building, turn queue, action validation, simple NPC diff
[[AI & Narration/Turn-Based System Turn-Based System]]	Off/sequential/random/initiative modes, turn queue, tick application on wrap
[[AI & Narration/Memory System Memory System]]	Memory types, importance, retrieval, context window, editor, generator, <code>memory_emotions</code>
[[AI & Narration/Narration System Narration System]]	3 narration modes, room/action narration, emote command

[[UI & Settings/Inspector Panels| 💻 UI & Settings]]

Doc	What it covers
[[UI & Settings/Inspector Panels Inspector Panels]]	10 inspector sub-views, dispatch by node type, auto-refresh, context menus
[[UI & Settings/Settings & Configuration Settings & Configuration]]	Backend routes, ConfigManager, toggleable settings, profile system, save/load
[[UI & Settings/Engine Config Engine Config]]	Task-304: server-side engine tuning constants (sound/heat/light) editable in the Settings menu, persisted to <code>data/engine_config.json</code> , applies live
[[UI & Settings/Rendering & UI Modules Rendering & UI Modules]]	lit-html via <code>window.Lit</code> , the classic-vs-deferred-module bootstrap race + fix, graph module split, file-size rule
[[UI & Settings/Event Log Export Event Log Export]]	Markdown event-log export, filter-respecting rows, Turn vs tick labeling, stream formatting + emote ordering

📝 Active Tasks

[[dev_tasks/todo/| 📌 Todo]] · [[dev_tasks/inprogress/| 🔄 In Progress]] · [[dev_tasks/review/| 🧐 Review]] · [[dev_tasks/done/| ✅ Done]]

Tasks live in the `dev_tasks/` folder in this vault. Each task file is an `.md` with notes, design decisions, and code references. Tasks are grouped by category (characters, environment, items, gameplay, triggers, graph, ui, prompting, testing, refactor) within each status folder. The folder-move workflow: `todo/` → `inprogress/` → `review/` → `done/`.

Quick Links

- **API health check:** `GET /api/health` → `{"status":"ok"}`
- **API restart:** `GET /api/restart` — resets world from `world_template.json`
- **Game root:** `http://127.0.0.1:4444`
- **Tests:** `node ../tools/test_all.cjs` (requires running server)

Last updated: 2026-08-27

Agent Engine

The Agent Engine drives LLM-powered character agents — autonomous NPCs that observe the world, think, speak, and act through the same `/api/action` endpoint as human players.

Architecture

The engine lives in `static/js/agent-engine.js` as the `AgentEngine` class (`window.VW.agent` , global `agent`). It coordinates several sub-modules:

Module	File	Role
AgentEngine	<code>agent-engine.js</code>	Main loop, turn orchestration, action normalization
PromptBuilder	<code>agent/prompt-builder/</code> (index)	Pure functions for building LLM prompts (split into submodules — see below)
TurnQueue	<code>agent/turn-queue.js</code>	Turn ordering and advancement
AgentMemory	<code>agent/memory-manager.js</code>	Memory storage + reflection (backend <code>Player.memories</code>)
PlanManager	<code>agent/plan-manager.js</code>	Multi-step plan generation
RateLimiter	<code>agent/rate-limiter.js</code>	API rate limiting
ContextWindowManager	<code>static/js/context-window.js</code>	Token budget and context pruning
ResponseParser	<code>agent/response-parser.js</code>	Parse/validate LLM JSON out, surface parse errors
ActionNormalizer	<code>agent/action-normalizer.js</code>	Normalize structured actions into <code>/api/action</code> commands
ThreatDetector	<code>agent/threat-detector.js</code>	Flag nearby threats for replanning
HumanTurnComposer	<code>agent/human-turn-composer.js</code>	Human-turn (composer) drive path
TurnSceneView	<code>agent/turn-scene-view.js</code>	Render current turn scene for the event stream

Agent Loop

Step Cycle (`step()`)

Each agent step processes one character's turn:

1. **Rate limit check** — waits if RPM limit is exceeded
2. **Turn queue** — determines whose turn it is (turn-based mode)
3. **Simple NPC check** — skips LLM work for `simple_npc` characters (they act via backend `tick_turn` / `process_simple_npcs`). Simple NPCs are still **in the turn queue**; their turn just advances without an LLM call. NPC actions are logged to the event stream via `record_turn_event()` .
4. **Rest/Sleep check** — skips if character is resting or unconscious
5. **Reflection** — every 5 turns, triggers memory summarization (reactive mode)
6. **Threat/needs-aware replan** — before think-decide, `PlanTracker.shouldReplan()` checks for new threats, blocked paths, or vitals crossing a critical threshold; if needed, regenerates the plan via `PlanManager.generate()`
7. **Reactive mode** (2 LLM calls — the conversation-loop design, task-176):
 - **Think-decide** (`buildReactionPrompt`): one call produces inner monologue + structured action + speech/volume + emote. This is a single combined call (no separate observe/decide prompts — those exports are dead).
 - **Act**: execute the normalized action via `ApiClient.action()` , optional AI narration, then the emote (gated on the action not failing or being rejected). Invalid verbs become a natural in-world rejection ("You try to climb the ceiling, but you can't do that") that flows into the react phase instead of vanishing silently.
 - **React** (`buildResultReactionPrompt`): character reacts to the action outcome with inner monologue, speech/volume, emote, and the **LLM-generated memory** field. The prompt makes clear only a moment has passed since the character's own action — others haven't had time to respond, so silence isn't a snub.

8. **Non-reactive mode** (1 LLM call): single `buildReactionPrompt` with `includeMemory=true` produces everything in one response; action executed asynchronously.
9. **Turn advancement** — advances the turn queue

Conversation history is retained across turns

`characterHistories[charName]` (agent-engine.js) persists per character and is **not** reset each turn — the think-decide and react phases both append to the same array. This prevents the agent from contradicting itself between phases (e.g. changing a favorite color). The `ContextWindowManager` prunes when over 30 messages / 9500 tokens (keeping system + last ~18 messages). History is cleared only on `start()` / `reset()` .

Character Control Modes & Autonomy (task-244)

Every character resolves to a control mode via `events.getControlMode(charName)` (`event-stream.js`):

Mode	Condition	Who drives
'npc'	<code>player.simple_npc</code>	Backend <code>tick_turn</code> — scripted behaviors, no LLM
'human'	<code>player.autonomy === false</code>	The human, via commands; the engine skips their turn
'llm'	default	Agent engine — full autonomous LLM turns

- The `autonomy` flag is **persisted on the player** in the backend graph, so a human stays human across reloads. Toggle it from the agent inspector.
- A human-mode turn pauses only that character's progression (agent-engine.js ~301) — the rest of the roster keeps cycling when it's their turn.

Per-Turn State Tracking & Instincts

- **Character state** — `events.getCharacterState(charName)` tracks `lastThought` , `lastSpeech` , `lastAction` , `actionHistory` (last 20 with tick + result), `currentArea` , `lastActionResult` . Consumers: plan manager (`lastThought`), prompt memory context (`actionHistory` → recent-actions block), agent lens, inspector.
- **Needs/threat-driven replanning (task-92)** — `PlanTracker.shouldReplan()` (`agent/plan-tracker.js`) regenerates the plan when vitals **cross** a critical threshold or a new threat appears, re-nudging at most every 5 turns to avoid churn.
- **Closeness gates behavior (task-94)** — `relationshipGuidance(closeness)` (`prompt-builder/character-state.js`) turns each relationship score into a behavioral directive on the People-here line ("you want them gone; refuse help..." at ≤ −50).
- **Speech salience + conversation instinct** — `prompt-builder/conversation-context.js` labels heard lines as *addressed to you* vs *overheard* (salience), and `buildConversationInstinct()` nudges characters to join ongoing conversations they're part of.
- **Parse-error inspection (task-238)** — malformed LLM responses are stored in an EventBus ring buffer (`events.logParseError` , last 20) and surface as clickable bubbles that expand the raw response, so bad outputs are debuggable from the stream itself.

Structured Actions (task-160)

The LLM emits a **structured action object** instead of free-text commands:

```
{
  "inner_monologue": "...",
  "action": "use_on",
  "item": "rusty_key",
  "target": "front_door",
  "speech": "Please fit, please fit.",
  "volume": "whisper",
  "emote": "slides the key into the lock"
}
```

- `volume` is `whisper` | `say` | `shout` | `scream` (default `say`). The volume word is the **key**, never a value inside speech — `{"speech": "whisper psst"}` is explicitly wrong in

the prompt. `speak / talk` normalize → `say`. Legacy top-level `say / whisper` keys still work as a fallback.

- `_normalizeStructuredAction()` (`agent-engine.js`) converts `{action, item, target, relation}` into a command string for `/api/action`:
- use `rusty_key` on `front_door` (multi-word names stay whole)
- use `create flame` (use `ALONE` for self-use items)
- put `ink_pen` on `table` (spatial placement, `relation` = `on/under/beside/behind/at/in`)
- give `rusty_key` to `Lyrie` (hand to a same-area character)
- steal `key` from `Miki`, `go north`, `rest 30`, etc.
- Free-text fallback for anything unsupported / legacy plain strings
- `_validateAction()` whitelist is synced 1:1 with the verbs the normalizer can emit (includes `place`, `give`, `hand`). Rejections are surfaced via `_surfaceRejectedAction()` into `config.lastActionResult` (shown as `=== LAST ACTION RESULT ===` next turn) — never a stored memory.

Action verbs the LLM is told about (ACTIONS table)

`go`, `dash`, `crawl`, `climb`, `jump`, `open`, `close`, `take`, `drop`, `use`, `use_on`, `put/place`, `give`, `steal`, `examine`, `read/search`, `wear/equip`, `remove/unequip`, `attack`, `grab`, `escape/struggle`, `rest`, `wait`, `inventory`, `stats`, `look`, `fumble`, `relieve` — plus ghost-only `manifest / vanish`.

- `crawl <dir>` / `climb <dir>` / `jump <dir>` are the passage-movement verbs (task-187):
`go` auto-crawls tight (one tier over `max_size`) and crawl-only ways; climb/jump ways need the matching verb and roll an Athletics check that can fail.
- `grab <character>` is a grapple attempt (falls back to `take` for items); `escape / struggle` break a grapple with a STR save (task-4).
- The player-state block injects **size context** when a `size_*` trait is present ("You are huge. Some passages are too tight for you...") via `buildSizeContext()` in `agent/prompt-builder/`.

Prompt Building

All prompt construction lives in `static/js/agent/prompt-builder/` (pure functions, no side effects). Submodules: `system-prompt.js`, `turn-prompts.js`, `character-state.js`, `conversation-context.js`, `memory-context.js`, `context-sections.js`, `schema-fragments.js`, `helpers.js`, `room-context.js`, `contextual-actions.js`.

System Prompt (`buildCharacterSystemPrompt()`)

- World lore (common knowledge shared by all agents)
- ACTIONS table (structured-object examples, one action per response)
- Rules (multi-word names stay whole, `use` vs `use_on`, `put` needs a `relation`, `give/steal` need a same-area target)
- Ghost/death instructions when applicable
- Items vs flavor distinction

Character identity ("You are X. Personality: Y.") is injected as the first line of every user message via `buildCharacterPreamble()`.

Prompt Templates

All templates share the same **user-message layout** (assembled from `buildRoomContextParts()` + `assembleMessageHead()`, reordered 2026-08-18):

```
[Tick N]
You are X. Personality: ...
=== YOUR STATE ===      ← vitals, emotion, insanity, traits, size, activity, grappled, ghost/dead
=== I REMEMBER ===      ← own block, right after YOUR STATE
Your appearance / Carrying
Room description → paths → items → People here (with inline relationship labels)
=== AVAILABLE ACTIONS ===
=== WITNESSED ===
[=== YOUR PLAN ===]
<phase section>
```

- **Relationships are NOT in** `=== YOUR STATE ===` — each "People here" line carries an inline type label when known (`jake halloway - a close friend - (awake)`), no scores.
- **Memory context is its own** `=== I REMEMBER ===` **block** right after YOUR STATE, not a state fragment.

- The react template keeps its minimal [Tick N] You are still in ... context, then
=== YOUR STATE === → === I REMEMBER === → === WHAT HAPPENED === .

Template	Phase	Memory field	Output
buildReactionPrompt	Think-decide / combined	✗ (reactive), ✓ (non-reactive)	{ "inner_monologue", "action", "item", "target", "speech", "volume", "emote" [, "memory"] }
buildResultReactionPrompt	React	✓	{ "inner_monologue", "speech", "volume", "emote", "memory": { "text", "importance", "tags" } }

buildObservationPrompt / buildDecisionPrompt are **dead exports** — defined and exported but never called. Not maintained.

Room Context & the Attention List (buildRoomContext())

buildRoomContext() (line 582) builds the area description injected into the agent’s user message. Since the interest system landed (2026-08-05, task-98 Phase 2), room items render as an **"Items that catch your attention"** list instead of the old "everything gets a full description" block:

- Unexamined only** — items already examined or taken (in player.discovered_items) are omitted; their facts live in === MY INVESTIGATION NOTES === .
- Interest-sorted** — items matching player.interest_tags (exact tag +2, keyword-in-name +1) surface first; then the rest.
- Weight-ordered** — within each tier, heavier items first (bigger = easier to see; missing/0 weight sinks).
- Capped at 15** (ATTENTION_MAX) — one uniform path for all room sizes, no small/large threshold. Emphasis (sorting) is always on; the cap just bites when a room has more than 15 unexamined items.
- Natural trailer, no truncation** — ends with There are more items around that you can look for. when the cap bit, or ...and not much else. when everything visible fit.
- Light logic unchanged** — pitch black/dim/dark-vision filtering is untouched; attention sorting only refines within each light branch.

People here + spatial position (task-135)

Each person line can include a **position suffix** from spatial_position / at_way_id :

```
- the woman (awake) beside the piano – A musician...
- the stranger at the north – A tall figure...
```

- Stranger labels** — unmet characters show unknown_display_name() (the man , the woman), not database names. The suffix uses the same anonymous label for character anchors.
- Relationship labels** — met characters get an inline type label (- a close friend -) before (state) , from buildRelationshipLabel() (no (65/100) scores).
- Targeting** — combines appearance handle + location (attack the man , go north). Backend _match_character_name() also resolves description words in the same area.
- Physical actions walk you there** — open/go/give/grab/attack set position automatically; examine room steps back. See [[Gameplay/Character Spatial Position]] and system-prompt.js Rules.

Memory in the agent

See [[AI & Narration/Memory System]]. Key points:

- One memory per turn** — the react phase’s memory field (subjective takeaway, importance + tags chosen by the LLM).
- _extractMemory() parses {text, importance, tags} ; tags auto-register into the tag library.
- storeMemory() POSTs to backend /memories/entry (with entity_ids = current area node).
- Speech/emotes/thoughts are log-only, not memories.

Agent Configuration

Property	Type	Description
personality	string	Core personality prompt
description	string	Appearance
state	string	awake , sleeping , dead , unconscious
simple_npc	boolean	Skip LLM calls, act via backend NPC tick
npc_behavior	string	wander , still , patrol , flee , guard
npc_action_interval	int	Ticks between autonomous NPC actions

Property	Type	Description
<code>npc_state</code>	string	<code>idle</code> , <code>curious</code> , <code>hostile</code> , etc.
<code>behaviors</code>	array	Event-driven behavior definitions
<code>emotion</code>	object	<code>{current, intensity, description}</code>
<code>traits</code>	object	<code>dark_vision</code> , <code>slasher</code> , etc.
<code>equipped</code>	object	Worn items by slot

Settings panel controls: API key/base/model/temperature, turn-based mode + order, reactive mode, ghost mode, manual mode, streaming, RPM/TPM limits.

Difference Between LLM Agents and Simple NPCs

- **LLM agents** (`simple_npc: false`): full LLM decision-making, retained conversation history, unified memory + reflection, multi-step plans, structured actions. 2 LLM calls/turn in reactive mode.
- **Simple NPCs** (`simple_npc: true`): no LLM calls, behavior driven by `behaviors[]` + `npc_behavior` pattern, controlled by backend `tick_turn`. Suitable for animals, minor NPCs, environmental entities.

Related

- [[dev_tasks/done/prompting/task-160-parameterized-actions-in-prompts|task-160: Parameterized actions]]
- [[dev_tasks/done/characters/task-178-unify-memory-systems|task-178: Unified memory]]
- [[Gameplay/Character Spatial Position]]
- [[AI & Narration/Memory System]]
- [[AI & Narration/Turn-Based System]]

LLM Providers

VirtualWorld uses an OpenAI-compatible API client (`LLMClient`) to power agent behavior, narration, memory reflection, plan generation, and AI-assisted content creation. The system supports multiple providers through a unified interface.

Configuration

Settings are managed by `ConfigManager` in `static/js/config.js` and persisted to IndexedDB via the `storage` abstraction.

ConfigManager Properties (config.js:6-58)

Property	Storage Key	Default	Description
<code>apiKey</code>	<code>api_key</code>	<code>''</code>	API key for the provider
<code>apiBase</code>	<code>api_base</code>	<code>https://api.openai.com/v1</code>	Base URL for API
<code>model</code>	<code>model</code>	<code>gpt-4.1-mini</code>	Model identifier
<code>provider</code>	<code>provider</code>	<code>openai</code>	Provider name
<code>temperature</code>	<code>temperature</code>	<code>0.7</code>	LLM temperature
<code>maxTokens</code>	<code>max_tokens</code>	<code>512</code>	Max tokens per response
<code>rpmLimit</code>	<code>rpm_limit</code>	<code>0</code>	Rate limit (requests/min)
<code>tpmLimit</code>	<code>tpm_limit</code>	<code>0</code>	Token rate limit
<code>streaming</code>	<code>streaming</code>	<code>false</code>	Enable streaming responses
<code>reactiveMode</code>	<code>reactive_mode</code>	<code>true</code>	thought→act→react phases
<code>showLogs</code>	<code>show_logs</code>	<code>false</code>	Log LLM requests

Provider Types

The `_getDefaultProfiles()` method in `config.js:275` defines built-in profiles:

```
'OpenAI (GPT-4.1-mini)': { apiBase: 'https://api.openai.com/v1', model: 'gpt-4.1-mini' },
'OpenAI (GPT-4o)':      { apiBase: 'https://api.openai.com/v1', model: 'gpt-4o' },
'LM Studio (Local)':   { apiBase: 'http://localhost:1234/v1', model: '' },
```

```
'DeepSeek': { apiBase: 'https://api.deepseek.com/v1', model: 'deepseek-flash' },
'Groq': { apiBase: 'https://api.groq.com/openai/v1', model: 'llama3-70b-8192' },
```

API Key and Base URL

API keys are configured in the Settings UI and stored in browser IndexedDB. They are **never sent to the backend** — all LLM calls are made **directly from the browser frontend** via `fetch()`. There are no server-side API keys or proxy routes.

The `LLMClient` sets the auth header on every request:

```
headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + this.apiKey }
```

For local providers (LM Studio, Ollama), the UI auto-detects `localhost` or `127.0.0.1` in the base URL and shows a text input field instead of a model dropdown (`settings-view.js:133`). The API key check is also bypassed for local endpoints:

```
// agent-engine.js:105
if (!config.apiKey && !config.apiBase?.includes('localhost') && !config.apiBase?.includes('127.0.0.1'))
```

How LLM Calls Are Made

All calls go through `LLMClient` in `static/js/llm-client.js`, a singleton (`window.llmClient`).

Chat Completions (`chat()` , line 35)

```
async chat(messages, options = {})
```

Parameters:

- `messages` — Array of `{role, content}` objects (OpenAI format)
- `options.model` — Model override
- `options.temperature` — Temperature override
- `options.streaming` — Enable SSE streaming
- `options.tools` — Array of OpenAI tool definitions (functions with JSON schemas)
- `options.tool_choice` — Optional tool choice mode (e.g. `'auto'`, `'none'`, `'required'`)
- `options.withTools` — If true, returns structured `{ content, tool_calls }`
- `options.onChunk` — Per-call chunk callback (`chunk`) => `void`, invoked as streamed content arrives
- `options.signal` — `AbortController` signal for cancellation

The method sends `POST {base}/chat/completions` with standard OpenAI-compatible JSON body.

Tool Calling (`chatWithTools()` , line 160)

```
async chatWithTools(messages, options = {})
```

Convenience helper for tool-calling agents (such as the Natural-Language Editor). Forces `streaming: false` and `withTools: true`, returning structured `{ content, tool_calls }` where `tool_calls` contains the parsed OpenAI function calling requests.

Manual Mode (line 50)

When `this._manualMode` is enabled (via the Settings "Manual Mode" checkbox), the `LLMClient` returns a pre-injected `_manualResponse` instead of making API calls. This lets users manually craft agent responses for debugging or story purposes.

Streaming (line 109)

The `_handleStream(resp, format, onChunk)` method parses SSE (Server-Sent Events) in OpenAI and LM Studio formats. It reads the response body as a stream, accumulating content chunks and calling the per-call `onChunk` callback for real-time UI updates.

The chunk callback is **per-call**, passed via `chat()`'s `options.onChunk`. There is no shared `_onChunk` instance field anymore, so concurrent `chat()` calls each get their own stream destination without clobbering each other. The caller owns a unique `streamId` and drives the matching `events.startStreaming(streamId)` / `events.appendStream(streamId, chunk)` / `events.finishStreaming(streamId)` lifecycle (see `agent-engine.js _callLLMMessages`).

Model List Fetching (`fetchModels()` , line 139)

Queries `{base}/models` to list available models. Returns `null` for DeepSeek (known incompatibility). Supports both `{data: [...]}` (OpenAI format) and bare array responses.

Fallback Models (`getFallbackModels()` , line 176)

Returns a hardcoded list of suggested models per provider:

Provider base URL	Models
openai	gpt-4.1-mini , gpt-4o , gpt-4o-mini , o1 , o3-mini
deepseek	deepseek-flash , deepseek-v4-pro (legacy: deepseek-v4-flash , deepseek-chat , deepseek-reasoner)
groq	llama3-70b-8192 , llama3-8b-8192 , mixtral-8x7b-32768 , gemma2-9b-it
openrouter	openai/gpt-4o , anthropic/claude-3.5-sonnet , google/gemini-2.0-flash-001 , meta-llama/llama-3.3-70b-instruct , deepseek/deepseek-chat
localhost / 127.0.0.1	local-model

Embeddings (`embeddings()` , line 167)

Generates text embeddings via `{base}/embeddings` . Only enabled for OpenAI and local endpoints. Uses the model name configured in **Settings** → **Embedding** (`config.embeddingModel` , defaults to `text-embedding-3-small`). Returns the embedding vector or `null` on failure.

The embedding source is configurable in the Settings panel:

- **Local (sentence-transformers)**: Uses a built-in `all-MiniLM-L6-v2` model loaded on demand — no API key needed. Configured as the default.
- **API (LLM Provider)**: Sends to your LLM provider's `/embeddings` endpoint with the configured model name.

Rate Limiting

Implemented in `static/js/agent/rate-limiter.js` as the `RateLimiter` class.

`waitMs()` (line 24)

Calculates how long to wait before the next API call based on `config.rpmLimit` (requests per minute). Maintains a sliding window of timestamps — if the window is full, returns the milliseconds until the oldest timestamp falls out of the 60-second window.

`getCooldown()` (line 40)

Returns remaining cooldown time in seconds. Used by the UI to display rate limit status.

Usage in AgentEngine

The rate limiter is checked at the start of each `agent-engine.js` step (line 75):

```
const waitMs = this._rateLimiter.waitMs();
if (waitMs > 0) {
  // Display countdown to user
  for (let r = sec; r > 0; r--) { ... }
}
```

Error Handling & Fallbacks

Retry Logic (llm-client.js:60-96)

`chat()` implements a 3-retry strategy with exponential backoff:

1. **HTTP 429 or 5xx**: Retries with `2^(attempt-1) * 1000ms` delay (1s, 2s, 4s)
2. **Network errors** (`Failed to fetch` , `NetworkError`): Same exponential backoff
3. **AbortError**: Returns `null` immediately (cancellation)
4. **Other errors**: Throws immediately after max retries

Errors are logged to the event stream via `VW.events.log()` .

Agent Engine Recovery

If an LLM call returns `null` (failure after retries), the agent engine:

1. Logs "❌ LLM call failed for {charName}" to the event stream
2. Sets `config.running = false` (stops the agent loop)
3. Displays "LLM Error - Stopped" status in the UI

This prevents infinite retry loops and makes failures visible to the user.

Health Check

The server exposes `/api/health` (`routes/health.py:41`) which reports `llm_enabled` status from `app.config['LLM_ENABLED']` .

Related tasks

- [\[\[dev_tasks/review/testing/task-88-llm_logging|task-88: LLM logging\]\]](#)

Memory System

VirtualWorld has a **single unified memory system**: every character's memories live in `Player.memories[]` (a structured list on the backend) and are written/read through one API and one editor. This replaced four parallel systems (backend kv-store, browser-localStorage `MemoryStore` , `EntityIndex` , and `world_knowledge`) in **task-178**.

The One Store

All memory lives in `Player.memories[]` — a list of entries. Serialization round-trips it (`engine/serialization.py`). The memory editor, the agent engine, and the MCP memory tools all operate on this same list.

Memory schema

```
{
  "id": "mem_...",
  "text": "subjective 1-3 sentence takeaway",
  "type": "observation|action|speech|thought|reflection|reaction|emote|location|...",
  "tick": 128,
  "timestamp": 1750000000,
  "importance": 7,
  "location": "Living Room",
  "entity_ids": ["area_living_room"],
  "embedding": null,
  "tags": ["valerius", "door"],
  "emotion": {"label": "afraid", "intensity": 6},
  "memory_emotions": [{"label": "embarrassed", "intensity": 4}, {"label": "aroused", "intensity": 7}],
  "salience_override": 0,
  "source": "auto|manual"
}
```

- `source` is `manual` for editor-added / seeded backstory (rendered with a **SEED** badge), `auto` for everything the agent generates at runtime.
- `tags` are **single-word conceptual categories** (fear, trust, mystery, amnesia) — not names, items, or places. `_sanitizeTags()` (`static/js/agent/memory-manager.js`) drops multi-word/hyphenated tags (`silent-stranger`) and any tag matching a known player/item/area name before storing. The LLM prompt instructs the same. New tags are auto-registered into the **tag library** (id-keyed files in `data/library/tags/`), so they dedupe naturally. See [\[\[Library System/Tags System\]\]](#).
- `emotion` is the **primary** single emotion (backward-compatible with the engine spike path).
- `memory_emotions` is the **full list** of emotions attached to this memory (`[{label, intensity}]`) — the richer, multi-emotion form the editor/generator writes. Recall re-feels **all** of them, not just the primary.
- `salience_override` (0–10) is a manual recall-boost; >0 shows a ⚡ **salience** badge.
- `tick` is the time the memory was created. A **negative** tick means *before the scenario started* — the editor's **"Turns before start"** field writes `tick = -N` to simulate a pre-scenario/seed memory.

Who writes memories

Writer	Where	Type
Agent react phase	<code>agent-engine.js</code>	<code>reaction</code> (LLM-generated subjective memory)
Agent non-reactive phase	<code>agent-engine.js</code>	<code>reaction</code>
Wake from unconsciousness	<code>agent-engine.js</code>	<code>thought</code>

Writer	Where	Type
Memory editor	static/js/inspector/memory-view.js	user-selected
Seeds/backstory	scenario files (world_template.json)	observation (source: manual)
MCP memory tools	mcp_server.py	user-selected

LLM-generated memories (task-178 / task-160)

The agent stores **one memory per turn** — a subjective takeaway the LLM writes in the react phase, not raw action/room dumps. The react prompt asks for:

```
{ "memory": { "text": "...", "importance": 7, "tags": [ "door" ] }}
```

_extractMemory() (agent-engine.js) parses this (plain-string fallback → importance 5).
Speech, emotes, and inner monologues are logged to the event stream but **not** stored as memories — people don't remember their own spoken lines.

Memory API

routes/memories.py :

Method	Endpoint	Description
GET	/api/players/<name>/memories	List all memories
PUT	/api/players/<name>/memories	Replace all memories
POST	/api/players/<name>/memories/entry	Add a memory entry
POST	/api/players/<name>/memories/entry/<id>	Update a memory entry
DELETE	/api/players/<name>/memories/entry/<id>	Delete a memory entry
POST	/api/players/<name>/memories/clear	Clear all memories
POST	/api/players/<name>/memories/retrieve	Score + return most relevant memories
POST	/api/players/<name>/memories/reflect	Store LLM insights as reflection memories
GET	/api/players/<name>/memories/spatial	KNOWN ROUTES block (BFS over visited_areas)

Frontend

static/js/agent/memory-manager.js

window.AgentMemory :

- storeMemory(charName, text, importance, type, tick, entity_ids, tags) — POSTs to the backend /memories/entry . Fire-and-forget. Also registers any new tags into the library.
- reflect(charName) — fetches memories (importance ≥ 6), asks the LLM for 1-2 insights, POSTs them to /memories/reflect (stored as type: reflection , importance 8). Runs every 5 turns in reactive mode. No world_knowledge blob anymore.

static/js/inspector/memory-view.js

The inspector Memories tab renders Player.memories[] directly:

- Importance-colored left border (red ≥ 8, orange ≥ 6, green ≥ 4, grey otherwise)
- SEED** badge for source: manual
- Tag chips (#tag) + emotion chips (from memory_emotions)
- Edit / delete per entry; Add + Clear All + 🧠 **Gen Memory** buttons
- Rich editor modal (sectioned cards matching the Settings styling):
 - Content** (textarea), **Classification** (Type / Importance / Tick / **Turns before start**), **Context** (Source / Location / Salience / **Entity references**), **Emotions (multiple)**, **Tags, Semantic** (embedding status + generate).
 - Entity references** is a searchable multi-select of area/item/character nodes → entity_ids .
 - Emotions (multiple)** uses a curated, grouped vocabulary (Core / Warm-Social / Anxious / Arousal / Down / Calm / Determined / Jealous) **plus a free-text custom label input** — so any agent-invented label can be added. Each emotion has its own intensity (1–10) and remove (×).
 - Turns before start** writes a negative tick to simulate a pre-scenario memory.

-  **Save** writes `memory_emotions` (and the primary `emotion`), then auto-embeds the text.

✦ Memory Generator (first-person seed memories)

The ✦ **Gen Memory** button opens a generator: describe the kind of memory (a food memory, a nightmare, a dream, a secret...) and the LLM writes it **in the character's first-person voice** as a **standalone seed** — explicitly *not* tied to the current scene, area, or scenario. It uses the character's actual `personality` + `description` as the identity block. The drafted text is editable before it's saved as a `source: manual` (SEED) memory and auto-embedded. See **Idea** → **Options** → **Memory preview** sections in the modal.

Memory emotion recall (residue)

When an emotionally-tagged memory surfaces, it re-feels a fraction of what it carried. Two paths:

1. **Self-recall** — `memory-context.js::_respikeFromMemories` fires while your own recall block is built; it re-feels every attached `memory_emotions` (affect map + subtle vital).
2. **Social recall** — `room-context.js::_fireSocialRecall` fires when another character *says something that touches one of your memories*; it re-feels it, and (when the speaker is a known person) also nudges your relationship toward them. An anonymized voice ("a woman's voice") still re-feels but attributes no relationship.

Both go through `POST /api/players/<name>/emotions/map`. See `[[Characters/Emotion & Affect System]]` for the dimension map, semantic label mapping, vital coupling, and relationship valence.

Spatial memory (separate, task-136)

`visited_areas`, `discovered_items`, `discovered_exits` are persisted on the player (`serialization.py`) and power the `KNOWN ROUTES` prompt block via `engine/spatial_memory.py` (BFS over the real graph, seeded from `visited_areas`). These are discovery state, not memories — do not confuse them with `Player.memories[]`.

What was deleted (task-178)

- `engine/agent_memory.py` (`AgentMemory` kv-store) — dead code, zero callers
- `static/js/memory-store.js` (browser `localStorage` embeddings) — the agent's real recall work lived invisibly per-browser
- `static/js/entity-index.js` (`VW.entityIndex`) — regex extractor producing junk entities
- `Player._memory`, `Player.knowledge`, `Player.world_knowledge` — dead/duplicate fields

Context window management

`ContextWindowManager` (`static/js/context-window.js`) prunes the per-character conversation history. `AgentEngine` config: `maxTokens: 9500`, `maxMessages: 30`, `recentTurnCount: 6`.

- History grows ~4 messages/turn (think-decide user+assistant, react user+assistant)
- Prune triggers >30 messages or >9500 tokens (~5-7 turns)
- Prune keeps: system message + last `recentTurnCount × 3` (18) messages; older turns replaced with `[Summary: N earlier turns omitted.]`
- **History is NOT reset per turn** — it persists across turns and is only cleared on `start()` / `reset()`. The LLM-generated `memory` field is the durable record once old messages get summarized away.

Related

- `[[dev_tasks/done/characters/task-178-unify-memory-systems|task-178: Unify memory systems]]`
- `[[dev_tasks/done/prompting/task-160-parameterized-actions-in-prompts|task-160: Parameterized actions (LLM memory field)]]`

Narration System

The Narration System adds atmospheric flavor text to game actions and room descriptions. It operates in three modes and integrates with both the agent engine and the LLM client.

Narration Modes

The mode is a 3-way toggle stored on `app.world.narration_mode` (backend) and `narrationUI.mode` (frontend).

Mode	Description
none	Default — static descriptions only, no narration
player	Player is prompted to narrate actions/descriptions via a modal
ai	LLM generates narrative flavor text automatically

Backend Storage (`routes/settings.py:99-117`)

Method	Endpoint	Description
GET	<code>/api/settings/narration</code>	Get current mode
POST	<code>/api/settings/narration</code>	Set mode (<code>none</code> , <code>player</code> , or <code>ai</code>)

The mode is persisted on the world object (`app.world.narration_mode`).

Frontend UI (`static/js/narration-ui.js`)

The `NarrationUI` singleton (`window.narrationUI`) provides:

```
class NarrationUI {
  constructor() // mode = 'none', loads saved setting
  setMode(mode) // 'none' | 'player' | 'ai', saves to backend
  getMode()     // returns current mode
  getNarratedRoomContext(roomContext, charName) // narrate room description
  getNarratedActionResult(actionOutput, charName, action) // narrate action result
}
```

The mode is indicated by a badge UI element (`#narration-indicator`) with visual classes:

- `narration-none` :  No Narration
- `narration-player` :  Player Narration
- `narration-ai` :  AI Narration

How Narration Works

Area Narration Flow

When the agent engine calls `_buildNarratedRoomContext()` (or `PromptBuilder.buildNarratedRoomContext()`):

1. Standard room context is built
2. If mode is `player` :
 - A modal appears with pre-filled room description text
 - Player can edit the narration and submit or skip
 - The edited text replaces the room description in agent context
3. If mode is `ai` :
 - A system prompt and user prompt are sent to the LLM
 - The LLM generates 2-4 sentences of atmospheric description
 - The generated text replaces the room description
4. The narrated text is also broadcast to the room as an emote (`*narrated text*`)

Action Narration Flow

After an agent executes an action (`agent-engine.js:275`):

```
const narrationMode = window.narrationUI?.getMode();
if (narrationMode === 'ai' && config.apiKey && config.model) {
  const narrated = await window.narrationUI.getNarratedActionResult(outputText, charName, finalAction);
  if (narrated) outputText = narrated;
}
```

1. The action result text is captured
2. If mode is `player` : modal shows action + result for player to narrate
3. If mode is `ai` : LLM generates 1-3 sentences narrating the outcome

4. The narrated text replaces the raw action output in the event log

Narrative Generation via LLM

Area Narration Prompt (`narration-ui.js:171-196`)

System message: "You are a narrative game master — a DM describing a scene to a player. Write in second person. Be atmospheric and vivid. Use sensory details. Keep it 2-4 sentences. Do NOT list items mechanically. Integrate notable items into the description naturally."

User prompt includes: room name, description, notable items, characters present, and exits. Temperature is set to `0.8` for creative variation.

Action Narration Prompt (`narration-ui.js:201-224`)

System message: "You are a narrative game master for a game. Describe the outcome of actions atmospherically. Keep it 1-3 sentences."

User prompt includes: character name, action description, and raw result text. Temperature `0.8`.

Narration API Endpoints

Area Context (`routes/narration.py:20`)

```
GET /api/narration/context/room?room=<name>
```

Returns structured data for narrating a room: room name, description, items, characters, exits, environment.

Action Context (`routes/narration.py:32`)

```
POST /api/narration/context/action
```

Returns structured data for narrating an action: actor, action type, description, room context.

Narration Injection (`routes/narration.py:48`)

```
POST /api/narration/inject
```

Inject narration text into the event log and turn events. Accepts `text`, `source` (player/ai/system), `room`, and `actor`.

Roleplay Action Commands (Emote)

The `emote` or `do` command allows players and agents to perform narrative roleplay actions without game-mechanical effects. These are pure flavor.

Syntax

```
do kisses Alice gently
emote sighs heavily
```

Backend

Emotes are broadcast to the room as narrative text. They appear in the event stream with the `msg-emote` class and are visible to all characters in the room.

Agent Usage

LLM agents can generate emotes in two modes:

- **Reactive mode (thought→act→react)**: The decision phase prompt asks for `"emote"` in the JSON response. Emotes are executed after the main action via `ApiClient.emote()`.

- **Non-reactive mode (combined)**: The reaction prompt also asks for `"emote"`. Emotes are executed asynchronously alongside the main action.

Rules that shape agent emotes:

- **emote is a field, not a verb** — the ACTIONS table lists it only under Rules with example JSON; the LLM must pair it with a real action (`wait`, `go`, `use_on`, ...). Using it as an action verb is rejected.

- **Emotes are speculative in the decision phase** — they only run if the action didn't fail or get rejected. A bad verb (e.g. `climb`) suppresses the emote and instead surfaces a natural in-world rejection ("You try to climb the ceiling, but you can't do that") that flows into the react phase.

- **The react prompt frames time**: only a moment passes between the agent's own action and its reaction — other characters haven't had time to respond, so silence isn't treated as being ignored.

Integration with Agent Engine

Narration feeds into agent perception in three ways:

1. **Area context:** The narrated description replaces the default room description the agent "sees"
2. **Action results:** The narrated outcome replaces the raw system output
3. **Event log:** Narration text appears in the room event log, visible to other agents as witnessed events

The `_buildNarratedRoomContext()` method at `agent-engine.js:497` (and its extracted counterpart in `agent/prompt-builder/`) handles the narrated room context for reactive-mode agents. It:

- Calls `narrationUI.getNarratedRoomContext()` to get the narrated text
- Replaces the default description in the room context string
- Broadcasts the narration to the room as an emote

This means narrated descriptions become part of what other agents in the room observe, creating a shared narrative layer.

Related tasks

- [\[\[dev_tasks/review/prompting/task-39-observe_rich_description|task-39: Observe rich description\]\]](#)
- [\[\[dev_tasks/review/environment/task-43-relieve_adds_smell|task-43: Relieve adds smell\]\]](#)
- [\[\[dev_tasks/review/environment/task-49-toggle_room_context_generation|task-49: Toggle room context generation\]\]](#)

type: system
status: done
area: agent
priority: high

Turn-Based System

The engine supports four modes of operation for how agents and the game world advance: **off** (continuous), **sequential**, **random**, and **initiative**. All are toggled from the Automation tab in the Settings modal.

Modes

Off (Continuous)

- `config.turnBased = false`
- No turn queue. The active character acts immediately when "Play" is clicked.
- The agent loop runs on whichever character is set as `controllingPlayer`.
- Steps are counted directly, not divided among queue positions.
- **No automatic vital decay** — decay only fires on `/api/turn/apply`, which only happens on queue wraps.

Sequential

- `config.turnBased = true, config.turnOrder = 'sequential'`
- Characters sorted alphabetically by name.
- Each character acts once per round in order.
- When the queue wraps (last character acts), the backend applies a tick: vital decay, environmental effects, NPC behavior, clock advancement.

Random

- `config.turnBased = true, config.turnOrder = 'random'`
- Queue is shuffled via `Math.random()` on each round.
- Otherwise identical to sequential.

Initiative

- `config.turnBased = true, config.turnOrder = 'initiative'`
- Each character rolls `d20 + DEX bonus` on queue init.
- Sorted descending by roll; alphabetical tiebreaker.
- Rolls are stored and displayed in the initiative sidebar.
- Otherwise identical to sequential.

Architecture

Frontend: Turn Queue

File: `static/js/agent/turn-queue.js`

The `TurnQueue` singleton builds and manages the ordered list:

1. `initialize()` — Gathers alive players (all if ghost mode), sorts by current mode, rolls initiative if needed, resets index/round counter.
2. `advance()` — Moves to next character. On wrap (`currentTurnIndex === 0`):
 - Calls `ApiClient.applyTurn()` → `POST /api/turn/apply` → `TickManager.tick_turn()`
 - Calls `ApiClient.clearTurnEvents()` → `POST /api/turn/clear`
3. `getCurrentCharacter()` — Returns the active character for the current step.

Queue state lives on the `AgentEngine` instance (`VW.agent`):

- `turnQueue` — ordered array of character names
- `currentTurnIndex` — 0-based position
- `turnNumber` — completed rounds
- `initiativeRolls` — `{charName: roll}` map

Backend: Tick Processing

File: `engine/tick_manager.py`

`TickManager.tick_turn()` fires on queue wrap and handles:

- Periodic condition processing (poison, sick, etc.)
- Baseline vital decay (hunger, thirst, sanity, bladder)
- Trait multiplier application
- Exhaustion → unconscious → death chain
- Environmental effects (temperature, air quality, noise, light, smell)
- Body temperature drift
- Active item usage (uses decrement, `on_tick` triggers)
- Sleep timers
- Clock advancement (+1 tick)
- Simple NPC behavior processing

Step Flow

```
Play/Step clicked
→ TurnQueue.build() if empty
→ AgentEngine.step()
  → /api/action (LLM decides + acts)
  → TurnQueue.advance()
    → wraps? → /api/turn/apply (decay) + /api/turn/clear (inc turn_number)
  → Set next character
→ Re-render UI
→ Repeat if running (2s delay between steps)
```

UI

Turn Mode Selection

Settings modal → Automation tab:

- **Checkbox:** Turn-Based Mode — toggles `config.turnBased`
- **Select:** Turn Order — Sequential / Random / Initiative (shown only when turn-based is checked)

Initiative Sidebar

Rendered by `ui-controller.js` `_renderInitiative()` :

- Round number
- Per-character row: position number, icon ( active /  done /  waiting), name, initiative roll (in initiative mode), status text
- Hidden entirely when turn-based is off or queue is empty

Step Display

- Turn-based: Turn N+1 — M/K (current step / total steps per round)
- Continuous: Step: runs/max (remaining)

Ghost Mode Interaction

Dead characters are excluded from the turn queue by default. When `config.ghostMode` is true, dead characters are included — they can observe and move but need skill checks to interact physically.

Reactive Mode (Separate Concern)

`config.reactiveMode` controls the **LLM interaction pipeline**, not the turn queue:

- **On (default)**: Three-phase cycle — think (observe) → decide → act → react
- **Off**: Single combined response (thought + speech + action in one LLM call)

Orthogonal to turn-based vs continuous.

Key Files

File	Role
<code>static/js/agent/turn-queue.js</code>	Queue init, sort, advance
<code>static/js/agent-engine.js</code>	Step loop, LLM orchestration
<code>static/js/config.js</code>	<code>turnBased</code> , <code>turnOrder</code> persistence
<code>static/js/ui-controller.js</code>	Initiative sidebar, step display
<code>static/js/ui/settings-view.js</code>	Settings form
<code>templates/index.html</code>	Settings checkbox + select
<code>engine/tick_manager.py</code>	<code>tick_turn()</code> — decay, environment, NPCs
<code>engine/logging_events.py</code>	Turn events logging
<code>routes/action.py</code>	<code>/api/turn/apply</code> , <code>/api/turn/clear</code>

Activities & States (task-131)

Characters can enter **persistent multi-turn activities** — resting, sleeping, waiting, meditating, bathing, sitting, lying down. Activities live on `Player.activity` and advance one step per `tick_turn()` (once per full turn cycle — **no fast-forward**, the clock only moves when everyone has acted).

Data model

```
player.activity = {
  "type": "sleeping",      # sleeping|resting|waiting|meditating|bathing|sitting|lying down
  "started_at_tick": 123,
  "target_item": "bed",    # the object being used (bed, bath, chair) or None
  "duration_ticks": None,  # optional max length; None = indefinite
  "elapsed_ticks": 0,
  "visible": True,        # whether others see/mention the activity
}
```

Purely descriptive at the data level. Mechanical gating comes from `player.state` (a property over `conditions` , see `player.py:172`) plus the command gate in `routes/action.py` . Serialized in `to_dict` / `serialization.py` , so activities survive save/load.

Commands

Command	Activity	State	Regen (net after decay)	Ends when
<code>rest [N] [on item]</code>	resting	resting	Energy +1/tick	interrupted by any action, damage, or duration elapses
<code>sleep [N] [on item]</code>	sleeping	sleeping	Energy +2/tick	<code>wake</code> , damage, loud noise (WIS save DC 10), Energy full, duration elapses
<code>wait [N]</code>	waiting	—	none	interrupted by anything
<code>meditate [N]</code>	meditating	meditating	Sanity +1/tick	interrupted, or duration elapses
<code>bathe [in item]</code>	bathing	busy	Hygiene +5/tick	Hygiene full → auto-dresses from the pile
<code>sit</code>	sitting	—	Energy +1/tick	<code>stand</code> / any action

Command	Activity	State	Regen (net after decay)	Ends when
lie down	lying down	—	Energy +2/tick	stand / any action
stand / get up	—	—	—	ends sitting/lying/meditating/waiting/resting
stop	—	—	—	ends any activity
wake [name]	—	—	—	wakes a sleeper (self or another)

Gating & interrupts

- **Blocking** (sleeping , bathing): most commands rejected with "You're sleeping in the bed — you can't do that right now." Allowed during both: look/stats/inventory/examine. Bathing additionally allows speech/emote/ stop ; sleeping allows wake .
- **Interruptible** (resting/waiting/meditating/sitting/lying down): any non-trivial action auto-ends the activity first ("You stop resting.") and then runs. Look/stats/inventory/examine/speech/emote do not interrupt.
- **Damage** wakes sleepers and interrupts activities (effects.py handle_damage , combat.py).
- **Loud noise** in the area can wake a sleeper each tick via a WIS save DC 10 (tick_manager.py).

Dress / undress / piles

strip and undress are **instant** but drop every worn item into a clothing_pile container node in the room (id pile_of_clothes_<name> ; tags container , clothing_pile). The pile is physical — examinable, takeable. dress / get dressed re-equips everything instantly (innermost first) and removes the pile when empty. bathe chains: instant strip → pile → bathing activity → auto-dress on finish.

Full stateful one-layer-per-turn dress/undress is a planned follow-up.

Visibility

- Turn events: "[name] is sleeping in the bed." (visible to others via WITNESSED, frontend buildRoomContext).
- Area description people list: "John (sleeping in the bed)" .
- Agent prompts: Activity: sleeping in the bed in === YOUR STATE === and the people list; state.players_in_area now carries activity .

Turn/agent handling

- agent-engine.js _isBusy skips characters whose state is sleeping/resting/meditating/busy or who have an activity — they consume a cheap skip slot, everyone else acts normally, and the round ticks once.
- Simple NPCs (npc_behaviors.py) skip while sleeping/unconscious or in an activity.
- conditions hierarchy gained resting / meditating / busy (display only; not blocking).

Characters Overview

VirtualWorld has three character types, all stored as Player objects (player.py:273). They are differentiated by the simple_npc flag and how they are driven (human, LLM agent, or scripted behavior).

Three Character Types

Type	simple_npc	Driven By	Use Case
Active Player (Human)	False	Human user via UI commands	The player controlling the game
LLM-Driven Agent	False	Agent engine (LLM reasoning)	AI characters with full personality simulation
Simple NPC	True	Scripted behaviors in npc_behaviors.py	Background characters, wandering animals, guards

All three types share the same Player class. The engine does not distinguish between "PC" and "NPC" at the data level — only by which player is currently "active" (see player_manager.py:19).

Player Class (player.py:273)

```
class Player:
    def __init__(self, name="Traveler"):
```

Core Identity Properties

Property	Type	Default	Description
name	str	"Traveler"	Character name
personality	str	""	Free-form personality description/prompt text
base_description	str	""	Naked/baseline appearance — what they look like with nothing on
description	str	""	Current outward-facing description (auto-generated from base + equipment)

Personality is the primary prompt text used by the LLM agent engine. In character library files, this is written in second person ("You are...").

See `routes/players.py:139` for the update API fields.

D&D Core Stats (`player.py:298-301`)

Six standard D&D attributes, range 1-20 (default 10):

```
self.stats = {
    "STR": 10, "DEX": 10, "CON": 10,
    "INT": 10, "WIS": 10, "CHA": 10
}
```

Stats are used in:

- **Combat**: STR for attack rolls, target's DEX/equipment defense for the defense roll (`combat.py:117` , `combat.py:57-60`)
- **Skill checks**: not directly used by the skill system (only skill values are checked)
- **Damage calculation**: STR bonus from `max(0, (attacker.stats.get("STR", 10) - 10) // 2)` (`combat.py:198`)
- **Item action costs**: traits can modify action costs

Vitals & Needs (`player.py:304-311`)

All vitals range 0-100 (except `Temperature` which is 25-45):

```
self.vitals = {
    "HP": 100, "Max_HP": 100,
    "Hunger": 100, "Thirst": 100,
    "Hygiene": 100, "Energy": 100,
    "Social": 100,
    "Bladder": 0, "Sanity": 100,
    "Entertainment": 100, "Temperature": 37.0
}
```

See **Vitals System.md** for full details.

Decay Rates (`player.py:314-317`)

Per-character override for baseline decay rates:

```
self.decay_rates = {
    "Hunger": 1, "Thirst": 1, "Energy": 1, "Social": 1,
    "Hygiene": 1, "Bladder": 1, "Sanity": 1, "Entertainment": 1
}
```

If a character does not specify decay rates, the engine's baseline decay values are used (`tick_manager.py:110-114`).

Skills (`player.py:320-324`)

```
self.skills = {
    "Athletics": 1, "Acrobatics": 1,
    "Stealth": 1, "Perception": 1,
    "Survival": 1, "Persuasion": 1
}
```

Skills are not fixed — characters can have any skill name. Values typically range -20 to +20. Character library examples show custom skills like `"Performance": 5` , `"Investigation": 3` .

See **Skills System.md** for full details.

Traits & Tags (player.py:325-336)

```
self.traits = {}      # trait_id -> param_value (True for booleans)
self.tags = []        # identity markers: ["vampire", "faction:guard"]
self.interest_tags = [] # what the character pays attention to: ["magic", "documents"]
```

Traits are defined in the trait library (engine/traits.py) and assigned via the Inspector UI. Tags are free-form identity markers checked by items, triggers, and conditions.

Interest tags drive the agent prompt's "Items that catch your attention" list: items whose tags (or name keywords) match an `interest_tags` entry surface first, ordered by weight (bigger = easier to see), capped at 15. Examined/taken items drop off the list automatically — their facts live in the agent's investigation notes. Set in the Inspector → agent Bio tab (🧙 Interest Tags).

See **Traits System.md** for full details.

Conditions (player.py:337-342)

```
self.conditions = {"awake": [{"duration": None, "source": None, "level": 0}]}
```

Conditions are a **multi-instance system**: `player.conditions` maps `condition_id` → `[instance, ...]`, so a character can hold several concurrent instances of the same condition (5 vials of poison = 5 stacked `poisoned` instances; drains sum, gates/mods are presence-based). Each instance is `{duration, source, level}` plus optional overrides (`periodic`, `ends_on`, `symptoms`, `known`, and gate overrides like `blocks_speech` / `drops_held_items`).

The single source of truth is `CONDITION_DEFINITIONS` in `player.py:30-215`. The catalog's `stack` field controls re-application:

- "accumulate" (`poisoned` / `sick`): re-apply appends an instance — drains sum
- "refresh" (`stunned` / `exhausted`): re-apply extends the duration / bumps `level`
- "noop" (`grappled` / `restrained` / `blind` /...): re-apply does nothing

Use `has_condition()` / `add_condition()` / `remove_condition()` / `end_instances(action)` — never raw list math. `add_condition()` resolves catalog exclusions and can attach `extra_conditions` bundles, `source_type` ("way" / "area" / "item" / "character" , used by `frightened` gates), and arbitrary gate `overrides`. `end_instances()` removes every instance whose effective `ends_on` includes the action (per-instance override wins over the catalog default).

Standard conditions in the catalog:

Condition	Blocks Action?	Periodic Effect	Notes
awake	No	—	Default state
dead	Yes	—	Auto-fails STR/DEX/CON saves
unconscious	Yes	—	Defense -5; <code>ends_on</code> wake/damage/timer; <code>sleep</code> applies this with <code>source: "sleep"</code> , <code>blocks_speech: False</code>
paralysed	Yes	—	Defense -5, duration 3
stunned	Yes	—	Defense -5, <code>refresh</code> stack, duration 2
grappled	Yes	—	Speed 0, attack -2; held by someone — WHO is tracked via a <code>grappled</code> graph edge (grappler → target), the <code>grappled_by</code> API field is derived from it
restrained	Yes	—	Defense -2, attack -2
prone	No	—	Speed ×0.5, movement <code>crawl</code> , attack/defense -2
busy	No	—	Occupied (rest/meditate/wait/sit/lie/bathe), <code>ends_on</code> stop
exhausted	No	Energy -3/tick	Levels 1-6, <code>refresh</code> stack, speed ×0.5→0
sick	No	Hunger -2, Thirst -2/tick	Hidden (<code>known: False</code>), <code>accumulate</code> , duration 8
poisoned	No	HP -5/tick	Hidden, <code>accumulate</code> , duration 10
blind	No	—	Auto-fails sight checks, attack/defense -2
deaf	No	—	Auto-fails hearing checks
mute	No	—	<code>blocks_speech</code>
frightened	No	—	Blocks re-entering/touching/attacking its <code>source</code>
charmed	No	—	Hidden (<code>known: False</code>)

sleeping is gone — `sleep` is an activity (task-131) that applies an `unconscious` instance. See **Activities & States.md** and **Conditions System.md**.

Condition hierarchy (most significant first, `player.py:218-226`): `dead > unconscious > paralysed > stunned > grappled > restrained > prone > busy > exhausted > sick > poisoned > blind > deaf > frightened > charmed > awake`.

The `state` property (`player.py:399-405`) is a **derived read-only view** — it returns the most significant condition for backward compatibility (UI shows it as a label, not a selector). Its setter **adds** the condition without wiping others (conflicts resolved by catalog exclusions); the inspector edits conditions directly via the add/remove controls. `state_timer` (`player.py:540-546`) is a compat property returning the longest finite duration across the state condition's instances.

Inventory

Items are graph nodes connected to the character via a `carrying` edge (task-105 edge refactor; legacy `carried_by` / `location` edges are migrated on load). Containers/nesting use the spatial edge types (`in`, `on`, `under`, `behind`, `beside`, `at`). See **Graph System.md** for the full edge list.

Equipment / Paperdoll (`player.py:383-390`)

```
self.equipped = {
    "head": [], "neck": [], "torso": [], "arms": [],
    "hands": [], "legs": [], "feet": [], "back": [],
    "waist": [], "accessory": [],
    "hand_left": [], "hand_right": []
}
```

Each slot is a list (stack order = innermost to outermost). Managed by `engine/equipment.py`. The equipment system integrates with the LLM for auto-generating appearance descriptions when items are equipped/unequipped.

Emotion System (`player.py:369-375`)

```
self.emotion = "neutral"      # neutral, happy, sad, angry, afraid, surprised, disgusted
self.emotion_intensity = 0.0  # 0.0 to 1.0
```

Emotion is set via `set_emotion()` (`player.py:596-602`) or auto-updated from action outcome text via `update_emotion_from_outcome()` (`player.py:621-654`). The `get_emotion_n1()` method (`player.py:604-619`) returns a natural language description.

Relationships (`player.py:377-380`)

```
self.relationships = {
    "other_name": {
        "closeness": 0,          # -100 to +100
        "last_interaction_tick": int,
        "interaction_count": int
    }
}
```

- `-100`: sworn enemy, `-50`: rival, `0`: neutral, `50`: friend, `100`: inseparable (`player.py:773-797`)
- Updated via `update_relationship()` (`player.py:754-771`); first meetings are tracked by `register_first_meeting()` (`player.py:656-677`), which stamps `first_sighting: True` so strangers stay anonymized until the next shared-area encounter

Memories (`player.py:392-394`)

```
self.memories = [] # {id, text, tick, timestamp, importance (1-10), type}
```

Managed by `add_memory()` (`player.py:799`, max 200 memories) and retrieved by `get_relevant_memories()` (`player.py:813`). Memory is the **unified store** (task-178) — the agent engine, inspector editor, and MCP tools all operate on this list. The old `agent_memory.py` key-value store and `world_knowledge` blob were deleted. See [\[AI & Narration/Memory System\]](#).

NPC Fields (`player.py:352-359`)

```
self.simple_npc = False
self.npc_behavior = "wander" # wander, flee, stationary
self.npc_action_interval = 3 # act every N ticks
self.npc_state = "idle"      # behavior state machine
```

```
self.state_enter_tick = 0      # tick when npc_state was entered
self.behaviors = []           # list of behavior definitions
```

See **NPC Behavior System.md** for full details.

Other Properties

- `activity` : current multi-turn activity (task-131) — `None` or `{type, started_at_tick, target_item, duration_ticks, elapsed_ticks, visible}` ; types: sleeping, resting, waiting, meditating, bathing, sitting, lying down. See **Activities & States.md**
- `unknown_name` : explicit stranger label used until another character meets this one (falls back to description/tag-derived labels, `player.py:683-736`)
- `discovered_exits` : set of `(area_name, direction)` tuples tracked during exploration
- `visited_areas` : set of area names the character has entered (used for Entertainment novelty bonus)
- `discovered_items` : set of item IDs the character has encountered (used for Entertainment novelty bonus)
- `recent_hearing` : list of `{speaker, text, tick, timestamp}` for recent speech heard
- `interest_tags` : what the character pays attention to (see Traits & Tags above)
- Removed: `item_statuses` , `world_knowledge` , `knowledge` (legacy fields deleted with the memory rework, task-178)

Serialization

`to_dict()` (`player.py:846-889`) produces a complete serializable dict — including `conditions` (list of condition ids), `condition_instances` (full `{cid: [instance, ...]}` data), `activity` , `interest_tags` , and `unknown_name` . Used by API responses.

Character Registry and Library

Registry Files

Characters are stored in two locations:

1. **Runtime registry**: `data/characters.json` — legacy flat JSON file
2. **Library files**: `data/library/characters/*.json` — one file per character (current system)

The `load_registry()` helper (`routes/helpers.py:124-147`) reads all `.json` files from `data/library/characters/` and returns them as a dict keyed by filename (without extension). The `save_registry()` helper (`routes/helpers.py:148-174`) writes one file per key.

Library File Format

Each character library file is self-contained JSON. Example from `Miki.json` :

```
{
  "name": "Miki",
  "personality": "You are Miki, a chaotic ASMR artist...",
  "stats": { "CHA": 12, "CON": 10, "DEX": 12, "INT": 14, "STR": 8, "WIS": 8 },
  "vitals": { "Energy": 100, "HP": 100, "Hunger": 80, ... },
  "skills": { "Investigation": 2, "Perception": 3, "Performance": 5, "Stealth": 4 },
  "traits": {},
  "state": "awake",
  "current_area": "Foyer",
  "inventory": ["phone", "granola_bar", "water_bottle", ...]
}
```

Extended format (from `jake.json`) also includes:

- `description` : rendered appearance text
- `emotion` : `{current, intensity}` object
- `relationships` : dict of relationship objects
- `simple_npc` , `npc_behavior` , `npc_state` , `behaviors` : NPC configuration
- `decay_rates` : per-character decay overrides

Legacy keys (`item_statuses` , `world_knowledge` , `memory_store` , `agent_history`) may still appear in older library files but are no longer read or written by the engine (memory rework, task-178).

Inventory Items in Library Files

Inventory can be specified as a list of strings (item names, matched by `library_id` in the items registry) or a list of dicts (inline item definitions). The import process creates graph nodes for each item (`routes/library_routes.py:134-167`).

Character Import

Characters can be imported into the world via:

1. **API:** POST /api/library/import/character/<id> (routes/library_routes.py)
2. **API:** POST /api/library/import/character/<char_id> with {active, room} (routes/library_routes.py:92)
3. **API:** POST /api/players/import with full player data (routes/players.py:241)
4. **UI:** Import button in the Character Library browser

The import process:

1. Creates a `Player` object with stats, vitals, skills, traits, tags
2. Sets personality, description, emotion, memories
3. Places the character in the specified room (or falls back to first available room)
4. Creates graph nodes for all inventory items
5. Adds the player to the engine via `add_player()`

Character Creation via API

POST /api/players creates a new player from scratch (routes/players.py:21-37):

```
player = Player(name)
player.stats = data.get('stats', player.stats)
player.vitals = data.get('vitals', player.vitals)
player.skills = data.get('skills', player.skills)
player.traits = data.get('traits', player.traits)
player.tags = data.get('tags', player.tags)
player.interest_tags = data.get('interest_tags', player.interest_tags)
app.world.add_player(player)
```

Character Updates via API

POST /api/players/<name> (routes/players.py:139) supports updating all player fields:

- state, current_area, new_name (with graph node migration)
- emotion, emotion_intensity
- personality, description, base_description
- stats, skills
- traits, tags, equipped, interest_tags
- behaviors, npc_state, npc_behavior, npc_action_interval, simple_npc
- conditions (via `load_conditions()`), relationships

Player Manager (engine/player_manager.py)

The `PlayerManager` class:

- **Registration:** `add_player()` creates a graph node and registers the player (`player_manager.py:30-49`)
- **Active player:** `set_active_player()` switches focus (`player_manager.py:51-60`)
- **Area queries:** `get_players_in_area()` lists players in a room, excluding ghosts when appropriate (`player_manager.py:168-191`)
- **Death tracking:** `get_all_dead_players()`, `get_all_alive_players()` (`player_manager.py:193-205`)
- **Trait checks:** `is_slasher()` delegates to `TraitSystem.has_effect()` (`player_manager.py:209-215`)
- **Item lookup:** `find_item_node()` searches inventory, room, and containers (`player_manager.py:110-163`)

Graph Integration

Each player has a graph node with ID `player_<name>` (type `character`). An `in` edge connects the player to their current room. Items are connected via `carrying` edges (inventory) or `equipped` edges (worn/held, slot in edge props).

Death System

A character dies when HP reaches 0. This can happen from:

- Combat damage (`combat.py:201`, death at `combat.py:218-224`)
- Starvation/dehydration/madness (vitals at 0 cause HP loss per tick, `tick_manager.py:182-190`)
- Hypothermia/heat stroke (core Temperature < 30 or > 42, `tick_manager.py:215-218`)
- Exhaustion (Energy at 0 three times, `tick_manager.py:165-180`)
- Toxic air (HP damage from `toxic air`, `tick_manager.py:229-240`)

On death:

1. State is set to "dead"
2. A body item is spawned via `spawn_body_item()`

3. Ghost mode can be enabled to allow dead characters to continue acting
4. Immortal trait prevents HP from dropping below 1

Character Library Examples

The library contains 8 characters as of writing:

File	Name	Type	Setting
Miki.json	Miki	LLM agent	Paranormal mansion (modern)
jake.json	Jake Halloway	LLM agent	Haunted mansion (Chaos teen)
kyrie.json	Kyrie Johansen	LLM agent	Haunted mansion (Wrestler/mechanic)
sammy.json	Sammy Lopez	LLM agent	Haunted mansion (Anxious observer)
kayla.json	Kayla Jenkins	LLM agent	Haunted mansion (Sharp-tongued)
Kaelen Voss.json	Kaelen Voss	LLM agent	Fantasy steampunk setting
Viktor.json	Viktor	LLM agent	Fighting pit (pit fighter)
Gromm.json	Gromm	LLM agent	Fighting pit (heavy bruiser)

Related tasks

- [[dev_tasks/review/characters/task-17-editable_characters|task-17: Editable characters]]
- [[dev_tasks/review/characters/task-27-character_import_from_file|task-27: Character import from file]]
- [[dev_tasks/review/characters/task-28-character_needs_system|task-28: Character needs system]]
- [[dev_tasks/review/items/task-32-enhanced_item_creation|task-32: Enhanced item creation]]
- [[task-89-character_inspector_polish|task-89: Character inspector polish]]
- [[dev_tasks/review/characters/task-38-npc_behavior_go_command|task-38: NPC behavior go command]]

Emotion & Affect System

VirtualWorld gives every character a **multi-dimensional emotional state** (the *affect map*) that sits *alongside* the vitals and relationships, and a set of **recall hooks** that let a memory's emotion actually land on a character when it surfaces. This page documents both the player-facing behaviour and the implementation.

Non-technical overview

A character doesn't just "feel happy." They carry a rolling **affect map** — a set of emotional dimensions on a 0–100 scale that drift toward a restful baseline over time. It's what the LLM reads as "mood" each turn (e.g. "*You are elated — everything feels bright and possible.*").

Three things change that map:

1. **Live events** — the engine/LLM nudges dimensions directly (happy , afraid , ...).
2. **Remembering your own memories** — when a memory surfaces in *your own* recall during a turn, it re-feels a fraction of the emotion it carried ("we remember, therefore we feel").
3. **Someone reminds you** — when another character *says something that touches one of your memories*, that memory re-feels in you too (even a whisper, even if you can't see who's speaking).

And importantly, a re-felt memory now **does something**: it nudges the matching **mental vital** (Sanity / Entertainment / Social) and, when the speaker is a **known person**, the memory also nudges your **relationship** toward them.

Example: "*Remember that time I made you come in the train?*" — if that maps to one of the listener's memories tagged aroused / embarrassed , the listener genuinely re-feels it (both are live dimensions), gets a small vital wobble, and — because the speaker is a named person they know — their relationship toward that person shifts slightly (trust / attraction). If the line came through a wall as "*a woman's voice*", the emotional hit still lands, but no relationship is attributed.

The affect map

Defined in `engine/emotion.py`.

Dimensions & axes

The map is **much richer than the old 7**. It now has **36 dimensions** grouped into axes. Each has a baseline (resting) level the map drifts toward.

Axis	Dimensions (baseline /100)
joy	happy 40 · excited 30 · elated 22 · proud 30
sadness	sad 8 · lonely 10 · melancholic 12 · nostalgic 18
fear	afraid 10 · anxious 12 · uneasy 12 · dread 8 · spooked 10
anger	angry 8 · irritated 12 · resentful 10
arousal	aroused 12 · eager 26 · craving 15 · curious 35
bond	affectionate 25 · loving 26 · grateful 24 · admiring 30
shame	ashamed 8 · embarrassed 10 · guilty 8
envy	envious 5 · jealous 8
disgust	disgusted 5 · repulsed 5
calm	calm 50 · content 46 · peaceful 46 · satisfied 40
surprise	surprised 15

Key functions:

- `baseline()` — a fresh neutral copy of the map.
- `spike(map, dim, delta)` — bump one dimension, clamped to 0–100. **Unknown dims are ignored** (returns unchanged), so an invented label never crashes.
- `decay(map)` — drift every dimension toward its baseline each tick (see `emotion.decay_per_tick`).
- `dominant(map)` — the most deviant dimension and its deviation.
- `describe(map)` — first-person mood paragraph for prompts; dimensions without hand-written bands get a safe generic fallback.
- `felt_from_llm(raw)` — normalize an LLM `{label,intensity}` into a (dim, delta) spike.

Mapping labels → dimensions

`LABEL_TO_DIM` (`engine/emotion.py`)

A large curated vocabulary → closest dimension. Covers the editor/generator word list (embarrassed, aroused, spooked, nostalgic, guilt, ...). `map_label(label)` does exact then substring match and returns `[(dim, weight)]`, or `[]` for a truly unknown word (graceful no-op).

Semantic fallback — `static/js/shared/emotion-mapper.js`

`window.EmotionMapper.resolve(label)` tiers:

- Keyword** — `LABEL_TO_DIM` (shared with the editor) + substring.
- Semantic** — if embeddings are configured, embed the label and pick the **nearest dimension anchor by cosine similarity** (`DIM_ANCHORS` , cached after first use). This is how a novel / agent-invented label still lands on the closest real dimension instead of being dropped.
- Otherwise `null` → the caller sends the raw label and lets the server's keyword map try (and if that fails, it stays a no-op). Nothing ever throws.

How memory emotions reach the character

There are **two recall paths**, and only one existed before.

1. Self-recall (remembering your own memory)

`static/js/agent/prompt-builder/memory-context.js` → `_respikeFromMemories(charName, topMemories)`.

Called while a character's prompt recall block is built. It now reads the `full memory_emotions` **list** (not just a single primary), resolves each label to a dimension via `EmotionMapper`, accumulates the deltas (scaled by intensity, $\times 1.7$), and POSTs once to `/api/players/<name>/emotions/map`. Payload: `{ mapped: { dimension: delta } }`. Rate-limited to one burst per character (5 s guard).

2. Social recall (someone mentions a memory of yours)

`static/js/agent/prompt-builder/room-context.js` → `_fireSocialRecall(charName, seeds)`.

Fired right after that character's **WITNESSED** block is assembled. For every spoken line the character actually perceives (same-room events *and* cross-room `recent_hearing`), it records `{speaker, text}`, then:

- **Matches** the spoken text against the character's memories:
- **Semantic** — embed the line → POST `/api/memory/embeddings/search` (the vectors already live server-side) → top result above a `0.5` threshold.
- **Keyword overlap** as fallback when embeddings are off.
- On a match, **re-feels that memory's** `memory_emotions` (via `/emotions/map`).
- **Name-gated relationship nudge**: includes `toward: <speaker>` only when the speaker is a real, known player (`worldState.players[speaker]`). An anonymized voice label ("*a woman's voice*") stays a pure re-feel — we never invent a relationship with an unknown person.
- Rate-limited (~12 s per character), one recall per turn.

Mental-vital coupling — VITAL_EFFECTS (engine/emotion.py)

Each dimension maps to a **mental vital** and a sign/strength. Applied **subtly** on recall so a single memory only shifts a vital by a few points.

Dimension	Vital	Effect
happy / excited / elated	Entertainment	+
proud / affectionate / loving / admiring	Social	+
afraid / anxious / dread / spooked	Sanity	–
ashamed / guilty / melancholic	Sanity	–
lonely	Social	–
calm / peaceful	Sanity	+
envious / jealous	Social	–
content / satisfied / nostalgic / curious / eager	Entertainment	+

Relationship coupling — RELATIONSHIP_VALENCE (engine/emotion.py)

Maps a re-felt dimension to a **relationship drive** (`trust / fear / disgust / attraction`) toward the speaker. Examples: `affectionate / grateful` → `trust+`, `afraid / dread` → `fear+`, `angry / resentful` → `trust–`, `disgusted` → `disgust+`, and the newer ones: `embarrassed / ashamed` → `trust+`, `aroused` → `attraction+`.

These write a **derive-compatible tagged experience** (`rel:<speaker>`, `<drive>:<value>`) so the existing `engine.derive` reducer folds it into the derived relationship profile (trust/fear/consent) that mechanics gate on — no new relationship machinery.

API

POST `/api/players/<name>/emotions/map` (`routes/player_ops.py` → `handle_map_player_emotions`)

The one endpoint both recall paths use. Request:

```
{
  "label": "embarrassed", // optional – server resolves via LABEL_TO_DIM
  "intensity": 7,         // optional – scale for keyword path
  "mapped": {"fear": 12.4}, // optional – client already resolved (semantic); takes precedence
}
```

```
"vitals": {"Sanity": -2}, // optional – explicit vital deltas
"toward": "elena vance" // optional – speaker; only documented when it resolves to a known person
}
```

Behaviour:

- Resolves dimension deltas (mapped , else label +keyword), applies each spike_emotion .
- Computes subtle **vital** deltas from VITAL_EFFECTS (unless explicit vitals given) and applies them, clamped 0–100.
- If toward resolves to a real player, writes a derive-compatible rel: /drive-tagged memory to nudge the relationship (emotion.rel_scale). Unresolvable → no relationship change.
- Unknown labels never error — they're a no-op.

POST /api/players/<name>/emotions (handle_spike_emotion)

Legacy single-label spike, now accepting the **expanded** dimension set.

Config knobs (engine/runtime_config.py , section emotion)

Key	Default	Meaning
emotion.decay_per_tick	1.5	How fast each dimension drifts back toward baseline.
emotion.llm_spike_max	15.0	Cap on an LLM-declared feeling spike.
emotion.recall_spike_scale	0.25	Intended scale for memory-recall re-feel (the client applies its own subtle factor).
emotion.spike_scale	1.7	Server-side scale for label+intensity mapping on /emotions/map .
emotion.vital_scale	0.25	How strongly re-felt emotions move mental vitals (0 = off).
emotion.rel_scale	0.25	How strongly re-felt emotions move the relationship toward the speaker (0 = off).

Files

File	Role
engine/emotion.py	dimensions/axes, spike / decay / describe , LABEL_TO_DIM , VITAL_EFFECTS , RELATIONSHIP_VALENCE
engine/derive.py	folds rel: /drive-tagged experiences into consumed/trust/fear profiles
routes/player_ops.py	handle_spike_emotion , handle_map_player_emotions
routes/players.py	route registration for /emotions and /emotions/map
static/js/shared/emotion-mapper.js	EmotionMapper.resolve (keyword + semantic)
static/js/agent/prompt-builder/memory-context.js	self-recall re-feel (_respikeFromMemories)
static/js/agent/prompt-builder/room-context.js	social-recall re-feel (_fireSocialRecall)

Related

- [[AI & Narration/Memory System]] — memory storage, editor, generator, and the memory_emotions field
- [[Characters/Vitals System]] — Sanity / Entertainment / Social
- [[Gameplay/Turn Queue & Human Turns]] — where the WITNESSED block is built

NPC Behavior System

VirtualWorld supports two NPC paradigms: **LLM-driven agents** (no simple_npc flag, driven by the agent engine) and **simple NPCs** (simple_npc = True , driven by scripted behaviors). This document covers the simple NPC system.

The simple_npc Flag

NPC type is determined by simple_npc on the Player object (player.py:105-111):

```

self.simple_npc = False          # True = scripted, False = LLM-driven
self.npc_behavior = "wander"    # wander, flee, stationary
self.npc_action_interval = 3    # act every N ticks
self.npc_state = "idle"         # behavior state machine
self.state_enter_tick = 0       # tick when npc_state was entered
self.behaviors = []             # list of behavior definitions

```

Differences Between NPC Types

Aspect	Simple NPC (scripted)	LLM-Driven Agent
<code>simple_npc</code>	True	False
Intelligence	Scripted rules	LLM reasoning
Processing	<code>process_simple_npcs()</code> tick loop	Agent engine sends <code>/api/action</code>
Icon	🐱 (cat emoji)	Standard player icon
Turn queue	Included (LLM step skipped; backend tick drives action)	Included
Behavior	<code>npc_behavior</code> + <code>behaviors[]</code>	Any in-character action
State	State machine (<code>npc_state</code>)	Free-form narrative

NPCBehaviorSystem (`engine/npc_behaviors.py`)

The `NPCBehaviorSystem` class manages all NPC behavior processing. It's initialized with references to the graph, player manager, trigger system, and game state (`npc_behaviors.py:15-19`).

`process_simple_npcs()` (`npc_behaviors.py:27-129`)

Called from `tick_turn()` every tick (`tick_manager.py:317-318`). Processes all simple NPCs in order:

- Skip non-NPCs:** Characters where `simple_npc` is False
- Skip dead** characters
- Build context:** Current tick, NPC state, player room, player inventory
- Evaluate behaviors:** Sort by priority, filter by trigger type and interval
- Legacy fallback:** If no behavior matched, use `npc_behavior` (wander/flee/stationary)

Behavior Evaluation

Behaviors from the `behaviors[]` array are evaluated in priority order (highest first):

```

sorted_behaviors = sorted(behaviors, key=lambda b: -b.get("priority", 0))
for behavior in sorted_behaviors:
    # Check trigger type match
    b_trigger = behavior.get("trigger")
    if b_trigger and b_trigger != trigger_type:
        continue
    # Check interval
    interval = behavior.get("interval", 1)
    if interval > 1 and self.gs.time_ticks % interval != 0:
        continue
    # Check conditions
    conditions = behavior.get("conditions", {})
    if conditions and not self.triggers._evaluate_conditions(conditions, context):
        continue
    # Execute actions
    actions = behavior.get("actions", [])
    if actions:
        outputs = self.triggers._execute_behavior_actions(pname, actions)

```

(`npc_behaviors.py:58-75`)

Legacy NPC Behaviors

If no behavior definitions match (and `trigger_type == "on_tick"`), the legacy `npc_behavior` field is used:

`wander` (`npc_behaviors.py:85-102`)

- Gets open exits from the current room
- Picks a random direction

3. Temporarily switches `active_player` to the NPC
4. Calls `movement.move_to_room()` to move
5. Logs "[NPC] <name> wanders <direction> to <room>."

flee (`npc_behaviors.py:103-129`)

1. Checks if any non-NPC, non-dead player is in the same room
2. If a threat is nearby, picks a random open exit
3. Moves through that exit
4. Logs "[NPC] <name> flees <direction> from a threat."

stationary

Does nothing — NPC stays in place.

Action Interval

Both the `npc_action_interval` field and per-behavior `interval` control how often NPCs act:

```
interval = getattr(player, 'npc_action_interval', 3)
if self.gs.time_ticks % interval != 0:
    continue
```

(`npc_behaviors.py:79-80`)

Default interval: 3 ticks. An interval of 1 means act every tick.

Behavior Definitions Format

Each behavior in the `behaviors[]` array is a dict:

```
{
  "priority": 10,          # Higher = checked first
  "trigger": "on_tick",    # Trigger type (on_tick, on_combat, etc.)
  "interval": 3,          # Act every N ticks (1 = every tick)
  "conditions": {         # Conditions dict (evaluated by trigger system)
    "state_equals": "idle",
    "has_property": {"key": "low_hp", "value": True}
  },
  "actions": [            # List of action dicts
    {"type": "message", "params": {"text": "The guard scans the area."}},
    {"type": "move", "params": {"direction": "north"}}
  ]
}
```

Visual authoring (task-226)

Behaviors are editable two ways: the character inspector's **Advanced** → **Behaviors** form, and the **behavior graph editor** — the same canvas as the trigger editor (`static/js/shared/ trigger-graph.js` , behavior mode), opened via the  Graph buttons in the Behaviors form or agent inspector. Nodes are behaviors/conditions/actions/states; **vertical position sets priority** (top = highest priority, so you reorder by dragging), and Save round-trips to the character's `behaviors[]` byte-identically.

Trigger Types

Defined in the trigger system. Common NPC triggers include:

- `on_tick` : Fired every tick (interval-controlled)
- `on_combat` : Fired when combat occurs nearby

Available Actions

Actions are executed by the trigger system's `_execute_behavior_actions()` (`trigger_system.py:438-632`). Standard action types:

Action	Fields	Description
<code>message</code>	<code>text</code>	Output narration to log
<code>speak</code>	<code>text</code>	NPC speaks out loud (broadcasts to room)

Action	Fields	Description
set_npc_state	state	Change npc_state (e.g., idle, scavenging, fleeing, eating)
damage	amount, target	Deal damage to player or self
heal	amount, stat, target	Restore a vital (HP, Hunger, etc.)
set_environment	stat, amount, area	Change room environment
spawn_item	item_id, display_name, description	Create an item
go	mode, area / room, areas	Move one step via ways — see modes below
teleport	area	Instantly move to any area (no pathfinding)

go movement modes (task-8)

Mode	Fields	Behavior
goto (default)	area or room	BFS pathfind; one step via move_to_area() (doors, costs, locks)
random	—	Random open exit from current area
patrol	areas (comma-separated)	Cycle patrol_route on the NPC; one step toward current waypoint; advances index on arrival

Patrol state persists on the player: patrol_route[], patrol_index. Use teleport when you genuinely want to skip ways.

Condition Types

Conditions are evaluated by _evaluate_conditions() (trigger_system.py:337-431). Supports logical operators and leaf types:

Logical operators (conditions.operator):

- and — all sub-conditions must pass
- or — any sub-condition passes
- not — negates sub-condition

Leaf conditions (conditions.type):

Type	Fields	Evaluates
eq	target, value	context[target] == value — checks context keys like npc_state
has_item	item, target	Checks if target has item in inventory
in_area	area, target	context[npc_area] == area
random_chance	chance (0.0–1.0) or value (0–100)	Probabilistic gate
tick_since_state	min_ticks	(current_tick - state_enter_tick) >= min_ticks
proximity	max_areas	Checks if player is within N areas of NPC (0 = same room)

The context dict available to conditions:

```
context = {
    "npc": player,                # NPC Player object
    "npc_state": player.npc_state, # e.g. "idle", "scavenging"
    "npc_area": player.current_area, # Area name
    "state_enter_tick": ...,        # Tick when npc_state was entered
    "current_tick": ...,           # Current game tick
    "player": player_obj,          # Active human player object
    "player_area": player_area,    # Active human player's area
}
```

Trigger Types for Behaviors

Behaviors can be triggered by different events, passed via process_simple_npcs(trigger_type) :

Trigger	When fired	Source
on_tick	Every tick (interval-controlled)	tick_manager.py
on_player_enter_area	When a player enters the NPC's area	movement.py
on_player_leave_area	When a player leaves the NPC's area	movement.py
on_item_taken	When an item is taken from the area	item_actions.py
on_player_examine	When a player examines something in the area	narration.py

npc_state State Machine

The `npc_state` field (`player.py:109`) tracks the NPC's current behavior state. It's a free-form string set by behaviors. The `state_enter_tick` (`player.py:110`) records when the current state was entered, enabling time-based state transitions.

No hardcoded state machine exists at the engine level — states and transitions are managed entirely through behavior definitions. Example state flow for the scavenging rat:

```
idle → fleeing (player close) → hiding (go to Kitchen) → scavenging (after 5 ticks)
    → eating (find food, heal Hunger) → scavenging (finish eating) → explore Cellar/Kitchen
```

Example: Scavenging Rat

The rat in the world template uses a full behavior state machine (`world_template.json` , player `rat`):

```
foraging ↔ eating (Hunger < 80) ↔ Kitchen/Cellar via go
idle → foraging (patrol)
fleeing → hiding → foraging (after ticks)
on_player_enter_area → whisker twitch + squeak
```

States: `idle` , `foraging` , `eating` , `fleeing` , `hiding` . Legacy `npc_behavior` is `stationary` so all movement comes from `behaviors[]` `go` actions (pathfinding via ways, one step per tick).

```
{
  "trigger": "on_tick",
  "priority": 9,
  "interval": 2,
  "conditions": {
    "operator": "and",
    "conditions": [
      {"type": "proximity", "max_areas": 0},
      {"type": "random_chance", "chance": 0.55}
    ]
  },
  "actions": [
    {"type": "set_npc_state", "state": "fleeing"},
    {"type": "speak", "text": "Squeak!"},
    {"type": "message", "text": "The rat squeaks in alarm and darts for cover."}
  ]
}
```

Legacy `npc_behavior` vs `behaviors[]`

Prefer `behaviors[]` for anything that should feel alive. The legacy string field (`wander` , `flee` , `stationary`) is a zero-config fallback when no behavior definition matches on `on_tick` :

Legacy value	What it does	Status
<code>wander</code>	Random open exit via <code>move_to_area()</code>	Works — good for placeholder NPCs
<code>flee</code>	Random exit when a non-NPC player shares the room	Works
<code>stationary</code>	Nothing	Works
<code>patrol</code> , <code>guard</code> , <code>follow</code> , <code>hunt</code> , <code>still</code>	Documented in places, not implemented in legacy fallback	Use <code>behaviors[]</code> instead

Keep legacy `wander/flee` as a cheap default for animals and extras with no authored tree. Remove it from characters you care about (set `npc_behavior: "stationary"` and put all logic in `behaviors[]`).

Internal wiring: `process_simple_npcs()` must pass `game_state=self.gs` into `_evaluate_conditions()` and `_execute_behavior_actions()` — without it, scripted actions silently no-op.

Hunt System

The `NPCBehaviorSystem` provides a BFS-based hunt system for aggressive NPCs:

`hunt()` (`npc_behaviors.py:133-159`)

Agent-facing hunt command:

1. Finds the nearest living, non-slasher player via BFS through room connections

2. Returns the direction to move toward them
3. Returns flavor text ("sniffs the air", "growsl in frustration")

`slasher_hunt()` (`npc_behaviors.py:161-218`)

AI-driven hunt for slasher characters:

1. Finds nearest living player
2. If in same room: calls `_slasher_attack()`
3. Otherwise: BFS pathfinding, forces open closed doors
4. Moves the slasher toward the target

Pathfinding

Both use `_get_path_to_room()` (`npc_behaviors.py:278-309`):

1. BFS from source room to target room
2. Traverses door nodes (open or closed)
3. Returns the first direction to take
4. Returns `None` if no path exists

`_get_nearest_player_to()` (`npc_behaviors.py:227-276`) finds the closest living player by BFS distance, excluding slashers from the target pool.

Context Building

When processing behaviors, a context dict is built (`npc_behaviors.py:45-56`):

```
context = {
    "npc": player,
    "npc_state": getattr(player, 'npc_state', 'idle'),
    "npc_area": player.current_area,
    "state_enter_tick": getattr(player, 'state_enter_tick', 0),
    "current_tick": self.gs.time_ticks,
    "player": player_obj,
    "player_area": player_area,
}
```

This context is passed to condition evaluation and can be extended with `extra_context` .

Combat Reactions

`process_npcs_on_combat()` (`npc_behaviors.py:23-25`) is a stub called from `combat.py` when combat occurs. Currently empty (`pass`) — future implementation will allow NPCs to react to nearby combat.

Creating Simple NPCs

Via API

Create a simple NPC via `POST /api/players` :

```
{
    "name": "Guard",
    "simple_npc": true,
    "npc_behavior": "stationary",
    "npc_action_interval": 2,
    "stats": {"STR": 12, "DEX": 10, "CON": 12, "INT": 8, "WIS": 10, "CHA": 8},
    "skills": {"Athletics": 3, "Perception": 2}
}
```

Via Import

Character library files can set `simple_npc` :

```
{
    "name": "Wandering Merchant",
    "simple_npc": true,
    "npc_behavior": "wander",
    "npc_action_interval": 5,
    "behaviors": [...],
}
```

```
    "inventory": ["goods", "coin_pouch"]
}
```

Via Inspector UI

The Inspector UI has a behaviors panel for configuring NPC behaviors, states, and conditions visually.

Edge Cases

Dead NPCs

Simple NPCs in state "dead" are skipped entirely — they never act, wander, or flee (npc_behaviors.py:35-36).

No Available Exits

If a wandering or fleeing NPC has no open exits from their current room, they stay in place. The code checks if not open_exits: continue (npc_behaviors.py:91, 118).

Interval Desync

NPCs with different npc_action_interval values act on different ticks. An NPC with interval 3 acts on ticks 0, 3, 6, 9... This ensures NPCs don't all act simultaneously, spreading out the processing load.

State Transitions

The npc_state field persists between ticks. Behaviors can change it via actions. The state_enter_tick records when the state was entered, allowing behaviors to check tick_since_state_change > threshold .

LLM-Driven Agents vs. Scripted NPCs

This is a critical architectural distinction (AGENTS.md:20):

Criterion	LLM Agent	Simple NPC
simple_npc	false	true
Processing	Agent engine sends actions	process_simple_npcs() tick loop
Personality	Rich personality prompt, LLM interprets	No personality — scripted behavior
Flexibility	Can do anything in-character	Limited to defined behaviors
Cost	LLM API calls per action	Zero cost (local scripted)
Implementation	agent-engine.js (frontend) + routes/action	engine/npc_behaviors.py
Memory	Full memory system (player.memories[] — unified store)	None (unless manually scripted)

LLM-driven agents are created the same way as simple NPCs (same Player class) but with simple_npc = False . The agent engine in agent-engine.js decides when they act by calling /api/action with the agent's name.

Agent Memory System

Memory is the unified Player.memories[] store (task-178) — there is no separate key-value agent_memory.py anymore. See [AI & Narration/Memory System].

- The agent writes **one memory per turn**: the LLM generates a subjective memory field in the react prompt ({text, importance, tags}), stored via AgentMemory.storeMemory() → POST /api/players/<name>/memories/entry .
- Plans live in the agent engine (_plans[charName] in agent-engine.js), not on the player.
- Reflection (AgentMemory.reflect()) summarizes high-importance memories into reflection memories every 5 turns.
- Spatial recall (KNOWN ROUTES) comes from engine/spatial_memory.py over visited_areas .

This is separate from the main player.memories[] list, which stores narrative memory entries for LLM context building.

Related tasks

- [[dev_tasks/todo/characters/task-8-npc_behavior_movement|task-8: NPC behavior movement]]
- [[dev_tasks/todo/characters/task-92-needs_driven_autonomous_replanning|task-92: Needs-driven autonomous replanning]]
- [[dev_tasks/review/characters/task-38-npc_behavior_go_command|task-38: NPC behavior go command]]

- [[bug_8-max-steps-counts-characters-not-turns|bug-8: Max steps counts characters not turns]]
- [[dev_tasks/todo/characters/task-94-closeness_as_behavioral_gate|task-94: Closeness as behavioral gate]]

Relationships System

Type: system · area: characters · status: accurate as-of 2026-08 · source: player.py, engine/speech.py, engine/combat.py, engine/items/transfer_actions.py, static/js/agent/prompt-builder/character-state.js

Relationships are the social state each character tracks toward every other character they've met. They drive **display labels**, **behavioral guidance injected into the prompt**, and a **grapple modifier**. They are *not* automatically derived from memories — they are mutated by a small set of authored *events*, or seeded manually.

Data model (Player.relationships, player.py:134)

```
relationships = {
    "<other_player_name>": {
        "closeness": -100..100,    # -100 sworn enemy - 0 neutral - 100 inseparable
        "last_interaction_tick": int,
        "interaction_count": int,
        "first_sighting": bool    # "name unknown" - stranger/masked label until met/spoken
    }
}
```

- **closeness** clamps to [-100, +100] (player.py:439).
- **first_sighting** is set True when seen but not yet introduced; it flips off once the name is known (via speech / name-tag read). It drives the stranger label (area_description.py:282-287, scene_snapshot.py:112-114).

The mutation method — update_relationship(other, tick, sentiment_change) (player.py:424)

The single write-path for closeness. On first contact it creates the entry and grants an **Entertainment novelty boost** (grant_meeting_entertainment(), mirrored from area/item-discovery boosts, task-136). Then:

```
rel["closeness"] = clamp(-100, 100, rel["closeness"] + sentiment_change)
rel["last_interaction_tick"] = tick
rel["interaction_count"] += 1
```

The docstring documents sentiment_change: -20..+20 per interaction, but that full range is **never used** — see the call sites below.

What actually affects relationships (the only mutation call-sites)

Event	File	Delta	Symmetric?
Speak to someone (a directed speech line)	engine/speech.py:217,220	+2 both directions	yes
Give an item to a character	engine/items/transfer_actions.py:65	+5	no - only the <i>recipient</i> grows (+5 toward giver); the giver's sentiment is unchanged
Attack / damage someone	engine/combat.py:110	-30 (min -100)	no - only the <i>target's</i> closeness toward the attacker drops

That's the whole set — **three mutation points**. Everything else only *reads* closeness (grapple modifier engine/grapple.py:61-69, prompt guidance, labels).

Natural-language labels (two parallel systems — slight drift)

Backend get_relationship_n1() (player.py:443): mortal enemy (<=-75) - enemy (<=-50) - rival (<=-25) - unfriendly (<0) - neutral (0) - acquaintance (<=25) - friend (<=50) - close friend (<=75) - **inseparable** (>75).

Frontend relationshipTypeName() (character-state.js:39): same tiers, except the top is **"inseparable friend"** (minor name drift). Inline label via buildRelationshipLabel() (character-state.js:54) renders as a close friend (no numeric score).

Behavioral guidance — relationshipGuidance(closeness) (task-94, character-state.js:87)

Injected into the prompt as a per-character behavioral directive:

closeness	directive
<= -50	"you want them gone; refuse help, keep replies hostile or silent"
<= -25	"keep interactions cold and minimal; never turn your back on them"
< 0	"you keep your guard up; brief, wary replies"
<= 25	"polite but reserved; courtesy without warmth"
<= 50	"you are friendly; chat openly and help when asked"
<= 75	"you are glad they are here; engage warmly, share news, watch out for them"
> 75	"you trust them completely; prioritize their safety, share secrets, stay close"

Relationship-driven mechanics (readers)

- **Grapple** (engine/grapple.py:61-69,305): DC = 10 + grabber Athletics + relationship modifier (+ extra-target penalty). Modifier = -(closeness // 25) * GRAPPLE_REL_PER_LEVEL — a friend is *harder* to grapple (higher DC), an enemy easier. Clamped.
- **Prompt label** per "People here" line (room-context): inline type label, no score.
- **Name-known / stranger mask**: first_sighting -> anonymous label (the man , the woman) until name known.

Serialization (player.py:621-627)

to_dict() emits **only** {closeness, interaction_count} per relationship.

Warning: **first_sighting** and **last_interaction_tick** are **NOT persisted**. They are dropped on save, so recency-based guidance can't work across a save, and the stranger/unmasked state may reset on load.

What's missing / broken (gaps found in this audit)

Status: audited against task-349. Items marked **[fixed]** are resolved; **[not-a-bug]** verified safe; the rest are open design follow-ups.

1. **Closeness barely responds to social texture**. Only speech (+2), give (+5), and combat (-30) move it. Flirting, helping, comforting, banter, shared activities, emotional beats, and even negative social friction (cold, dismissive, insulting) do not change closeness unless they happen to be a speech line.
2. **No time decay**. Nothing lowers closeness over time or from neglect — only an outright attack does. Relationships are monotonic upward (or crater from one attack). The -20..+20 docstring range is unused; real deltas are +2/+5/-30.
3. **No affect/valence weighting**. Speech tone, volume, and emotion are scored not at all — a whispered compliment and a shouted insult both move closeness by the same +2.
4. **[fixed] Asymmetric give**. Give updates only the recipient's closeness toward the giver; the giver's toward the recipient is unchanged. (Now symmetric — task-349.)
5. **[fixed] Serialization drops first_sighting and last_interaction_tick** — recency/stranger state not durable across saves. (to_dict now emits both — task-349.)
6. **[not-a-bug]** Originally flagged: relationshipGuidance returns undefined for closeness > 75. Verified the code already returns a "trust them completely" directive; the >75 tier exists. (Also now explicitly aligned — task-349.)
7. **[fixed] Two label systems drift** — backend get_relationship_nl ("inseparable") vs frontend relationshipTypeName ("inseparable friend"). (Frontend aligned to "inseparable" + article fixed — task-349.)
8. **Relationships are not derived from memories**. Seeded closeness:0 / interaction_count:0 can contradict authored memories of prior acquaintance (e.g. miki<->jake had prior banter memories but 0 closeness). Authoring must keep the two in sync; nothing computes one from the other.
9. **[fixed] Guidance tier boundaries don't match label tier boundaries** (guidance: -50/-25 vs label: -75/-50/-25). (relationshipGuidance tiers now mirror the label tiers — task-349.)
10. **[not-a-bug] First meeting may double-count**. Sight uses register_first_meeting(), which returns False if the relationship already exists and never calls update_relationship(), so interaction_count isn't bumped twice. Verified safe.

Related

- [[Characters/Characters Overview]]
- [[AI & Narration/Agent Engine]] (closeness gates behavior, task-94)
- [[Gameplay/Character Spatial Position]]

Skills System

Skills are numeric proficiencies that modify dice rolls for action resolution. The system is stateless — all dependencies (player manager, logging) are injected via the `SkillSystem` constructor (`engine/skills.py:12-26`).

Skill Values

Characters have a `skills` dict (`player.py:75-80`) mapping skill names to numeric values:

```
self.skills = {
    "Athletics": 1, "Acrobatics": 1,
    "Stealth": 1, "Perception": 1,
    "Survival": 1, "Persuasion": 1
}
```

Skills are not fixed to these six — characters can have any skill name. Example custom skills from library files:

- `"Performance": 5` (Miki, ASMR artist)
- `"Investigation": 3` (Jake, intuitive pattern-seeker)
- `"Investigation": 2` (Miki, investigator)

Values typically range 0-5 but there is no upper limit enforced.

Skill Checks

Mechanics

Skill checks use a D20 roll + skill value vs. a Difficulty Class (DC):

```
def skill_check(self, skill_name, difficulty_class=10, use_active_player=True):
    player = self.player_manager.get_active_player_obj()
    skill_value = player.skills.get(skill_name, 0) # defaults to 0 if skill missing
    roll = random.randint(1, 20)
    total = roll + skill_value
    success = total >= difficulty_class
```

(`engine/skills.py:43-91`)

Difficulty Classes

DC Range	Description
≤5	very easy
6-10	easy
11-15	medium
16-20	hard
>20	very hard

(`engine/skills.py:74-83`)

Return Value

Returns a tuple of (`success: bool`, `total: int`, `message: str`).

Example output:

```
[Skill Check] Perception vs DC 12 (medium): roll=8 + 3 = 11 => failure
```

(`engine/skills.py:86-91`)

When Skills Are Not Found

If a player does not have a skill, `player.skills.get(skill_name, 0)` returns 0 — the roll becomes just `1d20 + 0`. No error is raised.

Dice Rolling

```
def roll_dice(self, num_dice=1, sides=20, modifier=0):
    total = sum(random.randint(1, sides) for _ in range(num_dice))
    return total + modifier
```

(engine/skills.py:30-39)

Used by both skill_check() and the combat system for attack/defense rolls.

Player State Remedies

The player_state_remedy() static method (engine/skills.py:95-110) returns hints for recovering from conditions:

State	Remedy
sleeping	wake up
unconscious	receive medical attention
bound	struggle free
exhausted	rest and eat
injured	heal with medicine
dead	create a new character

LLM Call Logging

log_llm_call() (engine/skills.py:114-135) is a stub for logging LLM request/response pairs to the turn event stream. Currently disabled (immediate return at line 125). When enabled, it records prompts and responses (truncated to 500 chars) to logging_events.record_turn_event() .

Combat Integration

The combat system (engine/combat.py) uses SkillSystem for dice rolling and player lookups:

Attack Resolution (combat.py:26-105)

```
attack_roll = self.skills.roll_dice(1, 20, attacker.stats.get("STR", 10))
defense_roll = self.skills.roll_dice(1, 20, target.stats.get("DEX", 10))
```

- Attack roll = 1d20 + STR modifier
- Defense roll = 1d20 + DEX modifier
- If attack >= defense: hit

Damage Calculation

- **With weapon:** 1d<weapon_damage> + STR bonus (combat.py:46-55)
- Slasher characters get an additional attack_bonus on damage
- **Unarmed:** 1d4 + STR bonus (combat.py:72-73)
- STR bonus = max(0, (STR - 10) // 2) (combat.py:50)

Weapon Discovery

WEAPON_KEYWORDS (combat.py:10-14) lists 15 weapon types (cleaver, knife, hatchet, axe, blade, sword, dagger, machete, club, hammer, spear, shiv, chainsaw, crowbar). Items whose names match these keywords are treated as weapons.

Skill Checks in the Inspector UI

The Inspector UI shows player skills in the character editor panel. Skills can be edited inline as numeric values. There is no skill progression system — skill values are set manually by the user/developer or imported from character library files.

Skill-Based Action Resolution

Skills affect gameplay through:

- 1. **Direct skill_check() calls:** Trigger systems and commands can call skill_check() to resolve actions (e.g., fumble_around() for discovering hidden ways uses Perception DC 12)
- 2. **Item skill checks:** Items in the library can have skill_check properties (e.g., {"skill": "Athletics", "DC": 15}) — but this is stored in library data and checked manually by triggers
- 3. **Missing skill checks:** When a player lacks a required skill, the roll defaults to a flat 1d20 + 0

Skill Assignment via API

- POST /api/players creates a player with optional skills field
- POST /api/players/<name> updates skills via {"skills": {"Perception": 4, "Athletics": 2}}
- POST /api/players/import sets skills during character import

Frontend Display

Skills are displayed in the Inspector UI's character editor panel. Each skill shows its name and current value as an editable numeric input. Custom skills (non-default) are displayed alongside the defaults.

Related tasks

- [[task-3-equipment_system|task-3: Equipment system]]
- [[dev_tasks/review/items/task-54-weapon_system|task-54: Weapon system]]

Traits System

Traits are predefined character modifiers that alter gameplay through a tag-and-effect system. Each trait is defined in engine/traits.py with a unique ID, name, description, category, and a set of engine-recognized effects.

Design direction (2026-08-07): The full v2 design — traits vs conditions split, condition catalog, save_on world-event consequences, acquired traits — lives in [[Rules Engine/Trait & Condition System (Design)|Trait & Condition System (Design)]], tracked as [[dev_tasks/review/characters/task-trait-condition-system-v2|task: Trait & Condition System v2]]. This page documents the current implementation.

Architecture

Traits follow a registry/effect pattern:

- 1. **Definitions:** TRAIT_DEFINITIONS dict in engine/traits.py:71-233 — all known traits and their effects
- 2. **Assignment:** Each player has a traits dict (player.py:82-85) mapping trait_id → param_value
- 3. **Resolution:** TraitSystem class (engine/traits.py:239-404) resolves trait IDs into engine-readable effect values at runtime
- 4. **Library Files:** data/library/traits/*.json — one file per trait, synced with the engine definitions

The engine code never hard-codes trait names outside of traits.py . All trait effects are looked up dynamically through TraitSystem static methods.

Trait File Format

Trait library files (e.g., data/library/traits/dark_vision.json):

```
{
  "id": "dark_vision",
  "name": "Dark Vision",
  "description": "Can see in complete darkness.",
  "category": "physical",
  "params": null
}
```

Field	Type	Description
id	string	Trait identifier, matches TRAIT_DEFINITIONS key
name	string	Human-readable display name
description	string	What the trait does
category	string	"physical" or "mental"

Field	Type	Description
params	object null	Parameterized trait config: {type, label, placeholder}

Currently 29 traits are defined in `TRAIT_DEFINITIONS`.

Exploration / Novelty Traits (task-136)

These traits drive the Entertainment novelty system (`movement.py:226-240`, `tick_manager.py:127-143`, `item_actions.py`, `player.py`):

Trait	Effect on Entertainment	Effect on behavior
curious	+50% from new places/things	More likely to examine items, explore exits
adventurous	Entertainment doesn't decay in unfamiliar areas	More willing to take risks, go somewhere unknown
homebody	No boost from new places or things	Reluctant to leave familiar areas
wanderlust	Gains Entertainment from moving between areas even if visited before	Prefers to keep moving, rarely stays put
impatient	Faster Entertainment decay when inactive	Acts before considering consequences
patient	Slower Entertainment decay; tolerates repetitive activities	Less driven by boredom, stays put longer

Entertainment novelty boosts are granted for: first visit to an area (+15, `movement.py`), first discovery of an item (+8, `ItemActions._register_item_discovery`), and first meeting a new character (+10, `Player.register_first_meeting` / `update_relationship`).

Trait Assignment

Traits can be assigned to characters via:

- The Inspector UI (traits/tags panel in the character inspector)
- API: `POST /api/players/<name>` with `{"traits": {"dark_vision": true, "fast_healer": true}}`
- API: `POST /api/registry/characters` to save to the library
- Programmatically: `player.traits["dark_vision"] = True`

The `tags` field (`player.py:86-88`) is separate from traits — tags are free-form identity markers like `"vampire"`, `"faction:guard"`, `"synthetic"`. Traits have engine effects; tags are checked by items/triggers/conditions.

TraitSystem API

Query Methods

All static methods on `TraitSystem`:

Method	Returns	Description
<code>get_definition(trait_id)</code>	dict or None	Get the trait definition
<code>has_trait(player, trait_id)</code>	bool	Check if player has the trait
<code>get_trait_param(player, trait_id)</code>	any	Get parameter value for parameterized traits
<code>has_effect(player, effect_key)</code>	bool	Check if any trait grants an effect
<code>get_effects(player, effect_key)</code>	list	Collect all effect values for a key
<code>get_first_effect(player, effect_key)</code>	any	Get first matching effect value
<code>get_vital_multipliers(player)</code>	dict	Merged vital multipliers
<code>get_action_cost_mods(player)</code>	dict	Additive action cost modifiers
<code>get_sense_blocked(player)</code>	str or None	Blocked sense ("sight"/"hearing")
<code>get_disabled_slots(player)</code>	set	Disabled equipment slot names
<code>get_allergen_tag(player)</code>	str or None	Allergen this player reacts to
<code>get_energy_curve(player)</code>	dict or None	Energy curve config
<code>get_hp_regen_multiplier(player)</code>	float	HP regen multiplier
<code>is_immune_to_condition(player, condition)</code>	bool	Condition immunity check
<code>process_tick_effects(player, tick, area_node)</code>	list[str]	Apply per-tick trait effects

Effect Keys and What They Do

VITAL_MULTIPLIER (key: "vital_multiplier")

Multiplies the baseline decay rate for a specific vital. Applied in `tick_turn()` (`tick_manager.py:108-114`):

```
rate = p.decay_rates.get(stat, default_decay)
mult = trait_multipliers.get(stat, 1.0)
p.vitals[stat] = max(0, p.vitals[stat] - int(rate * mult))
```

Example: `glutton` multiplies Hunger decay by 2.0 → hunger drops twice as fast.

ACTION_COST_MOD (key: "action_cost_mod")

Additively modifies action costs. Applied in `apply_action()` (`tick_manager.py:27-31`):

```
trait_mods = TraitSystem.get_action_cost_mods(target)
for tk, tv in trait_mods.items():
    lk = str(tk).lower()
    cost[lk] = max(0, int(cost.get(lk, 0)) + tv)
```

Example: `hardy` reduces energy cost by 1 (`"energy": -1`).

DARK_VISION (key: "dark_vision")

Boolean. Player can see in total darkness (light level <= 20). Both `dark_vision` and `darkvision` trait IDs provide this effect (`traits.py:198-211`).

IS_SLASHER (key: "is_slasher")

Boolean. Horror monster flag that:

- Exempts the character from all vital decay (`tick_manager.py:104-106`)
- Grants dark vision
- Is checked by `player_manager.is_slasher()` for hunt targeting
- Both `slasher` and `is_slasher` trait IDs provide this

BLOCK_SENSE (key: "block_sense")

Blocks either `"sight"` or `"hearing"` . Affects narration (the LLM is told the character cannot perceive that sense).

Traits: `blind` (blocks sight), `deaf` (blocks hearing).

DISABLE_SLOT (key: "disable_slot")

Disables an equipment slot name. The character cannot equip items there.

Trait: `one_armed` disables `"hand_right"` .

HP_REGEN_MULTIPLIER (key: "hp_regen_multiplier")

Multiplies the natural HP regeneration rate (default 1 HP/tick when conditions are met). Applied multiplicatively in `tick_turn()` (`tick_manager.py:275-277`):

```
regen_base = 1
regen_mult = TraitSystem.get_hp_regen_multiplier(p)
p.vitals["HP"] = min(100, p.vitals["HP"] + max(1, int(regen_base * regen_mult)))
```

Traits: `fast_healer` (2.0x), `slow_healer` (0.5x).

ENERGY_CURVE (key: "energy_curve")

Defines a peak hour and off-peak energy modifier. Applied in `process_tick_effects()` (`traits.py:378-385`):

```
peak = curve.get("peak_hour", 12)
off_mod = curve.get("off_peak_mod", -2)
current_hour = (tick // 60) % 24
if abs(current_hour - peak) > 4:
    player.vitals["Energy"] = max(0, player.vitals.get("Energy", 50) + off_mod)
```

Traits: `night_owl` (peak 22, -2 off-peak), `morning_person` (peak 6, -2 off-peak).

ALLERGIC_TO (key: "allergic_to")

Parameterized trait. Player takes damage when near items/areas with a matching tag. Applied in `process_tick_effects()` (`traits.py:368-376`).

```
allergen = TraitSystem.get_allergen_tag(player)
if allergen and area_node:
    area_tags = area_node.properties.get("tags", [])
    item_tags = _collect_item_tags_in_area(area_node)
    if allergen in area_env.get("air", "") or allergen in area_tags or allergen in item_tags:
        player.vitals["HP"] = max(0, player.vitals.get("HP", 100) - 3)
```

Trait: `allergic` with `params`: `{type: "string", label: "Allergen tag"}`. The player must specify e.g. `{"allergic": "pollen"}`.

GROUP_ENERGY_DRAIN (key: "group_energy_drain")

Per-tick energy drain when more than N other characters are in the same room. Applied indirectly through `tick_turn()`.

Trait: `introvert` (group drain = -2 energy).

SOCIAL_GAIN (key: "social_gain")

Per-tick social vital gain from being near others.

Trait: `extrovert` (social gain = +2).

NO_ENTERTAINMENT_DECAY (key: "no_entertainment_decay")

Boolean. Prevents Entertainment from decaying naturally. In fact, Entertainment increases by the decay rate each tick (`tick_manager.py:116-118`).

Trait: `apathetic`.

WAKE_THRESHOLD (key: "wake_threshold")

Minimum noise level required to wake this character (1=whisper, 5=scream). Lower = harder to wake.

Traits: `light_sleeper` (threshold 3), `heavy_sleeper` (threshold 1).

IMMUNE_TO_CONDITION (key: "immune_to_condition")

Grants immunity to a specific condition. Checked by `is_immune_to_condition()`.

Trait: `immortal` provides immunity to "dead" — HP stops at 1.

Complete Trait Reference

Trait ID	Name	Category	Effects
<code>glutton</code>	Glutton	physical	Hunger decay ×2.0
<code>cleanfreak</code>	Clean Freak	physical	Hygiene decay ×1.5
<code>night_owl</code>	Night Owl	physical	Energy peak at 22:00, -2 off-peak
<code>morning_person</code>	Morning Person	physical	Energy peak at 06:00, -2 off-peak
<code>fast_healer</code>	Fast Healer	physical	HP regen ×2.0
<code>slow_healer</code>	Slow Healer	physical	HP regen ×0.5
<code>one_armed</code>	One-Armed	physical	Disables <code>hand_right</code> slot
<code>small_bladder</code>	Small Bladder	physical	Bladder fill rate ×1.5
<code>big_bladder</code>	Big Bladder	physical	Bladder fill rate ×0.5
<code>blind</code>	Blind	physical	Blocks sight
<code>deaf</code>	Deaf	physical	Blocks hearing
<code>introvert</code>	Introvert	mental	Group energy -2, social gain 0
<code>extrovert</code>	Extrovert	mental	Group energy 0, social gain +2
<code>apathetic</code>	Apathetic	mental	No entertainment decay

Trait ID	Name	Category	Effects
allergic	Allergic	physical	HP -3/tick near allergen
light_sleeper	Light Sleeper	physical	Wake threshold 3
heavy_sleeper	Heavy Sleeper	physical	Wake threshold 1
immortal	Immortal	physical	Immune to dead condition
dark_vision	Dark Vision	physical	Can see in darkness
darkvision	Dark Vision (Alt)	physical	Can see in darkness
slasher	Slasher	physical	Is slasher + dark vision
is_slasher	Is Slasher (Alt)	physical	Is slasher + dark vision
hardy	Hardy	physical	Action energy cost -1
size_tiny	Tiny Size	physical	~45 cm tall — fits any passage
size_small	Small Size	physical	~90 cm tall — crowds normal doorways slightly
size_normal	Normal Size	physical	~170 cm — the default when no size trait is set
size_huge	Huge Size	physical	~3.4 m tall — must crawl small tunnels, blocked by tiny
size_giant	Giant Size	physical	~7 m tall — blocked by anything under huge
size_titanic	Titanic Size	physical	~14 m+ — blocked by anything under giant

Size Traits (task-187)

The six `size_*` traits are **mutually exclusive** and drive the passage system (see [[World Building/Doors & Connections]]): ways carry a `max_size` property, and `engine/size.py` resolves a tier index from whichever `size_*` trait is present (default `normal` when none). One tier over a way's `max_size` auto-crawls; two or more tiers over blocks the move entirely. Crawl is flavor + gating only — it does not scale costs (the way's `cost.time` is a duration hint for future stateful actions, not per-action clock advancement). Size is purely a trait tier model — there is no height property.

How the Engine Applies Traits

Per-Tick Processing

`TraitSystem.process_tick_effects()` is called from `tick_turn()` (`tick_manager.py:267-270`):

1. **Vital multipliers**: Applied during decay in `tick_turn()` via `get_vital_multipliers()`
2. **Allergic reactions**: Checks room environment tags and item tags against the player's allergen
3. **Energy curves**: Checks current in-game hour against peak/off-peak configuration
4. **Condition immunity**: Checked by `conditions.py` before applying conditions

Action Cost Modification

When any action is performed, `apply_action()` (`tick_manager.py:14-46`):

1. Looks up base action costs
2. Gets trait-based action cost mods via `TraitSystem.get_action_cost_mods()`
3. Applies additive modifiers (e.g., `hardy` reduces energy cost)

HP Regen

Each tick, if the player is well-fed (Hunger > 25, Thirst > 25, Energy > 25, Sanity > 25, Temperature 35-39), natural HP regen occurs. The regen multiplier from traits is applied.

Sense Blocking

When the LLM generates narration, blocked senses are communicated via `get_sense_blocked()` so the narration omits sight or sound cues.

Trait Library CRUD

The library browser can create, read, update, and delete trait entries:

- `GET /api/registry/traits` — list all traits
- `POST /api/registry/traits` — create or update a trait entry
- `DELETE /api/registry/traits/<id>` — delete a trait

Storage is in `data/library/traits/` as individual JSON files.

Related tasks

- [\[\[dev_tasks/review/characters/task-24-traits_conditions_emotions_editor|task-24: Traits, conditions, emotions editor\]\]](#)
- [\[\[dev_tasks/inprogress/items/task-106-tag-library-and-multiselect|task-106: Tag library and multiselect\]\]](#)

Vitals System

Vitals are numeric meters (0-100, except Temperature) that track a character's physical and mental state. They decay over time and must be maintained through actions like eating, drinking, resting, and socializing.

Vitals Reference

Vital	Default	Min	Max	Decay/Tick	Critical at 0
HP	100	0	Max_HP	0 (damage only)	Death
Max_HP	100	—	—	—	—
Energy	100	0	100	1	Unconscious → Death (after 3x)
Hunger	100	0	100	1	HP -1/tick
Thirst	100	0	100	1	HP -2/tick
Hygiene	100	0	100	1	—
Social	100	0	100	1	Sanity penalty
Bladder	0	0	100	— (fills +1/tick)	Hygiene -30 when hitting 100
Sanity	100	0	100	1	HP -1/tick
Entertainment	100	0	100	1	Sanity penalty
Temperature	37.0	~25	~45	—	HP/Energy damage at extremes

(`player.py:60-67`)

Baseline Decay

Every tick, `tick_turn()` (`tick_manager.py:64-318`) applies baseline decay to all non-dead, non-slasher characters:

```
for stat, default_decay in self.player_manager.baseline_decay.items():
    if stat in p.vitals and stat != "Temperature":
        rate = p.decay_rates.get(stat, default_decay)
        mult = trait_multipliers.get(stat, 1.0)
        p.vitals[stat] = max(0, p.vitals[stat] - int(rate * mult))
```

Bladder is **not** in `baseline_decay` — it fills toward 100 separately (see [Bladder = 100](#)).

Each stat decays by its `decay_rate` × `vital_multiplier` per tick. Bladder is the exception — it fills instead of decays (see below). Characters can override decay rates per-stat via `decay_rates` (`player.py:69-73`).

Critical Vitals Effects

HP = 0 → Death

When HP reaches 0, the character dies. Cause of death is determined from other vitals:

```
cause_parts = []
if hunger <= 0: cause_parts.append("starvation")
if thirst <= 0: cause_parts.append("dehydration")
if sanity <= 0: cause_parts.append("madness")
if temperature < 30: cause_parts.append("hypothermia")
if temperature > 42: cause_parts.append("heat stroke")
```

(`tick_manager.py:166-183`)

On death:

- State is set to `"dead"`
- A body item is spawned
- Ghost mode can be enabled to continue playing

Energy = 0 → Unconscious → Death

When Energy reaches 0:

1. Character becomes "unconscious" with `state_timer = 5`
2. Exhaustion count increments
3. On 3rd exhaustion: character dies from "exposure" (`tick_manager.py:128-137`)

Hungry = 0, Thirst = 0, Sanity = 0

Each causes HP damage per tick:

- Hunger = 0: -1 HP/tick
- Thirst = 0: -2 HP/tick
- Sanity = 0: -1 HP/tick

(`tick_manager.py:141-149`)

Bladder = 100 (Full)

Bladder is unique — it **fills** over time instead of decaying: 0 = empty (relieved), 100 = full (need to go). On the tick it first hits 100, it triggers a -30 Hygiene penalty (`tick_manager.py:161-162`).

Fill rate is influenced by Thirst:

- Thirst > 75 (well hydrated): +2/tick
- Thirst 25–75: +1/tick (normal)
- Thirst < 25 (dehydrated): +0/tick (body conserves water)

Low Social / Low Entertainment

Both cause Sanity penalties:

- Social < 25: -2 Sanity/tick; < 50: -1 Sanity/tick
- Entertainment < 25: -2 Sanity/tick; < 50: -1 Sanity/tick

(`tick_manager.py:221-231`)

Entertainment Gains (novelty, task-136)

Entertainment rises naturally from novelty, tracked per character via `visited_areas` and `discovered_items` sets:

Source	Base boost	Where
First visit to an area	+15	<code>movement.py:226-240</code>
First discovery of an item (examine/take)	+8	<code>item_actions.py:_register_item_discovery</code>
First meeting a new character	+10	<code>player.py:register_first_meeting / update_relationship</code>

Trait modifiers: `curious` ×1.5, `homebody` 0. All boosts clamp at 100. Repeated visits/discoveries/meetings give nothing (set-based diminishing returns). Per-tick modifiers: `impatient` -3 when energetic, `patient` +1, `adventurous` +2 in unfamiliar areas, `no_entertainment_decay` (from `adventurous`) cancels decay (`tick_manager.py:127-143`).

Vitals Need Messages

When a vital crosses a threshold (75→50, 50→25, 25→10), the active player receives a warning message (`tick_manager.py:67-84, 154-164`):

Vital	75	50	25	10
Energy	"getting a bit tired"	"quite weary"	"exhausted"	"barely stay awake"
Hunger	"stomach rumbling"	"quite hungry"	"famished"	"so hungry you feel weak"
Thirst	"bit parched"	"throat is dry"	"very thirsty"	"extremely dehydrated"
Hygiene	"not as fresh"	"grimy"	"quite dirty"	"clean up immediately"
Social	"bit lonely"	"miss people"	"isolated"	"desperately crave contact"
Bladder (fills 1)	"bathroom soon"	"uncomfortably full"	"need a bathroom"	"serious discomfort"
Sanity	"bit unsettled"	"isolation getting to you"	"losing grip"	"barely hold it together"
Entertainment	"getting dull"	"need something to do"	"bored out of mind"	"monotony unbearable"

Environmental Effects on Vitals

Area environment properties affect vitals each tick (`tick_manager.py:184-260`):

Condition	Effect
Temperature > 30	Thirst -2/tick (rapid)
Temperature > 40	HP -1/tick
Temperature < 10	Energy -1/tick
Temperature < 0	HP -1/tick
Air = "stale"	Energy -1/tick
Air = "humid"	Social -1/tick
Air = "toxic"	HP -3/tick
Noise loud/dripping/scratches (while sleeping)	Energy -1/tick
Smell mold/rot/urine/etc	Hygiene -1/tick
Smell = "perfume"	Social +1/tick
Light < 20	Sanity -1/tick
Other players in room	Social +1/tick

Temperature System

Body temperature drifts toward room temperature:

- Area < 15°C: body cools at $(15 - \text{room}) \times 0.02$ per tick, min 25°C
- Area > 30°C: body heats at $(\text{room} - 30) \times 0.02$ per tick, max 45°C
- Otherwise: drifts back toward 37°C at ± 0.1 /tick

(`tick_manager.py`:232-244)

Body temperature effects:

```
| Core Temp | Effects |
|-----|-----|
| 35-37°C | Energy -1/tick |
| 33-35°C | Energy -2/tick, HP -1/tick |
| <33°C | HP -3/tick |
| 37-38°C | Thirst -1/tick |
| 38-40°C | HP -1/tick |
| >40°C | HP -3/tick |
```

(`tick_manager.py`:246-259)

Vitals Restoration

Rest / Sleep

The `rest()` command (`tick_manager.py`:325-341):

1. Sets player state to "sleeping" for N ticks
2. Each sleeping tick: Energy +3 (`tick_manager.py`:261-262)
3. Awakens after the rest period
4. Returns energy restored and current energy

Unconscious Recovery

While unconscious:

- Energy +4/tick (up to max 20)
- After 5 ticks: wakes up with Energy = 20

(`tick_manager.py`:93-102)

Natural HP Regeneration

HP regenerates 1/tick (modified by traits) when ALL conditions are met:

- Energy > 25
- Hunger > 25
- Thirst > 25
- Sanity > 25
- Temperature 35-39°C
- HP < 100

(`tick_manager.py:272-277`)

Regen amount: `max(1, int(1 × hp_regen_multiplier))` . Default is 1, doubled by `fast_healer` , halved by `slow_healer` .

Items

Items like food, water, medicine can restore vitals through triggers/effects. Items have `uses` and `action_costs` properties.

Slasher Exemption

Characters with the `is_slasher` effect (from the `slasher` trait) are exempt from all vital decay and environmental effects (`tick_manager.py:104-106`). They are horror monsters that do not need to eat, sleep, or maintain hygiene.

Death System

Causes of Death

1. **Combat**: HP reduced to 0 by attack damage
2. **Starvation**: HP depleted by hunger = 0
3. **Dehydration**: HP depleted by thirst = 0
4. **Madness**: HP depleted by sanity = 0
5. **Hypothermia/Heat Stroke**: HP/Energy depleted by temperature extremes
6. **Exhaustion**: Energy = 0 three times
7. **Toxic air**: HP drained by toxic room air
8. **Allergic reaction**: HP drained by allergen (trait)

On Death

1. `player.vitals["HP"] = 0`
2. `player.state = "dead"`
3. A body item is spawned via `spawn_body_item()` with cause of death
4. If ghost mode is enabled, the dead player can continue acting
5. The active player sees "GAME OVER: You have died from "

Dead players are excluded from:

- Vital decay processing (`tick_manager.py:90-91`)
- Simple NPC processing (`npc_behaviors.py:35`)
- Player-in-room listings (when `include_ghosts=False` , `player_manager.py:183-185`)

Vitals UI

Vitals are displayed in:

1. **Command output**: `stats()` command shows current vital values
2. **Inspector**: The character inspector shows all vitals as progress bars
3. **Need messages**: Automatic warnings when vitals cross thresholds
4. **Emotion system**: Low vitals influence emotion (e.g., low HP + damage = fear/anger)

Vitals by Character Library Examples

Characters often start with vitals below 100 to simulate pre-existing conditions:

Character	Energy	Hunger	Thirst	Hygiene	Sanity	Notes
Miki	100	80	80	100	—	Well-rested
Jake Halloway	70	45	40	85	90	Sleep-deprived, hungry
Kayla Jenkins	100	80	80	100	100	Fresh start
Kaelen Voss	66	59	49	79	98	Weary traveler, Temp 36.46°C
Sammy Lopez	100	75	70	100	95	Slightly on edge
Kyrie Johansen	100	85	85	100	100	Ready to go

Related tasks

- [\[\[dev_tasks/review/characters/task-28-character_needs_system|task-28: Character needs system\]\]](#)
- [\[\[bug_5-rat-11-10-hp-with-low-hp-warning 1|bug-5: Rat 11/10 HP with low HP warning\]\]](#)

Overview

The Light System manages room visibility through a numeric 0-100 scale, converted to a 5-level enum for gameplay restrictions. It lives in `engine/lighting.py` (class `LightingSystem`, ~80 lines). Light levels affect what actions are possible in a room and contribute to sanity decay.

Numeric Scale

Light is stored as an integer 0-100 in each room's `environment` dict:

```
env = {"light": 80} # bright
```

Default light for indoor areas created without explicit setting is 0 (pitch black). Outdoor areas (tagged `"outdoor"`) compute their light from the current game time, weather, and moon phase (task-230).

Light Level Enum

`LightingSystem.light_to_level(value)` (`engine/lighting.py:13`) converts numeric values to a 5-level string enum:

Range	Level	Gameplay Effect
≤20	<code>pitch_black</code>	look blocked, examine blocked, take blocked, use blocked
≤40	<code>dim</code>	look shows dim description (fumble around works)
≤70	<code>normal</code>	Normal visibility, no restrictions
≤90	<code>bright</code>	No additional restrictions
>90	<code>blinding</code>	No additional restrictions (no blindness mechanic)

The mapping also handles string enum values directly (e.g., `"dim"` → `"dim"`) and defaults unknown values to `"normal"`.

Reverse Mapping

`get_light_int()` (`engine/lighting.py:32`) converts string enums back to approximate integers:

Level	Approx. Value
<code>pitch_black</code>	10
<code>dim</code>	30
<code>normal</code>	55
<code>bright</code>	80
<code>blinding</code>	95

Gameplay Restrictions

From `AGENTS.md` and enforced by the backend action handlers:

`pitch_black` (≤20)

- `look` — Blocked. Player must use `fumble_around()` instead
- `examine` — Blocked
- `take` — Blocked
- `use` — Blocked

The backend checks light level in `area_description.py` (for look/examine) and `item_actions.py` (for take/use). When blocked, the system returns a contextual message.

`dim` (21-40)

- `look` — Shows a reduced "dim" description (room is visible but details are unclear)
- `fumble_around()` — Works to find hidden items/ways

`normal` (41-70)

Standard visibility, all actions available.

bright (71-90)

Standard visibility. No special bonuses.

blinding (>90)

Standard visibility. No special penalties currently implemented.

Ambient Light Calculation

`get_ambient_light(area_id, env)` (`engine/lighting.py:74`) computes a room's effective light level considering:

1. **Outdoor rooms:** computed from time of day + weather + moon phase (task-230)
2. **Indoor rooms:** static `environment.light` value (default 0 = pitch black)
3. **Light spill** from adjacent areas through open/transparent ways
4. **Light-producing items** in the area or carried/equipped

Time-of-Day Light (Outdoor Rooms)

For rooms tagged "outdoor", `get_time_of_day_light()` returns a base value:

Time	Light (clear weather)
05:00–06:00 (dawn)	40 (dim)
06:00–18:00 (day)	80 (bright)
18:00–19:00 (dusk)	40 (dim)
19:00–05:00 (night)	15 (pitch_black)

Weather modifier (task-227): clear = 100%, cloudy = 70%, rainy = 50%, stormy = 30%, foggy = 40%, windy = 80%.

Moon phase bonus (task-229): only at night, only outdoor. Full moon = +25, gibbous = +15, quarter/waning = +10, crescent = +5, new moon = +0. Stormy/foggy weather halves or nullifies the bonus.

Light Spill

For each adjacent room connected via an open door:

```
spill = source_light * 0.5
```

Transparent doors (windows, glass doors) allow light spill without being open:

```
spill = source_light * 0.3
```

The effective light is `max(own_light, best_spill)`. Only the brightest spill is used. Light only travels through ways with `current_state == "open"` (or `"transparent": true` for glass).

Tunable (task-304): the spill multiplier is the `light.spill_factor` engine-config key (default `0.5`), changeable live via Settings → **Engine Config** — no code edit needed. See [[UI & Settings/Engine Config]].

Dark Vision

`can_see_in_dark(player_manager, player_name)` (`engine/lighting.py:67`) checks if a player can ignore light restrictions:

Returns `True` when any of:

1. **Player is dead** (ghost state) — `player.state == "dead"`
2. **Player has dark_vision trait** — checked via `TraitSystem.has_effect(player, "dark_vision")`
3. **Player is a slasher** — slasher trait includes `dark_vision`

Fumble Around

`fumble_around()` (via `NarrationSystem`, delegated from `virtual_world_engine.py:593`) provides an alternative action for dark areas. This allows players to:

- Discover hidden ways (Perception DC 12)
- Get a tactile description of the room
- Find items by touch

This is the primary way to interact with `pitch_black` areas.

Sanity Decay

In `TickManager.tick_turn()` (`tick_manager.py:213-215`):

```
light = self.lighting.get_ambient_light(area_node.id, env)
if light < 20:
    p.vitals["Sanity"] = max(0, p.vitals["Sanity"] - 1)
```

Each tick, characters in areas with ambient light below 20 lose 1 Sanity. This applies to all alive, non-slasher players.

Ghost Immunity

Ghosts (dead characters) completely ignore light level:

- `can_see_in_dark()` returns `True` for dead players
- The sanity decay check is skipped for dead players (they skip the whole vitals block due to `if p.state == "dead": continue`)
- Ghosts can see, move, and interact in complete darkness

Light-Producing Items (Graph Scan)

Light contribution is determined by **graph scanning** (`lighting.py:get_item_light_contribution()`). Items in the area or carried/equipped by characters in the area are scanned for:

1. `"light_source"` tag — marks the item as capable of emitting light
2. `current_state == "lit"` — whether the item is currently on
3. `light_level` property — numeric (0–100) or string enum (`"dim"` , `"normal"` , etc.)

The system sums all matching items' `light_level` values (capped at 100). This replaces the old per-player `item_statuses` + `item_active_effects` system.

Toggleable Items

Items with the `"toggleable"` tag can be turned on/off via `use <item>` or `toggle <item>`:

- `engine/toggleable_items.py:toggle_item_status()` flips `current_state` between `"unlit"` and `"lit"`
- On turn-on: checks `uses > 0`, decrements uses by 1
- Fires `on_toggle_on` / `on_toggle_off` triggers
- If uses hit 0: flips back to `"unlit"`, fires `on_depleted`
- Verb adapts by tag: `electric` / `synthetic` → "turn on/off", default → "light/extinguish"

Example item properties for a hand lamp:

```
{
  "name": "hand lamp",
  "uses": 60,
  "light_level": "normal",
  "current_state": "unlit",
  "tags": ["light_source", "toggleable", "oil_lamp"],
  "triggers": [
    {"trigger_type": "on_tick", "effects": [{"type": "adjust_uses", "params": {"node_id": "self", "delta": -1}}]},
    {"trigger_type": "on_depleted", "effects": [{"type": "message", "params": {"message": "Your lamp goes out."}}]}
  ]
}
```

Per-Tick Fuel Drain

`tick_manager.py:283-308` scans carried/equipped items with `current_state == "lit"` each tick. For each lit item:

1. Fires `on_tick` triggers (which should contain `adjust_uses` to drain fuel)
2. When `uses` reaches 0: sets `current_state = "unlit"`, fires `on_depleted`

Trigger Integration

Triggers can modify light through effects:

set_environment

```
{"type": "set_environment", "params": {"light": "bright"}}
# or numeric: handles int or enum strings
```

Note: `set_environment` converts light values through `_light_to_level()` (`effects.py:266`).

adjust_environment

```
{"type": "adjust_environment", "params": {"light": -20}}
```

Incrementally adjusts light, clamped to -50 to 100 range.

Environment Properties

Light is stored alongside other environment properties on room nodes:

```
area_node.properties["environment"] = {
    "light": 80,          # 0-100 integer
    "temperature": 21,    # Celsius
    "air": "fresh",       # fresh, stale, humid, toxic
    "smell": "neutral",   # neutral, mold, rot, etc.
    "noise": "quiet",     # quiet, loud, dripping, etc.
}
```

UI Representation

- Light level is shown in the room description
- Inspector shows current light value for the room
- Agent engine JS uses `_lightToLevel()` to filter what items/actions to include in LLM prompts (the interest attention list sorts *within* the resulting light branch — see Agent Engine.md → Room Context)
- The frontend conditionally enables/disables action buttons based on light level

Related tasks

- [[dev_tasks/todo/environment/task-229-moon-phase-system|task-229: Moon Phase]]
- [[dev_tasks/todo/environment/task-230-outdoor-lighting-time-of-day|task-230: Time-of-Day Lighting]]
- [[dev_tasks/todo/environment/task-233-area-status-system|task-233: Area Status System]]
- [[dev_tasks/todo/environment/task-234-trigger-environment-forecast-overrides|task-234: Trigger & GM Integration]]
- [[dev_tasks/review/environment/task-18-fireplace_lighting_recipe|task-18: Fireplace lighting recipe]]
- [[dev_tasks/done/environment/task-110-see-through-windows|task-110: See-through Windows]]
- [[UI & Settings/Engine Config|Engine Config (task-304)]] — light spill factor is now a live tunable

Body Temperature

Core Temp Vital

Each player has a `Temperature` vital tracked separately from room temperature:

```
player.vitals["Temperature"] = 37.0 # default: 37°C (human body temp)
```

Distinct from Hunger/Thirst/Energy vitals — uses floating-point values instead of 0-100 integers.

Range

Limit	Value	Effect
Normal	37.0°C	Baseline
Min	25.0°C	Hard clamp
Max	45.0°C	Hard clamp

Temperature Drift (Per Tick)

In `TickManager.tick_turn()` (`tick_manager.py:232-244`), body temperature drifts toward the effective (equipment-adjusted) temperature:

```
area_temp = float(effective_temperature(float(env.get("temperature", 21)), bonuses))
core_temp = p.vitals.get("Temperature", 37.0)

if area_temp < 5:
    drift = (5 - area_temp) * 0.02
    p.vitals["Temperature"] = max(25.0, core_temp - drift)
elif area_temp > 35:
    drift = (area_temp - 35) * 0.02
    p.vitals["Temperature"] = min(45.0, core_temp + drift)
else:
    # comfortable range — drift back toward 37°C
    if core_temp < 36.5:
        p.vitals["Temperature"] = min(37.0, core_temp + 0.1)
    elif core_temp > 37.5:
        p.vitals["Temperature"] = max(37.0, core_temp - 0.1)
```

Drift Thresholds

Effective Temp	Drift Calculation	Direction	Notes
< 5°C	<code>(5 - eff_temp) × 0.02</code> per tick	Cooling	Body temp drops toward 25°C
5–35°C	<code>±0.1</code> per tick	Toward 37°C	Comfort zone — body self-regulates
> 35°C	<code>(eff_temp - 35) × 0.02</code> per tick	Warming	Body temp rises toward 45°C

Time-to-Danger Examples

Effective Temp	Drift Rate	Time to Dangerous (from 37°C)
-20°C	$(5 - (-20)) \times 0.02 = 0.50^\circ\text{C}/\text{tick}$ cooling	8 ticks to 33°C hypo
0°C	$(5 - 0) \times 0.02 = 0.10^\circ\text{C}/\text{tick}$ cooling	40 ticks to 33°C
5–35°C	0.1°C/tick toward 37°C	Never (stays comfortable)
40°C	$(40 - 35) \times 0.02 = 0.10^\circ\text{C}/\text{tick}$ warming	30 ticks to 40°C stroke
50°C	$(50 - 35) \times 0.02 = 0.30^\circ\text{C}/\text{tick}$ warming	10 ticks to 40°C, 17 to 42°C death

Drift rate is **constant per tick** (not proportional to core temp) — time-to-threshold calculations are linear.

Core Temperature Damage Table

After drift, the **new** core temp value causes damage each tick:

Core Temp	Effect	Condition
35.0–36.9°C (mild cold)	Energy -1	Shivering
33.0–34.9°C (severe cold)	Energy -2, HP -1	Hypothermia onset
< 33°C (critical cold)	HP -3	Severe hypothermia
37.1–38.0°C (mild heat)	Thirst -1	Sweating
38.1–40.0°C (severe heat)	HP -1	Heat exhaustion
> 40°C (critical heat)	HP -3	Heat stroke

Death Check

Core Temp	Cause of Death
< 30°C	Hypothermia
> 42°C	Heat stroke

Death also triggers `spawn_body_item` — a body item appears in the area with a death cause description.

From `tick_manager.py:174-177` :

```
if p.vitals.get("Temperature", 37) < 30:
    cause_parts.append("hypothermia")
if p.vitals.get("Temperature", 37) > 42:
    cause_parts.append("heat stroke")
```

HP Regeneration Gate

HP regeneration (natural healing) is blocked unless:

- Energy > 25, Hunger > 25, Thirst > 25, Sanity > 25
- **AND** core temp is 35–39°C

Being too cold or too hot prevents healing entirely, even with food and rest.

Indirect Cold Death (Exhaustion)

Even if core temp stays above 30°C, cold drains Energy. When Energy hits 0:

1. Player falls unconscious (`p.state = "unconscious"`)
2. `exhaustion_count` increments
3. After 3 exhaustion cycles → death from **exposure**

Cold can kill without reaching hypothermia temps if you can't warm up (`tick_manager.py:132-137`):

```
if p.exhaustion_count >= 3:
    p.state = "dead"
```

Key Code Locations

Concern	File	Lines
Player initialization	<code>player.py</code>	71-78
Core drift + damage	<code>tick_manager.py</code>	196-275
Death check	<code>tick_manager.py</code>	185-192
Exhaustion death	<code>tick_manager.py</code>	138-149
HP regen gate	<code>tick_manager.py</code>	288-293
Vital detail API	<code>routes/players.py</code>	361-432

Environment Temperature

Room Property

Stored as an integer in each room's `environment` dict:

```
area_node.properties["environment"]["temperature"] = 21 # default: 21°C
```

Property	Default	Range
<code>temperature</code>	21°C	-50 to 100°C

Default for new areas is 21°C (room temperature). The `adjust_environment` effect clamps to -50..100, but the engine itself has **no clamp** on the value — volcanic caves at 200°C or dragon fire at 500°C work fine (drift just scales proportionally).

How It's Set

1. **Default** — 21°C for new areas
2. **Effects** — `set_environment` (absolute) or `adjust_environment` (incremental) triggers
3. **Manual editing** — Inspector panel or world data
4. **NPC behaviors** — NPC actions can call `set_environment`

Heat Propagation

Temperature propagates between areas connected by open ways each tick via `propagate_temperature()` in `environment_propagation.py:24-69`. Areas tagged `exterior` act as infinite reservoirs — their temperature never changes, so an open door to the blizzard pulls the room's temperature toward the exterior temp. Closing doors stops propagation.

Wind in connected areas accelerates heat propagation (1.0× to 3.0× based on wind strength). See task-231.

Heat sources (`heat_source` tag with `target_temperature` / `heating_rate`) push room temp toward their target via `apply_heat_sources()` at `environment_propagation.py:110-149`, called before propagation each tick. See details below.

Tag an area as `exterior` via the area inspector's tag section or by adding `"exterior"` to `properties.tags`.

Temperature-Producing Items

Toggleable items can modify room temperature through `environment_modifiers`:

```
{
  "toggleable": true,
  "effect_target": "room",
  "environment_modifiers": {"temperature": 15},
  "uses": 200
}
```

When toggled on, the item adds its temperature modifier to the room's environment. These effects are tracked in `_item_active_effects` and synced across room changes.

Common Items

- **Fireplace** — large temperature boost (heating)
- **Stove** — medium temperature boost (cooking + heating)
- **Torch/Candle** — small temperature boost (minimal)
- **Air conditioner / Fan** — negative temperature modifier (cooling)

Heat Source Tag

Items with the `heat_source` tag and `target_temperature` / `heating_rate` properties actively heat their area via `apply_heat_sources()` in `environment_propagation.py:112-151`, called from `tick_manager.py:336-338`. Unlike `environment_modifiers` (instant toggle), heat sources apply gradual heating each tick toward their target.

```
{
  "tags": ["heat_source"],
  "light_level": "normal",
  "heat_source_target": 25,
  "heat_source_rate": 0.3
}
```

Applied in: `engine/environment_propagation.py:112-151`, called from `engine/tick_manager.py:336-338`.

Area Descriptions

Environment descriptions and temperature warnings use **effective temperature** (feels-like after insulation + wind chill + humidity), not raw ambient. Effective temperature is calculated in `equipment_bonuses.py`:

```
eff_temp = ambient + insulation + wind_chill + humidity_mod
```

- Wind chill is resisted by item `wind_resistance` (0–100%)
- Wet insulation penalty is resisted by item `water_resistance` (0–100%)
- Humidity modifier: dry = 0, humid = +2/-1, wet = +3/-2, flooding = +4/-3

Warnings trigger at:

- `effective_temp` < -5 — bitter cold warning
- `effective_temp` < 5 — cold warning
- `effective_temp` > 32 — heat warning
- `effective_temp` > 40 — extreme heat warning

See [[Temperature/UI & Display]] for thresholds.

Air Quality Interaction

Air Quality	Effect
"stale"	Energy -1/tick
"humid"	Social -1/tick
"toxic"	HP -3/tick

Hot + humid conditions are especially punishing (Thirst drain + discomfort).

Key Code Locations

Concern	File	Lines
Area environment default	area.py	8-14
Temperature propagation	environment_propagation.py	24-69
Apply heat sources	environment_propagation.py	110-149
Propagation + heat source call in tick	tick_manager.py	320-325
Environment trigger effects	effects.py	278-308, 394-428
Area descriptions	area_description.py	141-175, 202-210

Equipment & Temperature

Insulation

Equipped items with an `insulation` property shift the effective ambient temperature before it affects the player:

```
effective_temp = ambient_temp + sum_of_all_insulation
```

Positive insulation warms (traps body heat), negative cools (wicks heat away). Values **stack** across worn items.

Examples

Items	Total Insulation	At -12°C Feels Like	At 35°C Feels Like
Fur coat (<code>insulation: 14</code>)	+14	2°C	49°C
Coat + scarf (<code>14 + 3</code>)	+17	5°C	52°C
EVA suit (<code>insulation: 15</code>)	+15	3°C	50°C

Insulation helps in cold but hurts in heat — the same coat that keeps you warm in a blizzard will cook you in a desert.

Damage Resistances

Items with the `resistance` tag define type-based damage reduction. These **do not affect temperature** — they only reduce damage from typed attacks:

```
{ "resistances": { "fire": 5, "cold": 3 }, "tags": ["resistance"] }
```

Applied in:

- **Combat** — when weapon has `damage_type`
- **Environment** — toxic air damage

Toxic Air Protection

Toxic air damage (HP -3/tick) is mitigated by `toxic_resistant` resistance:

```
damage = max(0, 3 - toxic_resistance)
```

Any equipped item with the `toxic_resistant` tag grants resistance 3 (fully blocking standard toxic air).

Migration Notes

Per-type tags (`fire_resistant` , `cold_resistant` , `vacuum_sealed` , etc.) are removed. Use the single `resistance` tag + `resistances` dict instead.

Key Code Locations

Concern	File	Lines
Equipment bonuses (insulation)	<code>equipment_bonuses.py</code>	34-99

Temperature Trigger Integration

Conditions

The trigger system (`trigger_system.py`) supports two temperature-related condition types:

`temperature_below`

```
{
  "type": "temperature_below",
  "value": "10"
}
```

Returns `True` if current room temperature is below the threshold.

`temperature_above`

```
{
  "type": "temperature_above",
  "value": "30"
}
```

Returns `True` if current room temperature is above the threshold.

Both look up room temperature from `game_state._get_current_area_id()` → room node → `environment.temperature`.

Effects

Effect	What it does
<code>set_environment</code>	Override temperature (absolute), e.g. <code>{temperature: 22}</code>
<code>adjust_environment</code>	Increment/decrement temperature (relative), e.g. <code>{temperature: 5}</code>

`set_environment` (`effects.py:241`)

```
{"type": "set_environment", "params": {"temperature": 35}}
```

Replaces the temperature value entirely. Also sets other environment properties (light, smell, noise).

`adjust_environment` (`effects.py:357`)

```
{"type": "adjust_environment", "params": {"temperature": 10}}
```

Adds 10 to the current temperature, clamped to -50..100 range.

Heat Source Examples

Source	Effect	How
Fireplace (lit)	<code>set_environment</code> temp=22°C, on_tick +1°C until 28°C	Trigger effect
Radiator	On_tick <code>adjust_environment</code> +temperature	Trigger effect

Source	Effect	How
Open freezer door	On_tick adjust_environment -temperature	Trigger effect

The fireplace uses:

- on_use trigger: set_environment (temp=22, light=80, noise="crackling fire", smell="woodsmoke")
- on_tick trigger: adjust_environment (temperature+1), conditioned on temperature_below: 28

Key Code Locations

Concern	File	Lines
Temperature trigger conditions	trigger_system.py	557-597
Environment trigger effects	effects.py	278-308, 394-428

UI & Display

Vital Bar

- Range 25–45°C mapped to 0–100%: $((\text{value} - 25) / 20) \times 100$
- °C suffix on numeric display
- Color-coded bar:

Core Temp	Color	Hex
< 33°C (danger)	Red	#f85149
33–34°C (cold)	Blue	#58a6ff
35–39°C (safe)	Green	#3fb950
40°C (warm)	Yellow	#e3b341
> 40°C (danger)	Red	#f85149

Narrative Tiers

Core Temp	Description
< 33°C	"shivering uncontrollably — hypothermia setting in"
33–34°C	"shivering violently from the cold"
35°C	"cold and shivering"
38–39°C	"feeling very hot"
40–41°C	"dangerously overheated"
> 42°C	"heat is overwhelming — about to collapse"

Environment Description

Area descriptions and temperature warnings use **effective temperature** (feels-like after insulation), not raw ambient. From `area_description.py` using the authoritative `temperature_description()` and `temperature_warning()` functions:

Feels Like	env_summary text	Warning text
≥60°C	"The heat is infernal — you can't breathe."	"You are burning! Seek shelter or die!"
≥50°C	"Blazing heat — the air shimmers."	"The heat is cooking you alive!"
≥40°C	"Scorching hot."	"The intense heat is draining your energy!"
≥35°C	"Very hot."	"You're overheating — find shade or water."
≥30°C	"Hot."	"It's quite hot; you're feeling thirsty."
≥25°C	"Warm."	(none)
≥18°C	"Pleasant."	(none)
≥12°C	"Cool."	(none)
≥5°C	"Chilly."	(none)

Feels Like	env_summary text	Warning text
≥0°C	"Cold."	"The cold is biting."
≥-10°C	"Freezing."	"It's freezing! You need to warm up."
≥-25°C	"Bitterly cold."	"The cold is sapping your strength."
≥-50°C	"Arctic — the cold is lethal."	"Hypothermia is imminent — find warmth now!"
< -50°C	"Deadly cold — nothing survives."	"You are freezing to death!"

Graph Overlay

Rooms colored by environment temperature in the graph heat overlay:

- ≤ -20°C: dark blue
- -5-5°C: blue
- 15°C: lighter blue
- 25°C: neutral
- 35°C: orange
- 45°C: dark orange
- > 45°C: red

UI Panels

- Area temperature is displayed in the room info panel
- Body temperature is visible in the player stats panel
- Temperature-related effects (thirst, energy, HP changes) are logged as game messages
- Environment inspector shows temperature with the other environment properties

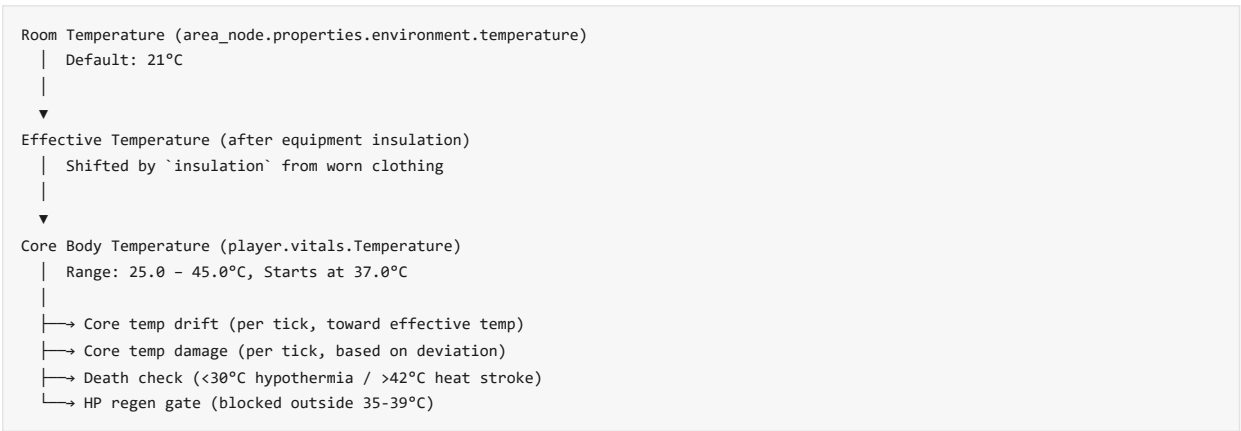
Key Code Locations

Concern	File	Lines
Temperature description (14 bands)	area_description.py	temperature_description(), temperature_warning()
Area descriptions	area_description.py	141-230
Feels-like serialization	serialization.py	_compute_feels_like()
Vital detail API	routes/players.py	361-432
Frontend vital bar	agent-view.js	233-275
Frontend narrative	agent-engine.js	180-184
Frontend alerts	ui-controller.js	118-119
Graph heat overlay	network-manager.js	375-386

Temperature System

Body temperature simulation tied to environment temperature, equipment insulation, and trigger effects. Affects HP, Energy, Thirst, and can cause death.

Data Flow



Pages

- **[[Temperature/Environment Temperature]]** — Room temp property, defaults, heat sources (`heat_source` tag, `environment_modifiers`), heat propagation (none yet), air quality
- **[[Temperature/Body Temperature]]** — Core temp vital, drift mechanics (thresholds 5°C/35°C), damage table, death checks, exhaustion death, HP regen gate
- **[[Temperature/Equipment & Temperature]]** — Insulation stacking, effective temp formula, resistance tag, toxic air protection
- **[[Temperature/Trigger Integration]]** — `temperature_below` / `temperature_above` conditions, `set_environment` / `adjust_environment` effects, heat source examples
- **[[Temperature/UI & Display]]** — Vital bar colors, narrative tiers, environment description thresholds, graph heat overlay, UI panels

Quick Reference

What	Where
Environment temp	<code>room.environment.temperature</code> , default 21°C
Core temp	<code>player.vitals["Temperature"]</code> , starts 37°C
Drift thresholds	Cold < 5°C, Hot > 35°C (effective temp)
Drift rate	$(\text{threshold} - \text{eff_temp}) \times 0.02$ per tick
Hypothermia death	Core temp < 30°C
Heat stroke death	Core temp > 42°C
Insulation formula	<code>eff_temp = ambient + insulation + wind_chill + humidity_mod</code>
Heat propagation	Temperature spreads between areas via open ways each tick (rate <code>heat.base_rate</code> = 0.05/tick, capped <code>heat.max_delta</code> = 2.0°C/tick — both tunable in Settings → Engine Config)
Description bands	14 bands from -50°C to 60°C — <code>temperature_description()</code> / <code>temperature_warning()</code>
Feels-like in state	<code>player.feels_like</code> serialized by <code>WorldSerializer._compute_feels_like()</code>

Key Code Locations

Concern	File	Lines
Player initialization	<code>player.py</code>	71-78
Core drift + damage	<code>tick_manager.py</code>	196-275
Death check	<code>tick_manager.py</code>	185-192
Exhaustion death	<code>tick_manager.py</code>	138-149
HP regen gate	<code>tick_manager.py</code>	288-293
Temperature propagation	<code>environment_propagation.py</code>	24-69
Apply heat sources	<code>environment_propagation.py</code>	110-149
Propagation + heat source call	<code>tick_manager.py</code>	320-325
Equipment bonuses	<code>equipment_bonuses.py</code>	34-99
Area environment default	<code>area.py</code>	8-14
Trigger effects	<code>effects.py</code>	278-308, 394-428
Trigger conditions	<code>trigger_system.py</code>	557-597
Temperature description (14-band)	<code>area_description.py</code>	<code>temperature_description()</code> , <code>temperature_warning()</code>
Feels-like serialization	<code>serialization.py</code>	<code>_compute_feels_like()</code>
Area descriptions	<code>area_description.py</code>	141-230
Vital detail API	<code>routes/players.py</code>	361-432
Frontend vital bar	<code>agent-view.js</code>	233-275
Frontend narrative	<code>agent-engine.js</code>	790-798

Concern	File	Lines
Frontend alerts	<code>ui-controller.js</code>	118-119
Graph heat overlay	<code>network-manager.js</code>	375-386

Related Tasks

- [[dev_tasks/review/environment/task-5-heat_propagation]] — Temperature spread between areas (done)
- [[dev_tasks/todo/environment/task-227-environment-forecast-schedule]] — Forecast schedule with weather/wind/temp mods
- [[dev_tasks/todo/environment/task-231-wind-system]] — Wind chill, wind resistance, wet insulation penalty
- [[dev_tasks/todo/environment/task-232-humidity-atmospheric-conditions]] — Humidity modifier on effective temp
- [[dev_tasks/todo/environment/task-233-area-status-system]] — Area statuses that modify temperature
- [[dev_tasks/review/environment/task-126-tag-based-light-and-heat-sources]] — Light/heat source tag system
- [[dev_tasks/review/characters/task-28-character_needs_system]] — Added the drift system
- [[dev_tasks/review/items/task-3-equipment_system]] — Added insulation + resistances
- [[dev_tasks/review/environment/task-18-fireplace_lighting_recipe]] — Fireplace as heat source example
- [[UI & Settings/Engine Config|Engine Config (task-304)]] — `heat.base_rate` / `heat.max_delta` are live tunables

Time & Weather

Overview

VirtualWorld has an in-game clock driven by a tick-based time system. Each player action consumes time ticks, and the world state advances accordingly. Environment properties (weather, wind, humidity, light, temperature, air, noise, smell) are managed through a forecast schedule system with GM override capability.

Terminology

Term	Definition
tick	A unit of game time. Each tick = <code>time_per_tick_minutes</code> (default: 5 minutes). <code>time_ticks</code> is a monotonically increasing counter. <code>advance_clock(1)</code> advances time by 1 tick.
tick_turn()	The full processing cycle that runs when a turn ends. Calls <code>conditions.process_tick()</code> , vitals decay, environmental effects, heat propagation, sound sources, NPC behavior, then <code>advance_clock(1)</code> . This is what <code>/api/turn/apply</code> invokes.
turn	One complete player action cycle: player takes action → <code>tick_turn()</code> runs → <code>/api/turn/clear</code> increments <code>turn_number</code> . One turn = one <code>tick_turn()</code> call = +1 tick = +5 game minutes.
turn_number	An integer incremented by <code>clear_turn_events()</code> AFTER <code>tick_turn()</code> completes. Used for logging, event tracking, and turn-gated conditions.
step	An NPC movement step or activity progress step. Not a global time concept — NPCs may take multiple steps per turn.

Game Clock

Managed by `TickManager` in `engine/tick_manager.py`.

Tick System

- Each turn** = 1 time tick
- Time per tick:** `time_per_tick_minutes` = 5 (configurable in `virtual_world_engine.py:47`)
- Clock start:** 08:00 by default (`clock_start_hour` = 8 , `clock_start_minute` = 0)

Time Calculation

`get_current_time()` (`tick_manager.py:69-81`):

```
total_minutes = time_ticks * time_per_tick_minutes
total_minutes += clock_start_hour * 60 + clock_start_minute
total_minutes = total_minutes % (24 * 60)
hours = total_minutes // 60
minutes = total_minutes % 60
seconds = total_seconds % 60
return f"{hours:02d}:{minutes:02d}:{seconds:02d}"
```

Time wraps at midnight (24-hour clock). Output format: `"HH:MM:SS"` (seconds always 0).

Date Tracking

`game_day`, `game_month`, and `game_year` are derived from `time_ticks` (task-228). Calendar config (`minutes_per_day`, `days_per_month`, `months_per_year`) is stored in world state and exposed via `/api/state` .

Time Advancement

- `advance_clock(ticks=1)` (`tick_manager.py:63`): Advances `time_ticks` counter only. Does not apply vital decay.
- `tick_turn()` (`tick_manager.py:83`): Advances clock AND applies all vital decay, environmental effects, condition processing, and NPC behavior. This is the main "end turn" method.
- `apply_action(action_name)` (`tick_manager.py:24`): Consumes time and energy based on action type's `ACTION_COSTS` .

Action Costs

Defined in `virtual_world_engine.py:83-92` :

Action	Time	Energy
move	1	1
open	0	1
close	0	1
look	1	0
use	1	1
take	1	1
drop	1	0
fumble	2	3

Action costs are multiplied by time ticks consumed. Trait modifiers are applied additively via `TraitSystem.get_action_cost_mods()` .

Time Advance Per Turn

`tick_turn()` always calls `advance_clock(1)` at the end (`tick_manager.py:448`). So each completed turn advances time by 5 minutes.

Rest Mechanic

`rest(minutes=10)` (`tick_manager.py:541`) sets player state to `"sleeping"` , then runs `tick_turn(skip_npcs=True)` for the duration. Each rest tick advances the clock by 1 tick and recovers Energy at +3/tick.

Forecast Schedule & Weather

Weather is managed through a **forecast schedule** (task-227) stored in world state as `forecast_schedule` . This is not a weather simulation — it is an authored schedule that `tick_turn()` applies each turn as the environmental baseline.

Forecast Modes

Mode	Description
authored	Designer places explicit entries at specific times. Most control.
deterministic	Weighted state machine with fixed probabilities (no randomness). E.g., <code>"clear": 70% → "cloudy": 20% → "windy": 10%</code> .
random	Same transition table, but uses <code>random()</code> each interval. Optional <code>seed</code> for reproducibility.
hybrid	Authored schedule defines base temperature/light; state machine layers weather on top.

Forecast Data Model

```
{
  "mode": "authored",          # "authored" | "deterministic" | "random"
  "seed": None,                # optional seed for reproducible random
  "granularity": "hourly",     # "hourly" | "weekly" | "yearly"
  "current_state": "clear",     # for state machine modes
  "transition_interval": 1,     # ticks between state rolls
  "transition_table": { ... }, # for state machine modes
  "entries": [
    {
      "offset": 0,              # minutes from period start
      "weather": "clear",       # clear | cloudy | rainy | stormy | foggy | windy
    }
  ]
}
```

```
        "wind": "breeze",      # none | breeze | wind | gale | storm | hurricane
        "temperature_mod": 0,  # +N / -N degrees
        "light_mod": 0,        # +N / -N applied to ambient light
        "air": "fresh",        # optional air override
        "message": "The morning is clear and calm."
    },
]
}
```

Period lengths:

- `hourly` : 1440 minutes (1 day) — entries repeat daily
- `weekly` : 10080 minutes (7 days) — entries repeat weekly
- `yearly` : 525600 minutes (365 days) — entries repeat yearly

GM Override

A separate `forecast_override` dict in world state supersedes the schedule. Can be:

- Set via API `POST /api/settings/forecast-override`
- Set via trigger effect `forecast_override` (task-234)
- Timed: includes `duration_ticks` and auto-reverts when expired

Integration with `tick_turn()`

In `tick_turn()`, after `advance_clock(1)`:

1. Query `ForecastSchedule` for current environment overrides
2. If active GM override exists, use that instead
3. Apply overrides to areas (globally or per-area)
4. If forecast entry changed since last tick, fire narration event using `entry["message"]`

Trigger effects (`set_environment` / `adjust_environment`) are temporary overlays. The forecast is the **baseline** that gets re-applied each turn.

Moon Phases

A deterministic 30-day cycle based on `game_day` (task-229):

Days (mod 30)	Phase	Night Light Bonus
0–4	new_moon	+0
5–9	crescent	+5
10–14	quarter	+10
15–19	gibbous	+15
20–24	full_moon	+25
25–29	waning	+10

Moon phase affects outdoor night ambient light (task-230). Full moon makes outdoor nights visibly brighter; new moon keeps them pitch black.

Time-of-Day Lighting

Outdoor rooms (tagged `"outdoor"`) get ambient light from the current game time, weather, and moon phase — not from a static `environment.light` value (task-230).

Light Levels by Time of Day

Time	Light (clear weather)
05:00–06:00 (dawn)	40 (dim)
06:00–18:00 (day)	80 (bright)
18:00–19:00 (dusk)	40 (dim)
19:00–05:00 (night)	15 (pitch_black)

Weather Modifier

Weather	Multiplier
clear	100%
cloudy	70%
rainy	50%
stormy	30%
foggy	40%
windy	80%

Stormy and foggy weather nullify or halve the moon phase bonus.

Indoor Rooms

Indoor rooms default to `light: 0` (pitch black) unless they have light sources or open connections to lit areas.

Transparent Doors

Doors with `"transparent": true` allow light spill without being open:

- Spill multiplier: `×0.3` (vs `×0.5` for open doors)

Wind System

Wind is an area environment property (`environment.wind`) with values: `none` / `breeze` / `wind` / `gale` / `storm` / `hurricane` (task-231).

Wind Effects

Effect	Description
Heat propagation	Wind accelerates heat exchange between connected areas (1.0× to 3.0× based on strength)
Wind chill	Lowers effective temperature, resisted by item <code>wind_resistance</code> (0–100%)
Wet penalty	Wet items lose insulation, resisted by item <code>water_resistance</code> (0–100%)
Energy drain	Strong wind drains extra Energy per move in exterior areas
Extinguishing	Gale+ can extinguish lit items (10–60% chance per tick)

Items can declare:

- `"wind_resistance": 50` — resists 50% of wind chill
- `"water_resistance": 30` — resists 30% of wet insulation penalty

Humidity & Atmosphere

`environment.humidity` values: `dry` / `humid` / `wet` / `flooding` (task-232).

Humidity	effective_temp modifier	Drying speed	Stealth
dry	0	fast	0
humid	+2 (hot) / -1 (cold)	slow (2x)	-5
wet	+3 / -2	very slow (4x)	-10
flooding	+4 / -3	no drying	-15

Air propagation spreads smoke, poison gas, and other atmospheric conditions through open connections (slower than heat propagation).

Area Statuses

Areas can have a `statuses` list with dynamic effects: `on_fire` , `flooded` , `poison_gas` , `blessed` , `darkness_magic` , etc. (task-233).

Each status has:

- `type` , `severity` (1–5), `duration` (null = permanent), `source`
- `tick_effects` : temperature deltas, air changes, light changes, HP damage, condition application
- `propagation` : spread rules through open ways
- `clear_conditions` : what ends the status

Area statuses are separate from character conditions (different requirements: spatial propagation, environment mutation, no state hierarchy). They can apply character conditions via the existing `apply_condition` effect.

Trigger Integration

Time-based trigger conditions (task-234):

- `on_dawn` , `on_dusk` , `on_day` , `on_night`
- `on_full_moon` , `on_blood_moon`
- All fire once per transition using a last-fired cache

Turn-based trigger types:

- `on_turn_start` — fires at beginning of `tick_turn()`
- `on_turn_end` — fires at end of `tick_turn()`

Note: `on_tick` is an item trigger that fires during `tick_turn()` — once per turn, not once per time tick. It exists so lit items can burn down. `on_turn_start` / `on_turn_end` are the hooks for areas/characters/NPCs.

Tick Manager Module

`TickManager` (`engine/tick_manager.py`) orchestrates:

Method	Purpose
<code>__init__</code>	Injects graph, player_manager, lighting, toggleable_items, trigger_system, npc_behaviors
<code>apply_action</code>	Deducts action costs from player vitals
<code>advance_clock</code>	Increments <code>time_ticks</code> counter (pure time advance)
<code>get_current_time</code>	Returns formatted game clock string
<code>tick_turn</code>	Full turn processing: conditions → vitals decay → env effects → temperature drift → item tick triggers → NPC processing → clock advance
<code>tick</code>	Legacy wrapper — only advances clock
<code>rest</code>	Multi-tick rest sequence with NPC processing skipped

Turn Events

`GameLogger` (`engine/logging_events.py`) manages per-turn event tracking:

- `turn_number` — monotonically increasing counter
- `turn_events` — list of event dicts for the current turn
- `clear_turn_events()` — advances turn counter and clears buffer
- `get_turn_events_for_area(area_name)` — returns events in a specific room (used by NPC awareness)

Vitals Decay Over Time

Every `tick_turn()` call applies baseline vitals decay:

Vital	Decay/tick
Energy	1
Hunger	1
Thirst	1
Social	1
Hygiene	1
Bladder	— (fills +1/tick)
Sanity	1
Entertainment	1

Sleeping doesn't prevent decay (except Energy gains +3/tick while sleeping).

Environmental Vitals Effects

Applied each tick in `tick_turn()` (`tick_manager.py:246-323`):

Property	Value	Effect
air	"stale"	Energy -1/tick
air	"humid"	Social -1/tick
air	"toxic"	HP -3/tick
noise	"loud" / "chaotic" / "dripping" / "scratches"	Prevents restful sleep
smell	"mold" / "rot" / "rotting food" / "ferment" / "urine"	Hygiene -1/tick
smell	"perfume"	Social +1/tick
light	<20	Sanity -1/tick
temperature	<5°C effective	Core temp drift → Energy/HP loss → hypothermia
temperature	>35°C effective	Core temp drift → Thirst/HP loss → heat stroke

Related Tasks

- [[dev_tasks/todo/environment/task-227-environment-forecast-schedule|task-227: Forecast Schedule Engine]]
- [[dev_tasks/todo/environment/task-228-date-tracking-game-calendar|task-228: Date Tracking]]
- [[dev_tasks/todo/environment/task-229-moon-phase-system|task-229: Moon Phase]]
- [[dev_tasks/todo/environment/task-230-outdoor-lighting-time-of-day|task-230: Time-of-Day Lighting]]
- [[dev_tasks/todo/environment/task-231-wind-system|task-231: Wind System]]
- [[dev_tasks/todo/environment/task-232-humidity-atmospheric-conditions|task-232: Humidity & Atmosphere]]
- [[dev_tasks/todo/environment/task-233-area-status-system|task-233: Area Status System]]
- [[dev_tasks/todo/environment/task-234-trigger-environment-forecast-overrides|task-234: Trigger & GM Integration]]
- [[dev_tasks/review/gameplay/task-48-time-advancement_per_turn|task-48: Time advancement per turn]]
- [[dev_tasks/review/ui/task-36-inspector_game_time|task-36: Inspector game time]]
- [[dev_tasks/todo/gameplay/task-90-delayed_event_queue|task-90: Delayed event queue]]

Sound Propagation

Sound spreads through the graph each tick (speech via `engine/narration.py`, item sound sources via `engine/sound.py` + `tick_manager._process_sound_sources()`): speech penetration, door way barriers, and ambient-noise dampening.

The exact tuning values (speech penetration, open/closed/locked door barriers, noise levels) are now live **engine-config keys** — editable in Settings → **Engine Config** without code changes (`sound.*` keys, see [[UI & Settings/Engine Config]]). The BFS propagation logic itself is documented in `dev_tasks/done/environment/task-149-sound-propagation-system.md` and `task-12-sound_propagation.md`.

Per-door override: `sound_barrier` (task-329)

Any way node can set a custom `sound_barrier` property (float) in the inspector — it replaces the state-derived barrier while the way is in a solid state (`closed`, `blocked`, `locked`), so a vault door can be 4 while a thin cottage door is 0.5. Unset → per-state Engine Config defaults as before. Open (0.5), hidden (2), and `see_through` (0.75) paths are untouched; when the property is absent, `see_through` still wins over state lookup. Implemented in `engine/sound.py` `get_way_barrier()`; editable via the way inspector's  Sound Barrier field.

Character Spatial Position

Task: [[dev_tasks/review/gameplay/task-135-character-item-spatial-relationships|task-135]] (ways + items + characters) · transit: [[dev_tasks/review/gameplay/task-224-transit-areas-entry-relative-exits|task-224]]

Text worlds have no implicit geometry. VirtualWorld stores **where a character is relative to something in the room** as graph edges from the player node:

Edge type	Meaning	Example witness line
at	At/near, engaging	Jake at the north · Jake at the piano
on	Standing on	Jake on the trapdoor
under	Underneath	Jake under the chandelier
behind	Behind	Jake behind the counter

Edge type	Meaning	Example witness line
beside	Next to (people, objects)	Jake beside the woman

One anchor at a time — a new positioning action clears the previous edge. Source of truth: `engine/character_spatial.py`.

Core rule: physical action walks you there

You do **not** need a separate `examine` before `open`, `go`, `give`, etc. Any action that physically involves something in the room **steps you to it** and sets the spatial edge.

Action	Position set
<code>open / close</code>	AT the way
<code>go / crawl / climb / jump / dash</code>	AT the way (on approach + on arrival in new room)
<code>use [item] on [door/way]</code>	AT the way
<code>use [item] on [room item]</code>	at/on/under/... from item tags + phrasing
<code>use [item] on [person]</code>	beside them
<code>give / steal</code>	beside target
<code>grab (success)</code>	grappler beside target
<code>attack [person]</code>	beside target
<code>put / place X on table</code>	relation on that surface (<code>on</code> , <code>under</code> , <code>at</code> , ...)
<code>examine [way/item/person]</code>	same as above, look-only
<code>examine room / here / area name</code>	clears position (step back, survey room)
<code>look</code>	no change (glance from current spot)

Plain `use` on inventory-only items (Create Flame, eat ration) does **not** set room position.

Implementation hooks: `approach_way`, `approach_item`, `approach_character` in `character_spatial.py`; called from `engine/movement.py`, `engine/item_actions.py`, `engine/combat.py`, `engine/grapple.py`.

Ways (doors, vents, transit)

Normal rooms

- `examine door` — AT the way; if open/see-through, beyond visibility ([[dev_tasks/review/ui/task-201-area-visibility-beyond-ways[task-201]]) uses this viewpoint.
- `open north` — walk to north exit, AT it, toggle state.
- `go north` — walk to exit, AT it, traverse; arrive **AT the same way node** from the far side.

Transit areas

Tag the area `transit` or `passage` (or `transit: true` on properties). When **AT an entry way**:

- Exit labels become **back** (way you're AT) and **forward** (other connection).
- `go back / go forward` resolve via `resolve_transit_movement()`.

See [[dev_tasks/review/gameplay/task-224-transit-areas-entry-relative-exits[task-224]] for authoring notes.

Items

Default relation from item tags (`default_relation_for_item()`):

Tags	Default relation
<code>in_roof</code> , <code>on_ceiling</code> , <code>ceiling</code>	<code>under</code>
<code>in_floor</code> , <code>on_ground</code> , <code>floor</code>	<code>on</code>
(none)	<code>at</code>

Override with phrasing: `examine chandelier from below` → `under`; `stand on rug` → `on`.

Characters

- `beside` is the default relation to another person (examine, give, steal, grab, attack, use-on).
- `examine self` does not set position.

Grab + drag

When a grappler `go` s through a way, **dragged targets** are moved to the new area and set **AT the same way** on arrival (`grapple.drag_all(..., way_id)`).

Witness text + targeting

Room look and Agent Lens append a spatial suffix to each person line:

```
People here:
- the woman (awake) beside the piano – A musician in a green cloak.
- the stranger at the north – A tall figure in a dark coat.
```

Stranger labels

Unmet characters use `Player.unknown_display_name()` (`the man`, `the woman`, `the stranger`, ...) — **never the database name** until met. The spatial suffix uses the same anonymous label for character anchors when the viewer hasn't met them.

Why this helps targeting

- Room context shows handle + position together** — `"the man at the north"` tells the agent both *who* (appearance label) and *where* (exit handle).
- Parser resolution** — `NameMatching._match_character_name()` resolves `attack the tall man`, `give key to the woman`, etc. by name, substring, fuzzy name, and **description words** in the same area.
- Structured state** — each player in API state carries:
 - `at_way_id` — way node id when AT a door (legacy/convenience)
 - `spatial_position` — `{ relation, target_id, target_type, target_name }` for full position

Agents are instructed in `system-prompt.js`: physical actions walk you there; `examine room` steps back.

API / serialization

Per player in world state (`engine/serialization.py`, `engine/player_manager.py`):

```
{
  "at_way_id": "way_Lab A_vent",
  "spatial_position": {
    "relation": "beside",
    "target_id": "player_Jane",
    "target_type": "character",
    "target_name": "the woman"
  }
}
```

`spatial_position` is `null` when nowhere specific in the room.

Frontend mirror: `static/js/agent/prompt-builder/room-context.js` (`spatialPositionSuffix` on people lines).

Code map

Module	Role
<code>engine/character_spatial.py</code>	Edges, phrases, approach helpers, transit roles

Module	Role
engine/movement.py	approach_way on go/open/close; AT on arrival; drag way_id
engine/item_actions.py	examine, use-on, give, steal, put/place
engine/combat.py	attack → approach_character
engine/grapple.py	grab → beside; drag → AT way
engine/area_description.py	People lines + spatial_position_phrase()
tests/test_character_spatial.py	Ways, transit, items, characters, examine room clear
tests/test_grapple.py	Dragged target AT way

Related docs

- [[World Building/Doors & Connections|Doors & Connections]] — way graph structure
- [[AI & Narration/Agent Engine|Agent Engine]] — room context + prompts
- [[Rules Engine/Combat System|Combat System]] — attack positioning

Turn Queue & Human Turns

How turn order works, how the human turn panel knows it's someone's turn, and why the always-available command line is not a bug.

Source of truth: `static/js/agent/turn-queue.js` (state lives on `AgentEngine` as `.turnQueue` / `.currentTurnIndex` / `.turnNumber` / `.initiativeRolls`).
 Config flags come from browser settings (`config.js` , `IndexedDB`):
`config.turnBased` , `config.turnOrder` , `config.ghostMode` ,
`config.controllingPlayer` .

The queue

- `TurnQueue.initialize()` builds the queue from all non-dead players (dead included when `ghostMode`), sorted by `config.turnOrder` :
- **sequential** (default): alphabetical
- **random**: shuffled
- **initiative**: d20 + DEX modifier per character, highest first, alphabetical tiebreaker (rolls stored on `initiativeRolls`)
- Index resets to 0; `turnNumber` resets too.
- `TurnQueue.reconcile()` re-runs initialization whenever the live roster differs from the queue (character added/killed mid-run) while preserving `turnNumber` so the clock doesn't reset.

Advancing

- `TurnQueue.advance()` steps `currentTurnIndex` forward (wrapping) and sets `config.controllingPlayer` to the character whose turn it now is — this is what the human turn panel keys off.
- When the index wraps past the last actor, **one full turn cycle ends**:
`endTurn()` increments `turnNumber` and calls `ApiClient.applyTurn()` , which advances the clock and applies decay/tick effects ONCE per cycle (never inside individual commands — see AGENTS.md activity gotcha).
 World state is refetched afterward so fresh `turn_events` render before the backend clears them.
- `random` order reshuffles each cycle (`reshuffleRandom` , task-310);
`rerollInitiatives()` re-rolls d20+DEX for initiative order while keeping the current actor current.

Human turns vs the command line

Two separate input paths, both intentional:

1. **The human turn panel** (turn modal/composer) appears when the queue slot belongs to a character under human control (`config.controllingPlayer`). It plays by queue rules: compose an action, end turn, advance passes control to the next actor.
2. **The always-available command line / guest-speaker path** writes into the event stream from outside any queue slot. This is a deliberate godmode-level override for director-style input — it does NOT wait for turns and does NOT consume the typing character's queued slot unless a feature explicitly ties them together.

Don't "fix" the command line being available out-of-turn, and don't make the panel appear outside the controlled character's slot — both behaviors are the design.

Guest speech ("Speak as guest...")

The always-available command line's speech box (`speak-input` , `main.js:speakAsGuest`) posts through the **normal player-speech route with a synthetic speaker**: `POST /api/players/ [player] A Guest/speak` (`routes/player_ops.py:handle_player_speak`) → `world.broadcast_speech` (`engine/speech.py` , `speech_level="normal"` , no whisper target). No character is required and none is created.

- **Not a secret / not per-character.** It has no recipient. It is a normal-volume **area broadcast**, anchored to the active character's current area (fallback: world current area → first area). Characters in other rooms hear it only via sound propagation — "normal" penetration 1 reaches adjacent areas through **open** (barrier 0.5) or **see-through** (0.75) ways; a closed door (1.0) blocks it; ambient noise dampens it (`engine/sound.py`).
- **Effects:** each listener gets a `recent_hearing` entry (ring of 20) and +3 Social; the line lands in `game_log` (last 50) and a `speak` turn-event; the area node's `on_speech` triggers fire with `{speech, speaker}` ; simple NPCs get `on_speech_heard` . The guest isn't a player, so the speaker gets no Social gain and name-learning from the line is skipped. The speaker name is anonymized in prompts ("the stranger" / "a voice").
- **How long agents see it:** via `turn_events` — same area, current turn only, wiped when the turn cycle wraps (`/api/turn/apply` then `/api/turn/clear`); via `recent_hearing` — per listener until 20 newer hearing entries push it out (the WITNESSED block renders the last 5 speech entries, so it can persist several turns in a quiet room, and is saved in world saves but stripped from scenario exports); via `game_log` — until 50 newer lines, visible to anyone reading `/api/state` (agent prompts don't use it); the GUI event stream copy is client-side only (lost on reload).
- The human turn panel's **interject** input is a different path: it posts `say ... as the controlled character` via `/api/action` (`human-turn-composer.js:interject`), also without consuming the turn.

The scene-first panel (task-333, full redesign)

The human turn panel is the mockup's three-zone layout: scene grid (clickable area/people/things/ways chips) beside a "What happened" feed, the You strip (vitals, conditions, carrying/wearing chips, activity, "what you know"), and a composer of four always-visible rows — `do` / `say` (+volume) / `emote` / `memory`. **One turn = do + say + emote together** in a single structured payload. Hover = free look (no turn cost); click = a context menu that only FILLS the draft — nothing fires until Act (compose-then-commit, with a toggleable confirm screen). Raw JSON mode and an advanced expander (relation, reserved where-picker, confirm toggle) live under the composer.

Turn shape mirrors the agents' think→act→react (task-334): compose → Act → (a successful dash grants ONE more action slot) → deterministic

auto-memory line → react phase (say/emote/memory bound to the result — no second world interaction) → pass. A "since your turn" digest with an interject input (guest speech, doesn't consume the turn) greets the next panel open.

- Data comes from `GET /api/scene/<player> ( engine/scene_snapshot.py )`: people (stranger-masked via `unknown_display_name()`), visible items with their real `available_actions` (`TriggerSystem._get_available_actions()` contract), ways out, and your own vitals/conditions/carrying/wearing (worn reads `EDGE_EQUIPPED` edges, not the desync-prone `player.equipped` cache).
- **Hidden way state is per-player knowledge**: locks/blocked/needs-force are only reported after this character examined the way or failed to go through it (`Player.known_way_aspects` , set in movement + examine). Until then a locked door reads as plain closed — to the panel AND to the room description.
- **Names are learned by HEARING them spoken** (task-339): a name said aloud in earshot — self-intro, direct address, third-party mention — teaches it to everyone present who doesn't know it yet. Seeing someone is recognition only (stable masked label); re-looking never reveals a name. Examine masks unmet names unless the character wears a `nametag` -tagged item.
- Way rules reflected by menus: doors auto-open on go/dash unless locked, blocked, or needing force (`needs_open` = the on-go skill check); there are no lock/unlock verbs (task-336 cancelled) — unlocking rides use-on keycards.
- Darkness degrades the scene client-side when `scene.area.dark` (ambient light < 40).

Multi-human

The queue is name-based, not agent-based: any number of queue entries can be human-controlled. Each human acts only when their entry comes up; agent slots fire automatically via the autopilot interval. Future react-phase/digest work (task-334) builds directly on these semantics.

Related

- task-310 random reshuffle · task-322 R5 vital thresholds consumed per turn · task-334 react phase design · task-241 non-turn-based stepping · task-333 scene panel (mockup: `docs/design/human-turn-panel-v2-mockup.html`)

Equipment & Paperdoll System

Overview

The equipment system (`engine/equipment.py`) manages equipping/unequipping items to body slots on a 100×100px paperdoll grid, with layered stacking (multiple items per slot), two-handed weapon support, full-body suit support, and LLM-powered appearance generation.

Paperdoll Grid

Defined in `static/js/inspector/paperdoll-view.js:10-24` , the paperdoll has 13 visual areas mapped to equipment slots:

Visual Area	Slot	Label
head	head	Head
neck	neck	Neck
larm	arms	Arms
torso	torso	Torso
rarm	arms	Arms
lhand	hands	Hands

Visual Area	Slot	Label
waist	waist	Waist
rhand	hands	Hands
hand_l	hand_left	L.Held
legs	legs	Legs
hand_r	hand_right	R.Held
back	back	Back
feet	feet	Feet

Plus an additional `accessory` slot (no grid area, shown below paperdoll).

Equipment Slots

Defined in `EquipmentSystem.EQUIP_SLOTS` (`equipment.py:14-27`):

```
EQUIP_SLOTS = {
    "head":    {"max_depth": 3, "label": "Head"},
    "neck":    {"max_depth": 2, "label": "Neck"},
    "torso":   {"max_depth": 5, "label": "Torso"},
    "arms":    {"max_depth": 2, "label": "Arms"},
    "hands":   {"max_depth": 2, "label": "Hands"},
    "legs":    {"max_depth": 4, "label": "Legs"},
    "feet":    {"max_depth": 3, "label": "Feet"},
    "back":    {"max_depth": 2, "label": "Back"},
    "waist":   {"max_depth": 2, "label": "Waist"},
    "accessory": {"max_depth": None, "label": "Accessory"},
    "hand_left": {"max_depth": 1, "label": "Left Hand"},
    "hand_right": {"max_depth": 1, "label": "Right Hand"},
}
```

Stacking (Layered Slots)

Each slot has a `max_depth` limit:

- `accessory` has no limit (`None`)
- `hand_left` / `hand_right` max 1 (only one held item per hand)
- `head` max 3 (e.g., coif → hood → helmet)
- `hands` max 2 (e.g., gloves → gauntlets — worn on hands, not held)
- `torso` max 5 (e.g., undershirt → shirt → vest → coat → cloak)

Items stack from **innermost (index 0)** to **outermost (last index)** in `player.equipped` lists (`player.py:131-137`).

Worn vs Held: The Two Hand Slot Families

`hands` and `hand_left` / `hand_right` are different slots that coexist independently:

Slot	Purpose	Max Depth	Example
hands	Wearing items on hands (gloves, rings, gauntlets)	2	leather gloves + steel gauntlets
hand_left	Holding an item in the left hand	1	sword, torch, shield
hand_right	Holding an item in the right hand	1	sword, torch, shield

A character can wear gloves on `hands` AND simultaneously hold a sword in `hand_left` — they are separate equipment slots with no conflict. The paperdoll visual areas reflect this: `lhand` / `rhand` (hands slot, for wearables) and `hand_l` / `hand_r` (hand_left/hand_right slots, for held items).

Items declare which category they belong to via `equip_slots` :

- Wearable gloves: `"equip_slots": ["hands"]`
- Held weapon: `"equip_slots": ["hand_left", "hand_right"]`

Slot Whitelist

Items declare which slots they fit via `equip_slots` property (`engine/equipment.py:37`). This is a whitelist — an item can be equipped to any slot in its list. **It does not enforce simultaneous occupancy** (e.g., a ring could list both hands but only occupies one at a time).

Example from `static/js/inspector/paperdoll-view.js:157-159` :

```
const equippable = inventory.filter(n => {
  const node = worldState.getNodeByIdentifier(n);
  const slots = node?.properties?.equip_slots || [];
  return slots.includes(slot) && !equippedIds.has(node?.id || n);
});
```

equip Command

Command: equip <item_name> or wear <item_name> (also wear <item> under <other_item>)

Backend: EquipmentSystem.equip_item() (engine/equipment.py:66)

Flow:

1. Validate player state (not sleeping/unconscious/dead/bound) — line 71
2. Find item in inventory (find_item_node) — line 74
3. Verify item is carried (EDGE_CARRYING) — lines 79-85
4. Check item has equip_slots — lines 87-92
5. If slot specified, validate it's in equip_slots — lines 94-98
6. Auto-detect slot via _get_slot_for_item() — lines 99-102
7. Check slot has room (_slot_has_area() checks max_depth) — lines 104-106
8. **Two-handed check:** If item has "two_handed" tag and equipping to a hand, check other hand has room — lines 146-151
9. Add to slot stack — lines 158-170
 - If under parameter given, inserts beneath the named item in the stack
10. **Two-handed marker:** If two-handed, adds __multi_slot_<item_id> marker to other hand slot — lines 183-186
11. **Full-body markers:** If "equips_all_slots" tag, adds __multi_slot_<item_id> marker to every other declared slot — lines 188-190
12. Execute on_equip triggers — line 192
13. Rebuild equipment description via _update_equipment_description() — line 194
14. Record turn event — lines 199-204

Two-Handed Weapons

Items with "two_handed" tag in tags occupy both hand_left and hand_right. The _get_extra_hand_slot() method (equipment.py:70-74) returns the opposite hand. A marker node __multi_slot_<item_id> is added to the second hand slot (equipment.py:153). These markers are filtered out by _is_marker() (equipment.py:65-67).

Full-Body Items (equips_all_slots)

Items with the "equips_all_slots" tag occupy **every** slot they declare in equip_slots at once — e.g. the EVA Suit declares ["torso", "legs", "arms", "head", "feet", "hands"] and fills all six when equipped. The real item lands in the first declared slot; a __multi_slot_<item_id> marker is dropped into each remaining slot (same marker pattern as two_handed, via _get_extra_slots() at equipment.py:77).

Behavior:

- **Occupancy, not sealing** — each covered slot's marker consumes one stack depth, so other clothing can still layer over/under it (boots under a suit, jacket over).
- **Blocked when full** — equipping errors if any covered slot has no room left (_slot_has_area() checks max_depth).
- **Narrative reports coverage** — get_equipment_narrative() renders the suit once with every covered slot: "You are wearing: EVA Suit over your torso, legs, arms, head, feet, hands." rather than just its primary slot.
- **Unequip** works via the shared marker cleanup (_clean_multi_slot_markers) — removing the suit clears all covered slots.

unequip Command

Command: unequip <slot> or unequip <item_name> (also remove <slot> / remove <item_name>)

Backend: EquipmentSystem.unequip_item() (engine/equipment.py:230)

Supports two modes:

1. **By slot:** Pops the outermost item from the slot stack — line 254
 - If pop returns a multi-slot marker, cleans up and removes the real item
2. **By item name:** Searches all slots for the named item — lines 298-327

Cleanup (_clean_multi_slot_markers , equipment.py:240-247): Removes __multi_slot_ markers from all slots when a two-handed or full-body item is unequipped.

Visual Equipment Display

`get_visible_equipment()` (`equipment.py:333`)

Returns the **outermost** item of each slot — what other characters see at a glance. For hand slots, it shows the last real item or a "(two-handed)" / "(part of a full-body suit)" note if only a marker remains.

`get_full_equipment()` (`equipment.py:414`)

Returns **all layers** of each slot as lists of item names. Used for self-view and LLM prompts.

`get_equipment_narrative()` (`equipment.py:444`)

Generates plain-English equipment description:

- **Self-view** (`viewer_name` is `None` or matches player): Shows all layers with "over" stacking:

"You are wearing: helmet over coif on your head; steel cuirass over gambeson on your torso..."

- **Other view**: Shows only outermost visible items:

"Miki is wearing a steel helmet on their head, holding a longsword."

LLM-Powered Appearance Generation

`_update_equipment_description()` (`equipment.py:342-393`) rebuilds `player.description` whenever equipment changes:

1. Builds a prompt from `base_description` (naked physical traits) + current equipment list

2. Calls LLM with prompt asking for a vivid 3rd-person description (2-4 sentences)

3. Falls back to **code-generated text** if LLM fails, using slot labels and item names:

"Miki is wearing leather boots on their feet, steel cuirass over gambeson on their torso..."

4. Stores result in `player.description`

The prompt (`equipment.py:356-363`):

```
You are writing a visual appearance description for a character in a fantasy RPG.

BASELINE APPEARANCE (naked physical traits):
{tall, with sharp features and dark hair}

CURRENT EQUIPMENT:
- head: coif → helmet (innermost to outermost)
- torso: gambeson → steel cuirass (innermost to outermost)

Write a vivid, natural 3rd-person description of how this character looks right now...
```

Hygiene Modifier

`get_hygiene_modifier()` (`equipment.py:403-410`) returns a social penalty:

- 0 at 100 Hygiene → -5 at 0 Hygiene

- Calculated as $-(100 - \text{hygiene}) // 20$

- Range: 0 to -5 in steps of -20 Hygiene

Equipment Bonuses (`engine/equipment_bonuses.py`)

Equipped items confer **stat bonuses** based on their tags and properties. Aggregated per-player each tick and during combat by `aggregate_bonuses()` .

Tag-to-Bonus Mapping

Tag	Bonus	Item Field
--- --- --- ---		
weapon	Base damage for attacks	<code>damage</code> , <code>damage_skill</code> , <code>damage_type</code>
armor , clothing	Damage reduction (DR)	<code>defense</code>
resistance	Type-based damage resistance	<code>resistances</code> dict
insulation	Temperature shift	<code>insulation</code> : N (shifts effective temp ±N°C)

defense — Damage Reduction

Equipping any item tagged `armor` or `clothing` with a `defense` field reduces incoming weapon/unarmed damage by that amount. Multiple items stack.

```
{ "defense": 1, "tags": ["clothing"] }
```

```
# combat.py - damage after defense:
damage = max(1, rolled_damage - target_defense)
```

damage — Weapon Damage (Unified)

Items with the `weapon` tag support a **unified damage field** that accepts dice notation or flat numbers:

Field	Type	Example	Description
<code>damage</code>	int or string	"2d6+3", "1d8", 8	Dice expression or flat number for damage
<code>damage_skill</code>	string	"Athletics"	Skill whose stat (STR/DEX/etc) adds modifier to damage
<code>damage_type</code>	string	"slashing", "fire"	Damage type — enables resistance checks on target

Damage is calculated as: **dice roll (or flat) + stat modifier from skill - target defense**, then reduced by **target resistance** if a damage type is set.

```
{
  "damage": "1d6",
  "damage_skill": "Athletics",
  "damage_type": "slashing",
  "tags": ["weapon"]
}
```

If `damage` contains a dice expression ("2d6+3" , "1d8"), it's rolled. If it's a bare number (8), it's used as flat damage with `roll_dice(1, 8, mod)` . Without the `weapon` tag, unarmed combat falls back to `1d4 + STR` with no type.

insulation — Temperature Shift

Items with the `insulation` tag can have an `insulation` property. When equipped, the player's **effective ambient temperature** is shifted:

```
{ "insulation": 14, "tags": ["clothing", "insulation"] }
```

Positive insulation warms (traps body heat), negative insulation cools (wicks heat away). Values from multiple worn items **stack**. For example:

- A coat with `insulation: 14` at -12°C feels like 2°C
- At 35°C, that same coat feels like 49°C (traps heat against you)

This affects core temperature drift and area-temp vitals effects (Thirst, Energy, HP from heat/cold).

Migration from `temp_range` : The old `temp_range: [min, max]` acted as a clamp, which was unrealistic (a winter coat would cap heat at 20°C). `insulation` shifts temperature equally in both directions — cold feels warmer, hot feels hotter.

resistances — Type-Based Damage Mitigation

Items with the `resistance` tag use a `resistances` property dict to define damage type reductions:

```
{ "resistances": { "fire": 10, "cold": 5, "toxic": 999 }, "tags": ["resistance"] }
```

Applied in tick processing (e.g., toxic air damage = `max(0, 3 - toxic_resistance)`) and in combat when the weapon has a `damage_type` set.

Migration note: Per-type tags (`fire_resistant` , `cold_resistant` , `toxic_resistant` , `magic_resistant` , `vacuum_sealed`) are removed. Use the single `resistance` tag with the `resistances` property dict instead.

Aggregation

`aggregate_bonuses(player, graph)` (`engine/equipment_bonuses.py:28`) is called:

- Per tick for environmental effects (`tick_manager.py`)
- Per attack for target defense calculation (`combat.py`)
- Iterates `EDGE_EQUIPPED` edges from the player's graph node
- Returns a dict of {`defense`, `damage`, `damage_dice`, `damage_skill`, `damage_type`, `insulation`, `resistances`}

Frontend UI (Paperdoll View)

The paperdoll is rendered in `static/js/inspector/paperdoll-view.js` :

- **Grid slots:** Rendered as `<div class="paperdoll-slot">` with `data-area` attribute for positioning

- **Filled slots:** Show item name + stack badge (if inner layers exist)
- **Empty slots:** Show `—` with an "Equip +" button
- **Right-click:** Opens context menu with Inspect/Unequip/Open Container
- **Stack badge:** `+N more` badge on layered slots; click opens popup with all layers
- **Equip picker:** `showEquipPicker()` opens modal listing inventory items that fit the selected slot
- **CSS:** Dark theme via CSS custom properties, paperdoll grid layout in `static/css/style.css`

Slot Occupancy Edge Cases

Recent Fixes

- `_is_marker()` (`equipment.py:55-57`): Filters `__`-prefixed IDs (multi-slot markers) from all slot operations
- `_clean_multi_slot_markers()` (`equipment.py:159-166`): Removes orphaned markers when a two-handed item is dropped or unequipped
- `drop_item()` calls unequip logic when a carried item is equipped (`item_actions.py:394-399`)
- Slot whitelist allows an item to declare multiple possible slots but does NOT allow simultaneous occupancy of multiple slots by the same item

Commands Summary

Command	Example	Description
<code>equip <item></code>	<code>equip steel_helmet</code>	Equip item to auto-detected slot
<code>wear <item></code>	<code>wear leather_gloves</code>	Alias for equip
<code>wear <item> under <inner></code>	<code>wear vest under coat</code>	Equip beneath existing item
<code>unequip <slot></code>	<code>unequip head</code>	Remove outermost item from slot
<code>unequip <item></code>	<code>unequip steel_helmet</code>	Remove named item
<code>remove <slot/item></code>	<code>remove head</code>	Alias for unequip
<code>undress</code>	<code>undress</code>	Remove outermost layer from all body slots
<code>strip</code>	<code>strip</code>	Remove everything from all slots

Related tasks

- [[task-3-equipment_system|task-3: Equipment system]]
- [[task-87-paperdoll_icon_art|task-87: Paperdoll icon art]]
- [[dev_tasks/review/items/task-54-weapon_system|task-54: Weapon system]]
- [[dev_tasks/done/bugs/bug_2-choices-equip-slots-white-bg|bug-2: Choices equip slots white bg]]
- [[bug_6-inspector-equip-slots-white-bg 1|bug-6: Inspector equip slots white bg]]

See Also

- `engine/equipment_bonuses.py` — Aggregation module
- `data/library/items/eva_suit.json` — EVA suit example (vacuum_sealed, insulation)
- `data/library/items/heavy_fur_lined_coat.json` — Cold weather gear example
- [[Combat System]] — Defense reduction and weapon damage
- [[Temperature System]] — Effective temperature from equipment

Inventory System

How Items Move Between Area and Player

The inventory system uses graph edges to track item location. There is `no inventory` list on the `Player` object that's used for gameplay — the legacy `Player.inventory` property (`player.py:81`) is present but not consulted by any engine code for inventory operations.

Graph Edge Model

- Items in a room: `EDGE_IN` = `"in"` edge from item node → room node (`graph.py:201`)
- Items carried by player: `EDGE_CARRYING` = `"carrying"` edge from item node → player node (`graph.py:207`)

When you `take` an item, the edge is moved:

```
# Remove from room
self.graph.remove_edge(item_node_id, area_id, EDGE_IN)
# Add to player
self.graph.add_edge(Edge(source=item_node_id, target=player_id, type=EDGE_CARRYING))
```

(item_actions.py:347-349)

When you `drop` an item, the reverse happens:

```
# Remove from player
self.graph.remove_edge(item_node_id, player_id, EDGE_CARRYING)
# Add to room
self.graph.add_edge(Edge(source=item_node_id, target=area_id, type=EDGE_IN))
```

(item_actions.py:403-405)

take Command

Command: `take <item_name>` (aliases: `get`, `grab`, `snatch`, `collect`, `pickup`, `pick up`, `pick` — normalized in `action.py:67-84`)

Backend: `ItemActions.take_item()` (`engine/item_actions.py:172`)

The full flow:

1. **Ghost check:** Dead players can't take items unless ghost mode is on (`item_actions.py:177-180`)
2. **State check:** Can't take while `sleeping`, `unconscious`, or `bound` (`item_actions.py:177`)
3. **Light check:** If ambient light < 20, raise error (`item_actions.py:185-188`)
4. **Find the item** (see "Item Resolution" below)
5. **Check "take" action:** Item must have `"take"` in its `actions` property (`item_actions.py:326-328`); uses `_contextual_failure` for contextual rejection messages
6. **Optional skill check** (`item_actions.py:330-336`)
7. **Execute on_take triggers** (`item_actions.py:338`)
8. **Move edge:** Remove from room/container, add `EDGE_CARRYING` to player (`item_actions.py:340-349`)
9. **Apply action costs** (`item_actions.py:352-353`)
10. **Record turn event** (`item_actions.py:356-357`)
11. **Notify NPCs:** Calls `process_simple_npcs("on_item_taken")` (`item_actions.py:364-366`)

Ambiguous Item Handling

When multiple items share the same name, `AmbiguousItemError` is raised (`item_actions.py:7-12`). The frontend receives `{"ambiguous_items": options_list, "needs_selection": True}` and prompts the user to pick via `take <name> <number>`. This is handled in `action.py:218-226`.

Item Resolution Priority

`take_item()` searches in this order (`item_actions.py:190-321`):

1. **Exact node ID** (if `item_id` parameter provided) — line 196-199
2. **Canonical item ID** via `player_manager.item_node_id()` + `_is_item_reachable()` — lines 201-206
3. **Exact name match** in room items (`EDGE_IN` from room) — lines 209-213
4. **Fuzzy name match** via `_match_item_name()` — lines 216-221
5. **Substring match** (name contains input) — lines 240-257
6. **Inside containers in room** (`EDGE_IN` from container node) — lines 259-283
7. **Inside containers in inventory** (`EDGE_IN` from container node carried by player) — lines 285-310
8. **Not found** — error with visible items list — lines 312-320

drop Command

Command: `drop <item_name>`

Backend: `ItemActions.drop_item()` (`engine/item_actions.py:370`)

The flow:

1. Ghost check → state check
2. Find item in inventory via `EDGE_CARRYING` edges
3. **If equipped:** Remove from equipment slot first (`item_actions.py:394-399`)
4. Execute `on_drop` triggers (`item_actions.py:401`)
5. Move edge to room (`item_actions.py:403-405`)

6. Apply action costs
7. Record turn event

examine Command

Command: `examine <item_name>` (aliases: `read`, `search`, `inspect`, `check` — `action.py:62-64`)

Backend: `ItemActions.get_item_desc()` (`engine/item_actions.py:28`)

Returns a detailed description including:

- Item description text (`item_actions.py:79`)
- Locked state message if `current_state == "locked"` (`item_actions.py:81-83`)
- Skill check details (if configured) (`item_actions.py:85-95`)
- `on_examine` trigger outputs (`item_actions.py:97-99`)
- Container contents if unlocked (`item_actions.py:101-110`) — sets `hidden = False` on contained items
- Available contextual actions (`item_actions.py:112-117`)

use Command

Command: `use <item_name>` or `use <item> on <target>` (with optional `"params"` in quotes for inscribing)

Backend: `ItemActions.use_item()` (`item_actions.py:519`) and `ItemActions.use_item_on()` (`item_actions.py:617`)

See [\[\[Items Overview#Item Actions\]\]](#) for details.

Inventory UI

Backend (`action.py:230-247`)

The inventory command (`i`, `inv`, `inventory`) returns:

- Equipped items grouped by slot with `[WORN]` label
- Carried items (inventory minus equipped items)

```
def get_inventory(self, player_manager) -> List[str]:
    player_id = player_manager._player_node_id(player_manager.active_player)
    seen = set()
    items = []
    for edge in list(self.graph.get_edges_for_target(player_id, EDGE_CARRYING)):
        node = self.graph.get_node(edge.source)
        if node and node.type == "item" and node.name not in seen:
            seen.add(node.name)
            items.append(node.name)
    return items
```

(`item_actions.py:161-170`)

Frontend (`ApiClient` in `static/js/api.js`)

The `ApiClient` class fetches world state via `GET /api/state` (registered in `action.py:32`) and sends actions via `POST /api/action` (registered in `action.py:44`). Inventory display logic lives in:

- `static/js/inspector.js` — Inspector panels showing inventory
- `static/js/inspector/paperdoll-view.js` — Paperdoll + inventory context menus

Context Menu (Right-Click)

Defined in `static/js/inspector/paperdoll-view.js:213-233`:

For inventory items, right-click shows:

- **Inspect** → opens node in Inspector
- **Equip** (if `equip_slots` is non-empty) → sends `wear <name>` command
- **Open Container** (if `"container"` tag) → opens node in Inspector
- **Drop** → sends `drop <name>` command

For paperdoll slots, right-click shows:

- **Inspect** → opens outer item in Inspector
- **Open Container** (if container tag)
- **Unequip** → sends `unequip <slot>` command

Stack expansion badges on layered slots (`paperdoll-view.js:59-62`) show `+N more` for inner layers. Click opens a popup showing all layers with individual unequip buttons.

Weight System

The `weight` property is defined on items (`item.py:4` constructor param, serialized in `item.py`) but there is **no weight limit or encumbrance system**. Players can carry unlimited items. Weight is stored for UI display purposes only.

Inventory Serialization

Inventory state (which player carries which items) is persisted through the graph edges. The `WorldSerializer` (`engine/serialization.py`) serializes all graph nodes and edges, so `EDGE_CARRYING` edges are preserved on save/load.

Known issue (`dev_tasks/my-thoughts-about-virtual-world.md:86`): `_item_active_effects` (toggleable item room effects) is NOT serialized — lost on save/load. Item statuses (`player.item_statuses`) have had serialization issues.

Error Handling

- `ValueError` for game-logic errors → caught at route level → `{"error": "..."}`
- `AmbiguousItemError` (has `.options` list of `{id, name, description}`) for fuzzy item matching → frontend shows numbered selection
- Light level < 20 blocks examine, take, and use operations
- State-based blocks (`sleeping` , `unconscious` , `bound` , `dead`) prevent most actions

Related tasks

- [[dev_tasks/todo/ui/task-6-item_interaction_context_menu|task-6: Item interaction context menu]]
- [[dev_tasks/review/items/task-103-weight-volume-container-limits|task-103: Weight volume container limits]]

Item States & Toggleables

Toggleable Items Overview

Toggleable items are items that can be switched between lit/unlit states, typically light sources like candles, flashlights, and lanterns. Managed by `engine/toggleable_items.py`.

Toggleable Tag

An item is toggleable if it has `"toggleable"` in its `tags` array. Tags can be a list or comma-separated string:

```
if isinstance(tags, str):
    tags = [t.strip() for t in tags.split(",")]
if "toggleable" not in tags:
    raise ValueError(f"The {item_name} can't be toggled on or off.")
```

Activating (use command)

Command: `use <item_name>` — the `use_item()` handler in `item_actions.py:600-602` detects `"toggleable"` tag and redirects to `toggle_item_status()`.

Backend: `ToggleableItems.toggle_item_status()` (`engine/toggleable_items.py:15`)

Flow:

1. Find item node — line 19
2. Validate player — line 23
3. Check `"toggleable"` tag — lines 27-31
4. Read `current_state` from item node properties (default `"unlit"`) — line 31
5. If turning on, check `uses > 0` , decrement uses — lines 37-43
6. Flip `current_state` on item node — line 46
7. Build result message — verb depends on tags (lines 49-54):
 - `electric / synthetic` → "turn on / turn off"
 - Everything else → "light / extinguish"
8. Execute `on_toggle_on` or `on_toggle_off` triggers — lines 56-60
9. If uses hit 0, fire `on_depleted` trigger, flip state back to `"unlit"` — lines 63-67
10. Record turn event — line 69

Verb Selection by Tag

The result message adapts to the item type:

Tag	Turning on	Turning off
electric , synthetic	"You turn the flashlight on."	"You turn the flashlight off."
(default: torch, candle, etc.)	"You light the torch."	"You extinguish the torch."

Toggle triggers (`on_toggle_on` / `on_toggle_off`) should provide their own flavored messages. The base message is only shown as fallback.

uses Depletion

When `uses > 0` and the item is toggled on, `uses` decrements by 1. When `uses` reaches 0:

- The `on_depleted` trigger fires
- `current_state` is set back to `"unlit"`
- The item cannot be toggled on again

Light Contribution via Graph Scan

Light contribution uses **graph scanning** (`lighting.py:get_item_light_contribution()`), not per-player tracking. Items in the area or carried/equipped by characters are scanned for:

1. `"light_source"` tag — marks the item as light-emitting
2. `current_state == "lit"` — whether the item is currently on
3. `light_level` property — numeric (0–100) or string enum (`"dim"` , `"normal"` , etc.)

The sum of all matching items' `light_level` is added to the area's ambient light (capped at 100). This replaces the old per-player `item_statuses` + `item_active_effects` + `effect_target` system.

Benefits:

- **No per-player tracking**: all players in a room see the same light
- **No deltas on area transitions**: state lives on graph nodes
- **Free serialization**: graph nodes are serialized automatically
- **Drop/pickup works naturally**: removing an item from the area removes its light

Drain Uses Per Tick When Active

In `tick_manager.py:283-308` , the system scans carried/equipped items with `current_state == "lit"` :

```
for edge_type in (EDGE_CARRYING, EDGE_EQUIPPED):
    for edge in self.graph.get_edges_for_target(player_node_id, edge_type):
        item_node = self.graph.get_node(edge.source)
        if item_node and item_node.type == "item" and item_node.properties.get("current_state") == "lit":
            lit_items.append(item_node)

for item_node in lit_items:
    tick_outputs = self.trigger_system.execute_triggers(item_node, "on_tick")
    if item_node.properties.get("uses", -1) == 0:
        item_node.properties["current_state"] = "unlit"
        dep_outputs = self.trigger_system.execute_triggers(item_node, "on_depleted")
```

Key behaviors:

- `on_tick` triggers (typically `adjust_uses`) handle fuel drain each tick
- When `uses` reach 0, `current_state` is set to `"unlit"` and `on_depleted` fires
- No more hardcoded decrement or `item_statuses` dict

Light-Producing Items — Graph-Scan Pattern

Light-producing items are defined by tags and properties on the item node:

```
{
  "name": "hand lamp",
  "uses": 60,
  "light_level": "normal",
  "current_state": "unlit",
  "tags": ["light_source", "toggleable", "oil_lamp"],
  "triggers": [
    {"trigger_type": "on_toggle_on", "effects": [{"type": "message", "params": {"message": "You light the lamp."}}]},
    {"trigger_type": "on_toggle_off", "effects": [{"type": "message", "params": {"message": "You extinguish the lamp."}}]},
  ]
}
```

```

    {"trigger_type": "on_tick", "effects": [{"type": "adjust_uses", "params": {"node_id": "self", "delta": -1}}]},
    {"trigger_type": "on_depleted", "effects": [{"type": "message", "params": {"message": "The lamp goes out."}}]}
  ]
}

```

Items with `light_level` but no `toggleable` tag are always-on stationary sources (e.g., a desk lamp that starts lit). Items with `toggleable` can be switched on/off by the player.

Lighting and Actions

Low light blocks these actions:

- **Examine:** Blocked if light < 20 (`item_actions.py:41-43`)
- **Take:** Blocked if light < 20 (`item_actions.py:185-188`)
- **Use on target:** Blocked if light < 20 (`item_actions.py:628-632`)
- **Sanity:** Decays -1/tick when light < 20 (`tick_manager.py:213-215`)

Progressive Item Status

Items can have multiple states via the `current_state` property. This is used by the trigger system for state transitions.

State Transitions via Effects

The `handle_set_state` effect handler (`effects.py:194-239`) transitions a node's state:

```

def handle_set_state(self, params, context, item_node=None, game_state=None):
    node_id = params.get("node_id", "")
    new_state = params.get("state", "open")
    target_node = self.graph.get_node(node_id)
    old_state = target_node.properties.get("current_state", "")
    target_node.properties["current_state"] = new_state
    # Fires on_state_exit and on_state_enter triggers recursively

```

Multi-Use Items (Uses as State)

Items with `uses > 0` are effectively multi-state — each use decrements the counter until 0, then the item is consumed. The `handle_drain` effect (`effects.py:470-497`) reduces uses without consuming the item:

```

def handle_drain(self, params, context, item_node=None, game_state=None):
    amount = int(params.get("amount", 1))
    uses = item_node.properties.get("uses", -1)
    if uses > 0:
        item_node.properties["uses"] = max(0, uses - amount)

```

The `handle_adjust_uses` effect (`effects.py:420-448`) can add or remove uses:

```

def handle_adjust_uses(self, params, context, item_node=None, game_state=None):
    delta = int(params.get("delta", -1))
    target_node.properties["uses"] = max(0, current_uses + delta)

```

Locked Items

Items with `current_state == "locked"` show a `locked_message` on examine (`item_actions.py:81-83`). Container items check for locked state before allowing contents to be accessed (`item_actions.py:263-264`).

Trigger Types Related to Item States

Trigger Type	When	Context
<code>on_toggle_on</code>	Item toggled on	<code>toggleable_items.py:75</code>
<code>on_toggle_off</code>	Item toggled off	<code>toggleable_items.py:77</code>
<code>on_depleted</code>	Item uses reach 0	<code>tick_manager.py:304</code>
<code>on_tick</code>	Each tick while active	<code>tick_manager.py:293</code>
<code>on_state_exit</code>	State changes from X	<code>effects.py:221-228</code>
<code>on_state_enter</code>	State changes to Y	<code>effects.py:230-238</code>

Trigger Type	When	Context
on_use	Item used	item_actions.py:553-556
on_examine	Item examined	item_actions.py:97-99
on_take	Item taken	item_actions.py:338
on_drop	Item dropped	item_actions.py:401

Complete Toggleable Item Lifecycle (Graph-Scan Model)

1. **Item created** with "light_source" tag, light_level property, and "toggleable" tag (if player can turn it on/off)
2. **Player picks up** item — EDGE_CARRYING edge created. Light contribution follows the item's location
3. **Player toggles on** → toggle_item_status() :
 - Flips current_state to "lit" on the item node
 - Lighting scan picks it up automatically
 - Fires on_toggle_on triggers
4. **Each game tick** (tick_turn()):
 - Lighting scan re-evaluates all lit items in the area
 - on_tick triggers fire (typically adjust_uses for fuel drain)
 - When uses = 0: auto-flip to "unlit", fire on_depleted
5. **Player moves to new room:**
 - Player's EDGE_IN edge changes. Next lighting scan sees items in new room instead of old
 - No explicit sync needed — state lives on graph nodes
6. **Player toggles off** → toggle_item_status() :
 - Flips current_state to "unlit"
 - Lighting scan no longer includes this item
 - Fires on_toggle_off triggers
7. **Player drops item** — EDGE_CARRYING edge removed, item reconnects to area via EDGE_IN . Lighting scan updates automatically

Known Issues

- Some library items (candles, oil_lamp, phone) still have legacy effect_target / effect_amount fields alongside light_level — need conversion to pure graph-scan pattern
- light_level field should be present on all light_source -tagged items

Entertainment Vital & Exploration

The Entertainment vital (player.vitals["Entertainment"]) decays naturally over time but can be boosted by exploration:

- **First-time area visit:** +15 Entertainment (×1.5 if curious trait, 0 if homebody)
- **Revisiting areas** (with wanderlust trait): +3 Entertainment
- **Met new characters:** small Entertainment boost (via add_entertainment_gain)
- Tracked per-character via visited_areas: set[str] and discovered_items: set[str]

Trait-Driven Behavior Hints

Traits inject behavior-guiding text into LLM prompts via buildTraitBehaviorContext() :

Trait	Prompt text
impatient	"You are impatient — you act quickly without overthinking."
patient	"You are patient — you can tolerate waiting and rarely act impulsively."
curious	"You are curious — drawn to examine things and explore unfamiliar places."
adventurous	"You are adventurous — willing to take risks to seek new experiences."
homebody	"You are a homebody — you prefer familiar surroundings and are reluctant to leave."
wanderlust	"You have wanderlust — you feel restless staying in one place too long."

Low Entertainment Prompt Injection

When Entertainment drops below thresholds, the prompt includes drive text in the === YOUR STATE === section:

Threshold	Prompt text
< 50	"You're starting to get bored. Consider doing something new or going somewhere else."

Threshold	Prompt text
< 25	"You're bored. Routine feels stifling. You're drawn to try something different."
< 10	"You're desperate for stimulation. Staying in place any longer is unbearable."

Related tasks

- [[dev_tasks/todo/items/task-102-progressive-item-status-multi-use|task-102: Progressive item status multi-use]]
- [[dev_tasks/review/items/task-20-item_locked_state|task-20: Item locked state]]
- [[dev_tasks/review/triggers/task-47-stateful_continuous_triggers|task-47: Stateful continuous triggers]]
- [[dev_tasks/review/characters/task-136-vital-entertainment-from-exploration|task-136: Entertainment vital from exploration]]

Items Overview

Item Class (item.py)

The `Item` class (`virtual_world/item.py:2`) is a simple data class that was historically used to create item instances. In the current architecture, items are stored as `Node` objects in the `WorldGraph` (`virtual_world/graph.py:8`), and the `Item` class is used mainly for legacy compat and as a reference for what properties an item node should carry.

```
class Item:
    def __init__(self, name, description, actions, uses=-1, effect_target=None,
                 effect_stat=None, effect_amount=0, action_costs=None):
```

Item Node Properties

Items in the graph are `Node` objects with `type="item"` (`virtual_world/graph.py:11`). All item data lives in the `properties` dict:

Property	Type	Default	Description
<code>name</code>	str	—	Display name (e.g. "Brass Key", "Crisp Red Apple")
<code>description</code>	str	""	Text shown on examine
<code>actions</code>	list[str] or str	—	Comma-separated or list: <code>examine,take,use,eat</code>
<code>uses</code>	int	-1	-1 = infinite, >0 = remaining uses before consumed
<code>weight</code>	float	0.1	Item weight (UI display only, no weight limit system)
<code>hidden</code>	bool	false	If <code>true</code> , item is invisible until discovered via <code>fumble_around()</code>
<code>action_costs</code>	dict	{}	Per-action cost overrides, e.g. <code>{"use": {"time": 2, "energy": 5}}</code>
<code>skill_check</code>	dict	{}	Skill check required to interact: <code>{"skill": "Perception", "dc": 12}</code>
<code>effect_target</code>	str or null	null	<code>"player"</code> , <code>"self"</code> , <code>"room"</code> , <code>"connection"</code>
<code>effect_stat</code>	str or null	null	e.g. <code>"HP"</code> , <code>"light"</code> , <code>"smell"</code>
<code>effect_amount</code>	int or str	0	Amount or value (strings used for environment overrides like <code>"lavender"</code>)
<code>tags</code>	list[str]	[]	Tags like <code>"food"</code> , <code>"key"</code> , <code>"toggleable"</code> , <code>"weapon"</code> , <code>"container"</code> , <code>"two_handed"</code> . Planned: migrate to <code>list[dict]</code> with <code>name</code> , <code>type</code> , <code>description</code> per [[dev_tasks/done/items/task-108-tags-as-dicts-with-type-and-description
<code>equip_slots</code>	list[str]	[]	Which body slots this item can be equipped to (e.g. <code>["head"]</code> , <code>["hand_left", "hand_right"]</code>). Convention (2026-08-21): every item tagged <code>clothing</code> or <code>armor</code> must carry at least one slot, primary first (<code>_get_slot_for_item</code> uses <code>equip_slots[0]</code>); extra slots declare additional coverage (jumpsuit: <code>["torso", "arms", "legs"]</code>). All 60 wearables in the library are compliant; backfilled via <code>tools/fix_item_equipment.py</code>
<code>defense</code>	int	0	Damage reduction when equipped (requires <code>armor</code> / <code>clothing</code> tag)
<code>damage</code>	int or string	0	Weapon damage — accepts dice notation <code>"2d6+3"</code> , <code>"1d8"</code> , or a flat number <code>8</code> (requires <code>weapon</code> tag)
<code>damage_skill</code>	string	—	Skill name for damage modifier stat lookup, e.g. <code>"Athletics"</code> → STR (requires <code>weapon</code> tag)
<code>damage_type</code>	string	—	Damage type for resistance checks, e.g. <code>"slashing"</code> , <code>"fire"</code> (requires <code>weapon</code> tag)

Property	Type	Default	Description
insulation	int	0	Shifts effective ambient temp by N°C (+warm, -cool). Stacks across worn items
resistances	dict[str, int]	{}	Type-based damage resistances, e.g. {"fire": 5, "cold": 3} (requires <code>resistance</code> tag)
current_state	str	"normal"	State used by triggers: "normal", "locked", "open", "closed", "lit", etc.
library_id	str	—	Reference back to the library item ID when built from the library
triggers	list[dict]	—	Library-only; converted to <code>EDGE_TRIGGERS</code> edges when placed in world
contents	list	[]	UI-only. See "Container Items" below.

Example: Apple (`data/library/items/apple.json`)

```
{
  "name": "Crisp Red Apple",
  "description": "A crisp red apple, cool from the cellar. Looks fresh and sweet.",
  "actions": "examine,take,use,eat",
  "uses": 1,
  "weight": 0.2,
  "hidden": false,
  "tags": ["food", "fruit"],
  "triggers": [
    {
      "trigger_type": "on_use",
      "effect_type": "heal",
      "effect_params": { "amount": 5, "message": "You bite into the sweet apple." }
    }
  ]
}
```

Example: Candle (`data/library/items/candle.json`)

```
{
  "name": "candle",
  "description": "An unmarked black candle...",
  "actions": "examine,take,use",
  "uses": 10,
  "weight": 0.3,
  "effect_target": "room",
  "effect_stat": "light",
  "effect_amount": 35,
  "action_costs": { "use": { "energy": 0, "time": 1 } }
}
```

Node ID Conventions

From `virtual_world/.opencode/AGENTS.md:68-75` and `engine/node_ids.py` :

- `item_<name>` — e.g. `item_rusty_key`, `item_Brass Key`
- Area names normalized: lowercase, spaces → underscores via `_area_node_id()`
- Item IDs are generated `item_<name>` via `NodeIDHelper.item_node_id(name)` (`virtual_world_engine.py:226`)
- When `add_node()` detects a duplicate item ID, a random 8-char UUID suffix is appended (`graph.py:51-57`)

Item Placement

Items exist in the world graph through edge relationships:

In a Room

`EDGE_IN` = `"in"` edge from item node → room node (`graph.py:201`)

```
Edge(source="item_rusty_key", target="area_living_area", type="in")
```

In a Player's Inventory

`EDGE_CARRYING` = `"carrying"` edge from item node → player node (`graph.py:207`)

```
Edge(source="item_rusty_key", target="player_traveler", type="carrying")
```

Spatial Positioning (Replaces `EDGE_CONTAINS`)

Items in or on containers use spatial `EDGE_IN` , `EDGE_ON` , `EDGE_UNDER` , `EDGE_BEHIND` , `EDGE_BESIDE` , or `EDGE_AT` edges (`graph.py:201-206`). The old `EDGE_CONTAINS` and `EDGE_LOCATION` constants are deprecated — container contents are now expressed with directional positioning. The `contents` property on items is informational only (used by UI). Actual container contents must be spawned via triggers (`spawn_item` effect).

Item States

Toggleable On/Off

Items with `"toggleable"` in their `tags` can be toggled on/off via `toggle <name>` . State tracked in `player.item_statuses` (`player.py:103`), a dict of `{item_node_id: "on"|"off"}` .

Uses Remaining

The `uses` property controls consumability:

- `-1` : Infinite uses (never consumed)
- `>0` : Decrement each use. When it reaches 0, the item is removed from the graph.

Handled in `item_actions.py:502-511` , `item_actions.py:601-611` , and `tick_manager.py:287-308` .

Progressive Item Status (`current_state`)

Items can have arbitrary states like `"normal"` , `"locked"` , `"lit"` , `"open"` , `"closed"` . State transitions fire `on_state_exit` and `on_state_enter` triggers (`effects.py:220-238`).

The examine handler checks for `current_state == "locked"` and shows a `locked_message` (`item_actions.py:81-83`).

Item Actions

Every item has an `actions` property (comma-separated string or list) that defines what verbs work on it:

Action	Method	Description
take	<code>ItemActions.take_item()</code> (<code>item_actions.py:172</code>)	Pick up from room to inventory
drop	<code>ItemActions.drop_item()</code> (<code>item_actions.py:370</code>)	Drop from inventory to room
examine	<code>ItemActions.get_item_desc()</code> (<code>item_actions.py:28</code>)	Describe item/details
use	<code>ItemActions.use_item()</code> (<code>item_actions.py:519</code>)	Activate item (triggers, effects)
eat	<code>ItemActions.eat_item()</code> (<code>item_actions.py:418</code>)	Consume as food
drink	<code>ItemActions.drink_item()</code> (<code>item_actions.py:421</code>)	Consume as drink

The `take_item` flow (`item_actions.py:364-...`):

1. Ghost check → state check → light check
2. Find item in room (exact → word-boundary → alias → fuzzy → inside containers)
3. Check "take" in item actions
4. Optional skill check
5. Execute `on_take` triggers
6. Remove `EDGE_IN` edge, add `EDGE_CARRYING` edge
7. Register item discovery (+Entertainment, task-136)
8. Apply action costs
9. Record turn event

Duplicate items & selection

Multiple items can share the same display name (e.g. two Jumpsuits placed from the same library item). They are distinguished by **unique node ids** — `WorldGraph.add_node` appends a random 8-char suffix to a duplicate id (`graph.py:51-57`), and `/api/build/item-from-library` (`routes/library_routes.py`) generates unique ids up front. Display names stay identical.

When a `take` (or `use` / `drop`) resolves to several matches:

- **Identical copies** (same name, description, tags, state — e.g. two jumpsuits from the same library item) are **auto-selected**: the engine picks the first without asking (`_auto_select_identical` , `item_actions.py:345`).

- **Distinct items** still raise `AmbiguousItemError` → the UI shows a numbered "Which one?" prompt.
- `take <name> <n>` picks the n -th match; the number branch is **case-insensitive with fuzzy fallback** (`routes/action.py`), matching the resolver's behavior.

Note: placements snapshot the library entry at build time. Fix a library item's props (e.g. `equip_slots`) and propagate to already-placed nodes with `POST /api/items/<node_id>/refresh-from-library` (needs `library_id`; bug_14).

The `drop_item` flow (`item_actions.py:370-416`):

1. Ghost check → state check
2. Find item in inventory (`EDGE_CARRYING`)
3. If equipped, remove from equipment slot (`unequip`)
4. Execute `on_drop` triggers
5. Remove `EDGE_CARRYING`, add `EDGE_IN`
6. Apply action costs

The `use_item` flow (`item_actions.py:519-615`):

1. Ghost check → state check
2. Find item in inventory
3. Check "use" in actions OR presence of trigger edges
4. Execute triggers (both `on_use` and type-specific)
5. Skill check
6. Apply action costs
7. Apply effect (self/player/room)
8. Decrement uses → remove if depleted

action_costs on Items

Items can specify per-action cost overrides via `action_costs` (`item.py:9`). Format:

```
{ "use": { "time": 2, "energy": 5 } }
```

These are passed through `player_manager.apply_action()` in `tick_manager.py:14-46`, which merges them with base costs from `ACTION_COSTS` in `virtual_world_engine.py:77-86` and applies trait modifiers.

Item Matching

The `NameMatching` class (`engine/matching.py:10`) provides four-tier matching:

1. **Exact match** (case-insensitive)
2. **Word-boundary substring** — input must appear as a whole word inside a name (and vice versa). Raw substring is NOT used, so `stove` no longer matches `Stovepipe` `Leather Boots` (`Pair`). Inputs shorter than 2 chars are rejected before this tier.
3. **Alias match** — input matches an item's `aliases` property (e.g. `kindling` has aliases `["twigs", "dry twigs", "firewood"]`). Aliases survive save/load (`serialization.py`) and the items registry.
4. **Fuzzy difflib** (cutoff 0.7 for items, 0.6 for exits)

`_is_item_reachable()` (`matching.py:133-160`) checks:

- Directly in the room (`EDGE_IN` from room)
- In player inventory (`EDGE_CARRYING` from player)
- Inside an examined (unhidden) container in the room
- Inside an examined (unhidden) container in player inventory

Flavor Targets (background objects)

Objects that only exist in description text (a stove, a loose tile, the moon) are handled **narratively**, not as items. After real item/exit/character resolution fails, `examine` falls back to `_describe_flavor_target()` and `use X on Y` falls back to `_descriptive_target_failure()` (`item_actions.py:181`, :948), which scan the area + item + way descriptions for the phrase and return an in-character response ("You examine the stove, but it does not seem to be of any use."). Flavor is the **last resort**, so it can never block a genuine target.

use X on Y target parsing

`routes/action.py` takes the **full remainder after** `on` as the target — never a single token — so `use create flame on dried flower crown` targets the whole crown

instead of truncating to "dried". Quoted tokens after `on` are treated as an explicit target + optional params pair (e.g. `use quill on "letter" "some text"` → `inscribe`), tracked by `tokenize_command_detailed` (`routes/helpers.py`).

Container Items

Container items (with `"container"` tag) use spatial `EDGE_IN` / `EDGE_ON` / `EDGE_UNDER` / `EDGE_BEHIND` / `EDGE_BESIDE` / `EDGE_AT` edges. The old `EDGE_CONTAINS` constant is deprecated — container contents are now expressed with directional positioning. The `contents` property on items is UI-only — actual spawning happens via triggers using the `spawn_item` effect.

Container items are referenced in:

- `get_item_desc()` at `item_actions.py:102-110` — shows contents when examining an unlocked container
- `take_item()` at `item_actions.py:260-310` — allows taking items from inside containers

spawn_item Triggers

The `spawn_item` effect (`effects.py:143-175`) creates items in the current room:

- Looks up `params.get("item_id")` and creates a bare Node with `type="item"`
- Places via `EDGE_IN` edge
- Items spawn **without** trigger edges — for `on_use_on` behavior, rely on the legacy `locked_with` path

locked_with Legacy Path

When a key item is used on a door (`use_item_on` at `item_actions.py:693-701`):

1. Finds the door node from the matched exit direction
2. Checks for `EDGE_UNLOCKS` edges from item → door (`graph.py:172`): `Edge(source=item_id, target=way_id, type="unlocks")`
3. If found, sets door state to `"closed"`
4. Also checks `effect_target == "connection"` for direct state changes (`item_actions.py:703-709`)

Item Library (routes/library_routes.py)

The item library stores reusable item definitions in `data/library/items/*.json`. The `/api/build/item-from-library` endpoint (`routes/library_routes.py`) converts library items to world items:

1. Looks up item in library by `item_id`
2. Creates/updates a `Node(id="item_<name>", type="item")` with properties copied from library data
3. Places in the target room/container/character
4. Converts library `triggers` array to `EDGE_TRIGGERS` edges with `logic_trigger` nodes

Library items support all properties listed above plus:

- `triggers`: Array of trigger definitions with `trigger_type`, `effect_type`, `effect_params`, `target_name`, `condition`

Tags

Tags are a core query system ([dev_tasks/done/items/task-98-tags-as-core-query-system|task-98]). They classify items for engine behavior, triggers, and UI display. See the [[Library System/Tags System|Tags System]] page for the full list of mechanical tags.

Current shape: `list[str]`

Each tag is a lowercase string. All engine checks use simple `in` membership:

Tag	Used by	Effect
"toggleable"	<code>toggleable_items.py:27</code>	Grants ability to toggle on/off
"two_handed"	<code>equipment.py:62</code>	Occupies both hand slots
"equips_all_slots"	<code>equipment.py:70</code>	Occupies every slot in <code>equip_slots</code> (full-body suits)
"food"	<code>trigger_system.py:688</code> , <code>item_actions.py:444</code>	Enables <code>eat</code> action
"drink"	<code>trigger_system.py:691</code> , <code>item_actions.py:446</code>	Enables <code>drink</code> action
"openable"	<code>trigger_system.py:642</code>	Enables <code>open</code> action
"container"	<code>item_actions.py:102-110</code>	Shows contents on examine
"weapon", "armor", etc.	<code>item-library.js:109-134</code>	UI type icon and color

Tag query methods (`graph.py:94-154`)

- `get_items_by_tag(tag, area_id?)` — find items by tag, optionally in a room